

KAIROSSWAP · SECURITY REVIEW

KairosSwap Follow Up Review.

A security review of Follow Up Review.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
KairosSwap	Follow Up Review	Guardian	July 13 – August 18, 2026	August 25, 2026

§ 00

Contents.

210 pages — overview, confidence ranking, scope, 126 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 08
	Main Review · 73 findings	P. 17
	Remediation Review · 53 findings	P. 140
§ 05	Disclaimer	P. 209
§ 06	About Guardian	P. 210

§ 01

Overview.

Guardian reviewed Follow Up Review across 2 review rounds. The final report includes 16 High, 40 Medium, 31 Low, 39 Informational findings.

AUDIT FIRM	Guardian	REPOSITORY	kairosswap/protocol
CLIENT	KairosSwap	METHODOLOGY	Smart contract · Manual · Main Review · Remediation Review
PROJECT	Follow Up Review	MAIN REVIEW COMMIT	580875df571a
PREPARED BY	Guardian	REMEDATION REVIEW COMMIT	a12bdd556e4e
AUDIT WINDOW	July 13 – August 18, 2026		
FINAL REPORT	August 25, 2026		

1.1 Confidence Ranking

1 / 5

Very Low Confidence

Guardian assigns a Confidence Ranking of 1 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.



1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	16	0	10	1	5
Medium	40	0	33	1	6
Low	31	0	31	0	0
Informational	39	0	39	0	0
All severities	126	0	113	2	11

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence ASSIGNED	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Follow Up Review in `kairosswap/protocol`. The scope includes 6,750 source lines across 37 files.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope 1/2.

FILE	NSLOC	TOTAL
packages/hardhat/contracts/adapters/KairosBuyerAdapter.sol	307	772
packages/hardhat/contracts/adapters/KairosBuyerAdapterFactory.sol	51	129
packages/hardhat/contracts/adapters/KairosMarketAdapter.sol	207	796
packages/hardhat/contracts/adapters/KairosMarketAdapterFactory.sol	49	124
packages/hardhat/contracts/Admin.sol	247	392
packages/hardhat/contracts/lib/Errors.sol	82	107
packages/hardhat/contracts/lib/OraclePokeLib.sol	44	111
packages/hardhat/contracts/lib/RateIndexLib.sol	207	442
packages/hardhat/contracts/lib/SwapEvents.sol	126	185
packages/hardhat/contracts/lib/SwapFormulas.sol	287	780
packages/hardhat/contracts/lib/Utils.sol	1,073	2,158
packages/hardhat/contracts/lib/Views.sol	487	729
packages/hardhat/contracts/lib/VirtualShareLib.sol	245	348
packages/hardhat/contracts/oracles/adapters/ChainlinkAdapter.sol	71	93
packages/hardhat/contracts/oracles/adapters/PythAdapter.sol	76	200
packages/hardhat/contracts/oracles/BaseRateAverageIndex/AaveBaseBorrowRateMinimum.sol	27	35
packages/hardhat/contracts/oracles/BaseRateAverageIndex/IndexBaseRateV1.sol	245	590
packages/hardhat/contracts/oracles/BaseRateAverageIndex/MorphoBaseRateV1.sol	280	593
packages/hardhat/contracts/oracles/BaseRateAverageIndex/RawMorphoBaseRateV1.sol	35	69
packages/hardhat/contracts/oracles/BaseTimelockedOracle.sol	635	1,029
packages/hardhat/contracts/oracles/cumulativeIndex/AbstractCumulativeIndex.sol	19	39
packages/hardhat/contracts/oracles/cumulativeIndex/AbstractCumulativeIndexMorpho.sol	42	62
packages/hardhat/contracts/oracles/cumulativeIndex/CumulativeIndexBorrowAave.sol	25	37
packages/hardhat/contracts/oracles/cumulativeIndex/CumulativeIndexBorrowCompound.sol	55	80
packages/hardhat/contracts/oracles/cumulativeIndex/CumulativeIndexBorrowMorpho.sol	38	73
packages/hardhat/contracts/oracles/cumulativeIndex/CumulativeIndexSupplyAave.sol	30	49
packages/hardhat/contracts/oracles/cumulativeIndex/CumulativeIndexSupplyCompound.sol	55	80
packages/hardhat/contracts/oracles/cumulativeIndex/CumulativeIndexSupplyMorpho.sol	46	92
packages/hardhat/contracts/oracles/lambda/FixedRatePricingOracle.sol	105	186
packages/hardhat/contracts/oracles/RiskRateOracle.sol	139	244
packages/hardhat/contracts/oracles/wrappers/BaseUpgradableOracleWrapper.sol	187	426
packages/hardhat/contracts/oracles/wrappers/UpgradableCumulativeRateOracle.sol	24	48
packages/hardhat/contracts/oracles/wrappers/UpgradableRateOracle.sol	24	49
packages/hardhat/contracts/oracles/wrappers/UpgradableSidedPremiumOracle.sol	48	79
packages/hardhat/contracts/oracles/wrappers/UpgradableTenorRateOracle.sol	109	239
packages/hardhat/contracts/SwapCore.sol	775	1,650

Files in Scope 2/2.

FILE	NSLOC	TOTAL
packages/hardhat/contracts/SwapPositionWrapper.sol	248	447

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
H-01	JIT LPs capture forfeited projected gains	Unexpected Behavior	HIGH	ACKNOWLEDGED	p. 17
H-02	Independent rings invert paired base quotes	Unexpected Behavior	HIGH	ACKNOWLEDGED	p. 19
H-03	LP share mutations can sandwich liquidation	Unexpected Behavior	HIGH	ACKNOWLEDGED	p. 21
H-04	Unsealed maturity indices change payouts	Unexpected Behavior	HIGH	ACKNOWLEDGED	p. 23
H-05	Bucket PnL caps after netting swaps	Math	HIGH	ACKNOWLEDGED	p. 25
H-06	Paired collateral caps let hedged traders profit	Unexpected Behavior	HIGH	ACKNOWLEDGED	p. 28
H-07	Morpho adjustment speed reverses rate forecast	Math	HIGH	ACKNOWLEDGED	p. 30
H-08	BUY_FLOATING early exits omit compounding	Math	HIGH	ACKNOWLEDGED	p. 32
H-09	Buyer NAV ignores executable liquidation value	MEV	HIGH	ACKNOWLEDGED	p. 35
M-01	Stale Morpho target underquotes BUY_FIXED swaps	Math	MEDIUM	ACKNOWLEDGED	p. 38
M-02	Cumulative oracle upgrades reprice live swaps	Validation	MEDIUM	ACKNOWLEDGED	p. 40
M-03	Just-in-time LPs capture accelerated swap fees	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 42

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-04	Opposing swap masks the liquidation guard	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 44
M-05	Morpho target conversion underquotes growth	Math	MEDIUM	ACKNOWLEDGED	p. 46
M-06	Conservative NAV lets entrants capture gains	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 49
M-07	Oracle upgrades enable a paired-swap LP drain	Upgradeability	MEDIUM	ACKNOWLEDGED	p. 51
M-08	Dead shares drain fresh LP refills	Validation	MEDIUM	ACKNOWLEDGED	p. 55
M-09	Uncapped oracle call blocks emergency fallback	DoS	MEDIUM	ACKNOWLEDGED	p. 56
M-10	Temporary liquidity underprices paired swaps	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 59
M-11	Morpho APY blend overquotes BUY_FLOATING	Math	MEDIUM	ACKNOWLEDGED	p. 61
M-12	Dust deposit consumes utilization bootstrap	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 64
M-13	Queued tenor upgrade lets LPs capture new mark	MEV	MEDIUM	ACKNOWLEDGED	p. 66
M-14	Bucket averages break compounded accrued NAV	Math	MEDIUM	ACKNOWLEDGED	p. 68
M-15	BUY_FLOATING averages invent accrued losses	Math	MEDIUM	ACKNOWLEDGED	p. 70
M-16	Accrued-only buckets ignore fee-time covariance	Math	MEDIUM	ACKNOWLEDGED	p. 72
M-17	Live Aave floor cuts lock paired swap profit	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 74
M-18	Morpho supply index can be clamped	Oracle	MEDIUM	ACKNOWLEDGED	p. 76
M-19	Realized pool gains are retroactively seasoned	Logical Error	MEDIUM	ACKNOWLEDGED	p. 78

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-20	Top-ups can shorten active LP profit vests	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 80
M-21	Direct settlement blocks adapter recovery	DoS	MEDIUM	ACKNOWLEDGED	p. 82
L-01	Market creation accepts wrong-tenor risk premium	Validation	LOW	ACKNOWLEDGED	p. 84
L-02	Escrow cannot bypass a blocked recipient	DoS	LOW	ACKNOWLEDGED	p. 85
L-03	Aave upgrade can make index flash-controlled	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 87
L-04	Bucket tenor averaging misprices LP shares	Math	LOW	ACKNOWLEDGED	p. 89
L-05	Unpriceable swap blocks unrelated cash recovery	Warning	LOW	ACKNOWLEDGED	p. 91
L-06	Stale market creation spends unbounded live fee	Validation	LOW	ACKNOWLEDGED	p. 93
L-07	Force-deallocation rounding leaks to co-LPs	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 95
L-08	Wrapper activation accepts unusable rates	Validation	LOW	ACKNOWLEDGED	p. 97
L-09	Protocol fee event overstates revenue	Events	LOW	ACKNOWLEDGED	p. 99
L-10	Morpho index can change unexpectedly	Oracle	LOW	ACKNOWLEDGED	p. 100
L-11	Extreme tenor gaps break interpolation	DoS	LOW	ACKNOWLEDGED	p. 102
L-12	High-price mints enrich incumbent LPs	Rounding	LOW	ACKNOWLEDGED	p. 104
L-13	Core operators lose rights after wrapping	Access Control	LOW	ACKNOWLEDGED	p. 106
L-14	Rounded index ratio can brick settlement	DoS	LOW	ACKNOWLEDGED	p. 107

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-15	Settled event can emit before credit	Events	LOW	ACKNOWLEDGED	p. 109
L-16	Floating NFTs show zero rate	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 110
I-01	Natspec for minCollateral is incomplete	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 111
I-01.2	Short TWAR windows can misprice long swaps	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 112
I-02	Lib ignore pattern hides source folders	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 113
I-03	Unused owner on SwapCore	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 114
I-04	Zero owner transfer bypasses cancel event	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 115
I-05	Pending fee updates cleared silently	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 116
I-06	The admin variable can be marked immutable	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 117
I-07	Creation fee delivery can fail open	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 118
I-08	Creation fee foregone event is incomplete	Events	INFORMATIONAL	ACKNOWLEDGED	p. 119
I-09	Market ID omits two config flags	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 120
I-10	Global nonce comment is stale	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 121
I-11	No-op bucket updates bypass rate checks	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 122
I-12	First snapshot equality is not short-circuited	Gas Optimization	INFORMATIONAL	ACKNOWLEDGED	p. 123
I-13	Aave supply closeouts assume reserve activity	Math	INFORMATIONAL	ACKNOWLEDGED	p. 124
I-14	Misbehaving Aave data provider note	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 126

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-15	Risk premium invalidity uses different error	Error	INFORMATIONAL	ACKNOWLEDGED	p. 127
I-16	Redundant rate duration in settlement	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 128
I-17	Liquidation lacks maintenance buffer	Math	INFORMATIONAL	ACKNOWLEDGED	p. 129
I-18	Reuse cached market id	Gas Optimization	INFORMATIONAL	ACKNOWLEDGED	p. 130
I-19	Document live gap availability risk	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 131
I-20	Comment misstates early-exit duplicate handling	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 132
I-21	Renunciation can freeze a paused oracle	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 133
I-22	Morpho adapters misreport index decimals	Warning	INFORMATIONAL	ACKNOWLEDGED	p. 135
I-23	Unused credit helper remains	Superfluous Code	INFORMATIONAL	ACKNOWLEDGED	p. 136
I-24	Factory NatSpec is imprecise	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 137
I-25	Adapter list can be spammed	DoS	INFORMATIONAL	ACKNOWLEDGED	p. 138
I-26	Supply rounding comment is misleading	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 139

REMEDIATION REVIEW

H-01	Force-deallocation forfeits settlement reserves	Unexpected Behavior	HIGH	PARTIALLY RESOLVED	p. 140
H-02	Force-deallocation transfers value to co-LPs	Math	HIGH	RESOLVED	p. 142
H-03	Late vault deposits capture settlement reserves	Unexpected Behavior	HIGH	ACKNOWLEDGED	p. 144
H-04	Failed base quotes bypass liquidation reserves	Unexpected Behavior	HIGH	RESOLVED	p. 146

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
H-05	Accrued minting exposes projected exit gains	Logical Error	HIGH	RESOLVED	p. 148
H-06	Mixed maturities misprice forced-exit vesting	Math	HIGH	RESOLVED	p. 150
H-07	Delayed risk report pairs with newer base rate	Oracle	HIGH	RESOLVED	p. 152
M-01	Five-axis bucket pricing can exceed the gas cap	DoS	MEDIUM	RESOLVED	p. 153
M-02	Zero cap allocation blocks recovery reporting	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 155
M-03	Raw cumulative exits can skip compounding	Math	MEDIUM	ACKNOWLEDGED	p. 157
M-04	Departed premium extrema transfer LP value	Unexpected Behavior	MEDIUM	RESOLVED	p. 158
M-05	Transient Morpho floor forces stale settlement	Unexpected Behavior	MEDIUM	ACKNOWLEDGED	p. 160
M-06	Repeated pricing scans lock LP deallocation	DoS	MEDIUM	ACKNOWLEDGED	p. 162
M-07	Paired collateral omission underprices LP burns	Math	MEDIUM	RESOLVED	p. 164
M-08	Tiny exits can prolong vesting	DoS	MEDIUM	ACKNOWLEDGED	p. 166
M-09	Stale minimum notional depresses burns	Business logic errors	MEDIUM	RESOLVED	p. 167
M-10	Dust swap poisons its bucket's LP burn price	Logical Error	MEDIUM	PARTIALLY RESOLVED	p. 168
M-11	Bearer update permits obsolete risk premium	Frontrunning	MEDIUM	ACKNOWLEDGED	p. 170
M-12	Committee changes alter prior report authority	Signatures	MEDIUM	ACKNOWLEDGED	p. 171
M-13	Early-exit fee can overflow before payout cap	DoS	MEDIUM	RESOLVED	p. 173

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
M-14	Dust top-up resets seasoned shares' anchor	Validation	MEDIUM	ACKNOWLEDGED	p. 174
M-15	Per-swap clipping makes exit order matter	Logical Error	MEDIUM	ACKNOWLEDGED	p. 175
M-16	Self-hedged Vault misreports internal payments	Configuration	MEDIUM	ACKNOWLEDGED	p. 176
M-17	Settlement order poisons the utilization EMA	Logical Error	MEDIUM	ACKNOWLEDGED	p. 177
M-18	Unpoked base quote overstates BuyerAdapter NAV	Oracle	MEDIUM	ACKNOWLEDGED	p. 178
M-19	Arithmetic index interpolation underquotes TWAR	Oracle	MEDIUM	RESOLVED	p. 179
L-01	Market adapter omits idle balances	Logical Error	LOW	ACKNOWLEDGED	p. 180
L-02	Zero-duration marks omit growth	Logical Error	LOW	ACKNOWLEDGED	p. 181
L-03	Dust buckets hide forced-exit liquidation risk	Logical Error	LOW	ACKNOWLEDGED	p. 182
L-04	Floored time moments create phantom margin	Rounding	LOW	ACKNOWLEDGED	p. 183
L-05	Forced-exit vesting ignores LP burn bounds	Logical Error	LOW	ACKNOWLEDGED	p. 184
L-06	Member dust margin makes splits nonadditive	Rounding	LOW	ACKNOWLEDGED	p. 185
L-07	Negative premiums make new markets unusable	Validation	LOW	ACKNOWLEDGED	p. 186
L-08	Payment flooring makes split swaps nonadditive	Rounding	LOW	ACKNOWLEDGED	p. 187
L-09	Vesting floor lets partial burns consume reserve	Rounding	LOW	ACKNOWLEDGED	p. 188
L-10	Sole-owner adapter cannot recover wiped pool	DoS	LOW	ACKNOWLEDGED	p. 189

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-11	Pool-sized dust routing taxes LP withdrawals	Logical Error	LOW	ACKNOWLEDGED	p. 190
L-12	Stale disable call erases newer fee epochs	Business logic errors	LOW	ACKNOWLEDGED	p. 191
L-13	Tenor rounding can invert fixed and raw quotes	Rounding	LOW	ACKNOWLEDGED	p. 192
L-14	Utilization seasoning depends on call cadence	Math	LOW	ACKNOWLEDGED	p. 193
L-15	Aave checkpoints change supply-index growth	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 194
I-01	Short TWAR windows misprice long swaps	Oracle	INFORMATIONAL	ACKNOWLEDGED	p. 196
I-02	Batch oracle comment overstates guard	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 197
I-03	Require fresh data after L2 recovery	Oracle	INFORMATIONAL	ACKNOWLEDGED	p. 198
I-04	Wrapper activation can commit downstream cycles	Validation	INFORMATIONAL	ACKNOWLEDGED	p. 199
I-05	Bucket removals mask invariant failures	Best Practices	INFORMATIONAL	ACKNOWLEDGED	p. 201
I-06	Undocumented bucket convention parameter	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 202
I-07	Expired rates veto strict bounds activation	Oracle	INFORMATIONAL	ACKNOWLEDGED	p. 203
I-08	Callback settles swap before wrapper event	Events	INFORMATIONAL	ACKNOWLEDGED	p. 204
I-09	Aave adapters accept uncommittable projections	Compatibility	INFORMATIONAL	ACKNOWLEDGED	p. 205
I-10	Dust deposit captures sole LP's vesting reserve	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 206
I-11	Seasoned sibling captures fresh LP profit	Business logic errors	INFORMATIONAL	ACKNOWLEDGED	p. 207

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
I-12	Reserve clipping erases later settlement jumps	Logical Error	INFORMATIONAL	ACKNOWLEDGED	p. 208

HIGH **H-01**

ACKNOWLEDGED

JIT LPs capture forfeited projected gains

CATEGORY Unexpected Behavior
LOCATION `contracts/lib/Utils.sol:801-822`
REVIEW Main Review

DESCRIPTION

`calculateBucketUnrealizedPnL` includes the remaining projected fixed and floating payments when it values active LP shares. A buyer-favorable remaining payment therefore reduces the mint price, down to the swap's pool-backing cap. `calculateEarlyExitSettlement` applies a different rule: when the same remaining payment favors the buyer, lines 814-817 discard it and settle only the accrued payments. The discarded liability immediately disappears from LP NAV when the swap closes.

`supplyCollateral` remains open immediately before this state change and mints shares at the depressed active-swap price. There is no LP exposure epoch or settlement checkpoint. Consequently, an observer can front-run a public `exitSwapEarly` transaction, acquire a large fraction of the pool for little capital, let the buyer forfeit the projected gain, then withdraw the liability reversal after `lpProfitVestSeconds`. The vest delays withdrawal but does not reduce the permanent gain.

The native PoC uses a 180-day `BUY_FIXED` cumulative market, 18x leverage, a 1,000,000 USDC incumbent pool and a 725,000,000 USDC swap within the reported 730,000,000 USDC capacity. After half the term, a legitimate one-percentage-point projection change from 5% to 6% makes the remaining leg buyer-favorable. A 100,000 USDC front-running deposit mints at a 0.006849315068 share price. Early exit raises the share price to 0.067023689814. After the default 12-hour vest, the observer withdraws 978,545.871359 USDC. Its 878,545.871359 USDC profit exactly equals the incumbent LP's loss against the no-deposit control.

The attacker needs a visible pending early exit, temporary capital and enough remaining swap backing for the projected liability to depress NAV. It does not need to own the swap, manipulate an oracle or hold a privileged role. The tested extraction removes 87.85% of the incumbent pool with one transaction ordering, which supports High severity despite the 12-hour exposure to new swaps.

RECOMMENDATION

Do not release a forfeited remaining-payment liability directly into ordinary LP NAV. When early exit discards a buyer-favorable projected payment, place the resulting NAV reversal

H-01 JIT LPs capture forfeited projected gains

CONTINUED

in a separate reserve and vest it over the canceled remaining term. This keeps the share price continuous at exit and requires fresh capital to remain exposed for the period whose value it wants to earn.

Alternatively, introduce LP exposure epochs. Shares supplied after a swap was created must not receive that swap's settlement gains unless they also assume a proportional part of its exposure under an explicit transfer mechanism.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/JitForfeitedProjectionCapture.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1018>

HIGH **H-02**

ACKNOWLEDGED

Independent rings invert paired base quotes

CATEGORY Unexpected Behavior
LOCATION contracts/lib/Utils.sol:211-242
REVIEW Main Review

DESCRIPTION

`probeOraclesForMarketCreation` checks each base oracle in isolation. It requires the cumulative `BUY_FIXED` oracle to advertise a compounded quote and the cumulative `BUY_FLOATING` oracle to advertise a raw quote, but it does not prove that both rates came from the same observation history. The paired markets can therefore use two `IndexBaseRateV1` instances that share a cumulative source while maintaining different permissionless observation rings.

`SwapCore.buySwap` later reads only the selected market's base oracle. A sparse fixed ring can interpolate the one-day boundary from an older observation and the live source, while the raw ring uses a fresh observation exactly at that boundary. In the validated Aave supply seed, the 30-day bumped fixed quote is `50099441850158283` WAD and the raw floating quote is `59999999999999045` WAD. Thus, the compounded quote is lower than the raw quote despite both reads being valid.

This requires distinct fixed and raw deployments over the same source. Their permissionless poke histories must diverge so the higher-rate boundary is stored only in the raw ring. The attacker can create that state by poking only raw, although an honest keeper that also pokes fixed at the boundary prevents it. Market-creation pokes and a later public `updateMarketRateIndex` call do not reconstruct a boundary that fixed already missed.

With two-sided liquidity, enough collateral headroom and an inversion larger than both markets' markups and protocol fees, an unprivileged trader can buy equal-notional opposite sides at the same timestamp and shared reference index. The realized floating base payments cancel, leaving the LP pools to fund the quote difference. A deterministic 10,000,000 USDC-per-side test produced a gross LP loss of `6493609439` atomic USDC, paid `3287671232` atomic USDC in protocol fees and left the trader with `3205938207` atomic USDC profit. Neither LP backing nor buyer collateral capped the settlement. Larger notionals increase the transfer until one of those caps changes the payoff.

The loss can become material under the scoped Aave V3 USDC borrow parameters. A second exact test uses the configured 12-hour TWAR and spacing, 3.5x leverage, one-day term, 1% utilization slope, 85% kink, 20% maximum kink fee, signed 50 bp risk premiums and a 20 bp

H-02 Independent rings invert paired base quotes

CONTINUED

protocol fee. A 5% interval followed by a 20% interval makes sparse fixed quote `124996146087726310` WAD while fresh raw quotes `199972607742762750` WAD. At 80% of floating-market capacity, below the kink, the uncapped paired claim is `818778889122` atomic USDC and the backing cap pays `800000000000`. Consequently, a 1,000,000 USDC floating pool falls to 200,000 USDC while the trader retains `743992329118` atomic USDC after both protocol fees. This higher-impact case still requires fixed to miss the rate-change boundary and no honest keeper to synchronize it before entry.

Example:

1. Both base models begin from the same seeded cumulative index.
2. During a rising-rate interval, a public caller pokes only the raw model at the one-day trailing boundary.
3. One day later, the raw model uses that exact boundary while the sparse fixed model interpolates its boundary from the older left anchor and current live index. Both quotes remain valid, but fixed bumped base is below floating raw base.
4. The trader opens equal-notional `BUY_FIXED` and `BUY_FLOATING` positions with the same reference index and timestamp. A third party may update both market rate indices between calls without removing the historical divergence.
5. At maturity, the shared floating base payments cancel across the pair. The residual base inversion exceeds markups and fees, transferring aggregate value from the two LP pools to the trader.

— RECOMMENDATION

Make the paired fixed and raw rates derive from one shared observation state. For example, use one ring that returns both the raw rate and its compounded transform together with the same snapshot identifier. `createMarket` should reject base-oracle pairs that cannot prove this relationship. As an immediate defense, every new entry can also read both corresponding base quotes and reject the pair when the fixed quote is below the floating quote, but that check does not replace shared history when positions can be opened at different times.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_POCs/CAND_001_HighSeverityEnvelope.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1053>

HIGH **H-03**

ACKNOWLEDGED

LP share mutations can sandwich liquidation

CATEGORY Unexpected Behavior
LOCATION contracts/SwapCore.sol:553-610
REVIEW Main Review

DESCRIPTION

`supplyCollateral()` and `withdrawCollateral()` price shares with `getPoolSharePrice()`, which includes accrued P&L and the projected remaining payments of every active bucket. Liquidation uses a different basis: `Utils.isSwapLiquidatable()` compares each swap's accrued-only payment obligation with its posted collateral or LP backing. Once that threshold is met, `Utils.liquidateSwap()` realizes the whole collateral boundary rather than the fair projected amount. Buyer liquidation credits all buyer collateral to the pool, while pool liquidation removes all LP backing and splits it between the buyer and liquidator.

Neither direct LP function checks whether an active swap is already liquidatable. A caller can therefore mint or burn shares immediately before a permissionless liquidation at a price that omits the liquidation's executable collateral transition. The same economic state produces materially different ownership outcomes depending only on transaction ordering.

The native PoC reproduces both directions with cloned states and an unrelated liquidator:

- In a cumulative `BUY_FLOATING` market, 52% of a 365-day term has elapsed. Fixed and floating payments are exactly 52 and 102 USDC, so the buyer owes its full 50 USDC collateral and can already be liquidated. The projected remaining quote is zero, leaving fair LP P&L near +2 USDC and a 1.002 share price. Supplying 1,000 USDC before liquidation and withdrawing after the 12-hour profit vest returns 1,023.976023 USDC. Supplying after the identical liquidation returns 999.999999 USDC. The 23.976024 USDC difference exactly reduces the incumbent LP's claim.
- In a cumulative `BUY_FIXED` market, the pool's accrued debt is exactly 47.5 USDC, equal to 50 USDC backing minus the 2.5 USDC liquidation incentive. Projected remaining economics make fair LP P&L zero and the share price 1.00. Burning 500 of 1,000 vested shares before liquidation returns 500 USDC. If liquidation executes first, those shares return 475 USDC. The attacker avoids 25 USDC of loss and the incumbent LP bears the same additional 25 USDC loss.

The liquidator receives the same 2.5 USDC reward in both orderings and the buyer receives the same amount in each paired execution. Both comparisons hold time, oracle index, rate exposure, swap terms and final settlement constant; the only difference is whether the

H-03 LP share mutations can sandwich liquidation

CONTINUED

direct LP share mutation precedes liquidation. The profit vest temporarily caps the first scenario's immediate withdrawal at the deposit anchor, but it expires after 12 hours and does not return the captured entitlement to incumbent shares. Both swaps are still active at liquidation, so the expired-unsettled guard does not apply.

The exact threshold tests also establish that liquidation reverts one token atom below the boundary and succeeds at equality and one atom above it for both buyer-side and pool-side liquidation. Existing `KairosMarketAdapter` tests show that its permissionless `forceDeallocate()` guard can block some liquidation gaps, but trusted allocator calls remain exempt and direct `SwapCore` LP calls have no equivalent check.

The transfer scales with the liquidatable collateral or backing and with the fraction of pool shares minted or burned around liquidation. Exploitation requires a visible liquidatable position, temporary LP capital for the mint direction or an existing vested LP position for the burn direction, correct ordering, and—in the mint direction—waiting through the profit vest while exposed to later pool activity.

— RECOMMENDATION

Use one liquidation-aware executable NAV for direct LP mints and burns. Before changing shares, either process all currently executable liquidations or value each independently liquidatable swap at the state that its immediate liquidation would leave. The mint price must not be lower than the post-liquidation value when the buyer is liquidatable and the burn price must not be higher than the post-liquidation value when the pool is liquidatable.

If evaluating every swap during an LP operation is too expensive, maintain conservative per-market metadata as swaps and indices change or fail closed by disabling direct share mutations while any active position is liquidatable. Apply the protection in `SwapCore`; an adapter-only guard cannot protect direct callers or trusted adapter routes. Keep the profit vest as defense in depth, but do not rely on it because a finite delay does not repair the permanent ownership transfer.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/CAND_002_LiquidationSharePriceOrdering.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1018>

HIGH **H-04**

ACKNOWLEDGED

Unsealed maturity indices change payouts

CATEGORY Unexpected Behavior
LOCATION contracts/SwapCore.sol:1489-1500
REVIEW Main Review

DESCRIPTION

Normal settlement calls `updateMarketRateIndex` before resolving the index at `actualExpiry`. If the first upper snapshot is created long after expiry, `RateIndexLib.getIndexAt` linearly interpolates between the last pre-expiry point and that late point. Neither `_settleSwaps` nor `Utils.resolveExpiryIndex` limits the upper point's distance from expiry. Consequently, post-expiry source movement is partly assigned to the contractual term.

The same expiry is also not final after dead-oracle fallback. `resolveExpiryIndex` returns an extrapolated value to the current settlement and emits `ExpiryIndexExtrapolated`, but it does not store that value by reference oracle and expiry. If the oracle later recovers, another open position with the same entry timestamp and term uses the newly appended history instead.

The deterministic PoC uses a 30-day Cumulative market, a 100,000,000 USDC notional, 1x leverage, zero fees and exactly 1,000,000 USDC of buyer collateral and LP backing. The source is 1.01 RAY at maturity. Settling then resolves 1.01 RAY and produces zero net payment. If no `SwapCore` checkpoint occurs until day 60, when the same source history reaches 1.04 RAY, settlement resolves the day-30 index to exactly 1.02 RAY and transfers the full 1,000,000 USDC cap. `BUY_FIXED` charges the LP cap while `BUY_FLOATING` charges the buyer cap.

The fallback proof opens two positions in one block so they have identical entry timestamps, entry indices, terms, rates, collateral and expiry. The oracle stays unavailable through expiry plus grace. Position A settles at the 1.00-RAY fallback. The oracle recovers at day 60 with a 1.04-RAY live index, after which position B resolves the shared day-30 expiry to 1.02 RAY. The two positions therefore use different maturity indices and opposite payment directions solely because one settled before recovery.

Any account can call the update and settlement functions. A party that benefits from later index history can therefore wait or race settlement when no keeper sealed the maturity. The discrepancy can consume all collateral reserved for one position.

H-04 Unsealed maturity indices change payouts

CONTINUED

— RECOMMENDATION

Store one final resolved index for each `(referenceRateOracle, expiry)` and return it for every later settlement with that key. Write the value before settling the first position so fallback, recovery and multiple market directions cannot produce different results.

Do not accept an arbitrarily late upper interpolation point when the key is still unresolved. Require the upper snapshot to fall within a short maximum delay after expiry. If that bound elapses without a qualifying point, derive a deterministic value using only history available before the bound and commit it. Emit the committed index and add regression tests for delayed checkpoints, fallback followed by recovery, both market directions and multiple same-expiry positions.

Kairo must also operate reliable maturity keepers that checkpoint every active market at or immediately after each contractual expiry. The keepers should retry failed updates and alert operators when an expiry is not sealed within the permitted delay. This is necessary operational protection, but it does not replace the on-chain index commitment because permissionless settlement alone gives unrelated callers no direct reward for paying the gas.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test/audit/CAND_004_UnsealedMaturityIndex.test.ts

— RESOLUTION

Status: Acknowledged.

HIGH **H-05**

ACKNOWLEDGED

Bucket PnL caps after netting swaps

CATEGORY Math
LOCATION contracts/lib/SwapFormulas.sol:867-873
REVIEW Main Review

DESCRIPTION

`SwapFormulas.calculateBucketUnrealizedPnL` first values all active swaps represented by a bucket as one weighted position. It then caps the resulting signed PnL once against `bucket.totalBuyerCollateral` or `bucket.totalPoolBacking`. Individual swap settlement and liquidation instead cap each position against that swap's own `collateralBalance` or `poolCollateralBacking`. Because clamping is nonlinear, opposite-sign swap PnLs can cancel inside the bucket before either aggregate cap is reached even though one underlying position has already exhausted its individual collateral bound. The bucket's aggregate fields may remain exact sums, but they cannot reconstruct the required sum of individually capped position values.

`Utils.calculateTotalUnrealizedPnL` consumes this aggregate result when `Utils.calculateLpSharePrice` prices LP shares. `SwapCore.supplyCollateral` and `SwapCore.withdrawCollateral` therefore mint or burn shares against a value that may differ materially from the amount the active positions can actually settle for. The expired-position virtual-settlement path does not prevent this discrepancy because it persists while every affected position is active.

The deterministic production-path test creates two same-block cumulative `BUY_FIXED` swaps in a pool containing 1,000 USDC and 1,000 shares. Their exact stored raw LP PnLs one second before expiry are -367.423469 USDC and +367.423437 USDC. The first position is capped at its 122.474488 USDC of LP backing, while the second remains below its buyer-collateral cap, so their exact realizable sum is +244.948949 USDC. The on-chain bucket calculation instead nets the raw values first and reports -0.000031 USDC. Assertions also prove that the stored bucket notional, weighted rates, timestamps, indices, fees, buyer collateral and pool backing equal the exact sums of the two active swaps, isolating the defect to cap-after-netting valuation rather than bucket bookkeeping.

An unprivileged entrant can supply collateral while this hidden positive settlement claim is omitted from the share price. In the cloned ordering test, the entrant deposits 1,000 USDC before the negative swap is closed through executable liquidation and the positive swap settles at expiry. After the normal LP vest expires, the entrant withdraws 1,122.474505

H-05 Bucket PnL caps after netting swaps

CONTINUED

USDC. Depositing from the identical state only after both swaps close returns 999.999999 USDC. The 122.474506 USDC difference equals the incumbent LP's loss within two token atoms. An independent liquidator receives the same explicit 6.123724 USDC reward in both orderings and protocol fees are zero, so neither amount is counted as attacker impact.

The accompanying differential test covers three to six synthetic active buckets in both `BUY_FIXED` and `BUY_FLOATING` directions, with mixed signs, heterogeneous rates, fees, entry timestamps, entry indices and notionals. It exercises no-cap, one-cap and two-cap regimes and confirms that material divergence appears once an individual cap binds. The issue permits timed deposits to dilute a preexisting LP claim when bucket NAV is understated. The inverse error also permits withdrawals at the expense of remaining LPs when bucket NAV is overstated, so the root cause can transfer a material share of pool collateral between LP cohorts.

Example of exploitation:

1. The attacker monitors the public market state and identifies two swaps in the same bucket whose raw PnLs offset, but where one swap has exceeded its individual collateral limit.
2. One second before expiry, Kairo reports a share price of approximately 1 USDC because the bucket PnL is nearly zero.
3. The attacker supplies 1,000 USDC through the public `supplyCollateral` function.
4. Because the share price is understated, the attacker receives approximately 1,000 shares. At the correct 1.244949-USDC price, the attacker should receive only approximately 803.246 shares.
5. The negative swap is liquidated through the normal public liquidation path. It removes only its 122.474488 USDC of LP backing.
6. At expiry, the positive swap settles normally and credits approximately 367.423465 USDC to the pool.
7. The hidden net gain is now realized in pool collateral. Because the attacker was given too many shares, the gain is divided roughly equally between the attacker and the incumbent LP.
8. The attacker waits for the normal LP profit vest to expire and withdraws.

Ordering Attacker's final balance

1. Deposit before swaps close 1,122.474505 USDC
2. Swaps close before deposit 999.999999 USDC

— RECOMMENDATION

Compute share-mutating NAV as the sum of per-swap fair PnL values, applying each active swap's own negative `poolCollateralBacking` and positive `collateralBalance` bounds before aggregation. Apply the same ordering in fair, accrued-only and virtual share-price paths.

H-05 Bucket PnL caps after netting swaps

CONTINUED

If enumerating every active swap is too expensive, replace the current bucket representation with cap-aware partitions or another data structure that preserves enough per-position information to calculate the sum of individual clamps exactly; aggregate notional and collateral totals alone are insufficient. Until exact cap-aware valuation is available, block LP supply and withdrawal whenever a bucket contains more than one active position whose individual cap could bind. Add deterministic and differential regression properties requiring share-price NAV to equal the sum of individually realizable active-swap PnLs across both market directions and every cap regime.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test/audit/CAND_003_PerSwapCapsBeforeAggregation.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1018>

HIGH **H-06**

ACKNOWLEDGED

Paired collateral caps let hedged traders profit

CATEGORY Unexpected Behavior
LOCATION contracts/SwapCore.solcontracts/SwapCore.sol:819-838
REVIEW Main Review

DESCRIPTION

`buySwap` calculates buyer collateral and LP backing from the selected market's own `baseRate`. In a scoped Cumulative pair, `BUY_FIXED` uses the compounded-equivalent bumped quote while `BUY_FLOATING` uses the lower raw quote. Consequently, equal-notional positions can give the winning `BUY_FIXED` swap more `poolCollateralBacking` than the losing `BUY_FLOATING` swap has `collateralBalance`.

Liquidation evaluates and closes each position independently. After a sufficiently large increase in the shared reference index, `BUY_FIXED` receives its higher LP-backing cap while `BUY_FLOATING` loses only its lower buyer-collateral cap. The equal-notional floating payments cancel across the pair. The difference between the two caps therefore becomes trader profit rather than directional rate exposure.

The deterministic PoC uses the planned 90-day and 2x parameters with the production `IndexBaseRateV1`, `RawIndexBaseRateV1` and `UpgradableTenorRateOracle` contracts over one identically seeded history. The models return a correctly ordered 258.528382% bumped quote and 200% raw quote. Both sides charge a 1% risk premium, the production utilization curve, a 1% annual protocol fee and a 2% liquidation incentive. An unrelated keeper liquidates both positions, so the trader receives neither bounty.

The fixed swap pays 9,562,008.666594 USDC of LP backing while the floating swap confiscates 7,447,926.440233 USDC of buyer collateral. After both protocol fees and both keeper bounties, the trader earns 1,625,938.318748 USDC. The fixed-side pool falls from 10,000,000 USDC to 437,991.333406 USDC. Exploitation requires a high entry base rate, enough liquidity and buyer capital for both positions, plus a later reference increase large enough to make both sides liquidatable. The loss is mechanical once those conditions hold and can remove most of one LP pool.

H-06 Paired collateral caps let hedged traders profit

CONTINUED

This does not require inverted or independently stale fixed/raw quotes. The tested histories are identical and the fixed quote is correctly greater than the raw quote. The defect is that the two correct quote representations also set two incompatible collateral caps for a paired liquidation.

— RECOMMENDATION

Use one pair-level collateral rate for both markets. For Cumulative pairs, derive it from one shared observation and conservatively use at least `max(fixedBaseRate, floatingBaseRate)` for the base component of both buyer-collateral and LP-backing calculations. Continue adding the selected side's utilization fee and risk premium to buyer collateral.

At minimum, opening equal-notional paired positions must preserve the invariant that the buyer collateral confiscated from the losing side covers the LP backing paid by the winning side after round-trip fees and liquidation incentives. Add regression tests for both reference-rate directions, all configured tenors and liquidation as well as maturity settlement.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/PairedLiquidationCollateralAsymmetry.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1034>

HIGH **H-07**

ACKNOWLEDGED

Morpho adjustment speed reverses rate forecast

CATEGORY Math

LOCATION `contracts/oracles/BaseRateAverageIndex/MorphoBaseRateV1.sol:120-123,582-592`

REVIEW Main Review

DESCRIPTION

`MorphoBaseRateV1` treats Morpho's `ADJUSTMENT_SPEED` as a constant that pulls the observed borrow rate back toward the current target. `_wTenor()` uses this constant to assume that a high trailing rate will fall over the swap tenor.

Morpho's `AdaptiveCurveIrm` uses `ADJUSTMENT_SPEED` differently. It first calculates the signed utilization error, then applies `speed = ADJUSTMENT_SPEED * err` to `rateAtTarget`. When utilization is above the 90% target, `rateAtTarget` rises. It stays unchanged at 90% and falls only when utilization is below 90%.

KAIRO can therefore forecast rates in the wrong direction. If a user opens a `BUY_FIXED` swap while Morpho is stressed and utilization remains above 90%, KAIRO may store a fixed rate that is far below the rate later realized by the Morpho cumulative index. At maturity, the difference is paid from the LP reserve to the buyer.

The Base-fork test starts with a synchronized Morpho target of 7.5868429% and a current borrow rate of 30.3473715%. KAIRO quotes only 13.0400718% for a 30-day fixed swap. Ten more days above 99% utilization raise Morpho's target to 29.8450073%. After another twenty days at 90% utilization, the realized floating rate is 42.2841501%. On a 323,291,930.462843 USDC position using the planned 3.5x configuration, the uncapped LP loss is 7,290,736.411206 USDC. Settlement transfers the full 990,000 USDC reserve to the `BUY_FIXED` buyer.

This issue requires a valid but stressed Morpho market and continued above-target utilization after the KAIRO swap is opened. Creating the test state required borrowing 149.159 million USDC from a market with 1.404 billion USDC supplied, so the test does not claim that the source market can be manipulated cheaply. Once this market state exists, however, any user can open the KAIRO swap without controlling Morpho, an oracle, transaction ordering or a privileged account.

The shared POC keeps Morpho's target synchronized at entry, so it does not rely on the existing stale-target issue. Replacing the separate target-conversion error with exact exponentiation changes the fixed quote only to 13.0401665%. The loss still exceeds the reserve

H-07 Morpho adjustment speed reverses rate forecast

CONTINUED

after applying that corrected quote, the maximum 20% utilization fee and the 0.5% risk premium. The test uses `BUY_FIXED`, so it also does not rely on the separate raw `BUY_FLOATING` APY-blending issue.

Example:

1. The scoped Morpho market stays above 99% utilization long enough to establish a high trailing borrow rate.
2. An unprivileged trader opens a near-capacity KAIRO `BUY_FIXED` swap at the downward-decaying forecast.
3. Continued high utilization makes canonical AdaptiveCurve raise `rateAtTarget` instead.
4. Utilization returns to 90%, preserving the now-elevated target for the rest of the tenor.
5. Maturity settlement caps the trader's gain at and transfers, the full reserved LP backing.

— RECOMMENDATION

Do not use `ADJUSTMENT_SPEED` as a bare utilization or spot-rate mean-reversion coefficient. Project the actual AdaptiveCurve state equation from current utilization and an explicit utilization scenario. The projection must include `speed = ADJUSTMENT_SPEED * err`, the resulting evolution of `rateAtTarget` and the utilization-dependent curve multiplier.

If the oracle is intended to be an empirical forecast instead of a canonical projection, use an independently fitted parameter and label it accordingly. Add a conservative stress floor that covers continued above-target utilization rather than relying on the entry target's clamp.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_POCS/MorphoWrongMeanReversionCanonicalDrain.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1070>

HIGH **H-08**

ACKNOWLEDGED

BUY_FLOATING early exits omit compounding

CATEGORY Math

LOCATION contracts/lib/Utils.sol:755-799

REVIEW Main Review

DESCRIPTION

Scoped `BUY_FLOATING` markets use `RateConvention.Cumulative` with a raw, unbumped base-rate quote. For Morpho, `RawMorphoBaseRateV1` returns the continuously compounded rate $r = \ln(1 + \text{apy})$. Charging this rate linearly produces less growth than the cumulative Morpho index. This is intentional: the buyer receives the lower fixed payment, pays the higher realized floating payment and the difference protects the pool.

`calculateEarlyExitSettlement` removes that protection when the buyer exits before maturity. It uses `_calculatePayments` to project the remaining floating payment as simple interest:

$$\text{notional} * r * \text{timeRemaining} / \text{YEAR}$$

`cumulativeRemainingCrossTerm` then adjusts that amount for index growth that occurred before the exit. It does not compound the projected rate during the remaining interval. Production therefore projects a Morpho closing index as:

$$\text{currentIndex} * (1 + x)$$

while the allowlisted `CumulativeIndexBorrowMorpho` projects the same remaining interval with Morpho's three-term Taylor factor:

$$\text{currentIndex} * (1 + x + x^2 / 2 + x^3 / 6)$$

Here, x is the projected annual rate scaled by the remaining tenor. The closeout omits $\text{currentIndex} * (x^2 / 2 + x^3 / 6)$. Aave variable-debt indexes have the same type of nonlinear future growth through their binomial projection.

A trader can profit from this difference by opening a weighted combination of `BUY_FIXED` and `BUY_FLOATING` positions on the paired curve. If the rate rises, the trader keeps the profitable fixed-rate position until maturity but exits the losing floating-rate position immediately. The fixed position receives the benefit of cumulative rate growth, while the floating closeout charges only simple future interest. LPs fund the missing floating obligation.

H-08 BUY_FLOATING early exits omit compounding

CONTINUED

The production PoC uses the documented 90-day launch parameters: 2x leverage, the configured utilization curve, a 25% early-exit fee, a 2% liquidation incentive, a 1.15% risk premium, 91 daily buckets and a 5-USDC collateral floor. It opens 3,000,000 USDC of `BUY_FIXED` and 1,000,000 USDC of `BUY_FLOATING`, then moves the raw rate from 178% to 194%. These are stressed but valid rates around Morpho's documented 200% target-rate regime.

Production charges the floating buyer `53,028.714778` USDC. Applying Morpho's three-term Taylor factor charges `218,848.163462` USDC, so the early exit leaves `165,819.448684` USDC uncollected. The weighted trader makes `116,951.533794` USDC under production accounting. With the correct floating payment, the same positions lose `48,867.914890` USDC. This value comes from the directional LP pools. It is not caused by either position reaching its collateral or LP-backing cap.

A second test removes uncertainty about the final index path. At day 60, the trader early-exits both legs and immediately locks `15,638.391835` USDC of token profit. The source-compatible floating closeout changes that same completed transaction sequence to a `7,234.983377` USDC loss. The `22,873.375212` USDC difference is only the remaining 30-day Taylor growth and all three relevant caps remain slack.

The issue requires an early-exit-enabled cumulative `BUY_FLOATING` market that uses a raw quote. The remaining rate must be positive and enough tenor must remain for the omitted compound growth to exceed the early-exit fee. The validated case also requires a stressed Morpho rate and enough liquidity on both sides of the paired market. The trader does not need privileged access or control over Morpho. The trader can open the positions first and exit only if the curve later moves into a profitable state. The day-60 proof closes both legs in the same state, so exploitability does not depend on predicting the reference index after the exits.

— RECOMMENDATION

Do not project a cumulative reference index from a raw quote using only `1 + r * timeRemaining / YEAR`. Calculate the remaining growth with the same model as the reference index, then derive the total floating payment from the projected closing index:

```
projectedClosingIndex = currentIndex * remainingGrowthFactor
```

```
totalFloatingBase = notional * (projectedClosingIndex / entryFloatingIndex - 1)
```

For Morpho borrow, use the same three-term Taylor factor as `CumulativeIndexBorrowMorpho`. For Aave variable debt, use Aave's binomial factor. For Aave supply, use its liquidity-index calculation. Utilization fees and risk premiums should continue to accrue linearly and separately.

H-08 BUY_FLOATING early exits omit compounding

CONTINUED

Use the resulting floating payment consistently in `projectedFavorsBuyer`, `netObligation`, adapter closeout values and active LP valuation.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/RawFloatingEarlyExitOmitsFutureCompounding.ts

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1040>

HIGH **H-09**

ACKNOWLEDGED

Buyer NAV ignores executable liquidation value

CATEGORY MEV
LOCATION `contracts/lib/Utils.sol:2091-2108`
REVIEW Main Review

DESCRIPTION

`getBuyerSettlementValue()` is used by `KairosBuyerAdapter` to value its swaps when the parent Vault calculates its share price. For a live swap that allows early exit, the function first calculates the buyer's value from accrued payments. It then replaces that value with the voluntary early-exit payout whenever the early-exit payout is lower. The function does not first check whether the pool can already be liquidated.

This produces an incorrect value when liquidation and early exit would pay different amounts. Liquidation considers accrued payments only and ignores the remaining term. Once the accrued buyer claim reaches `poolCollateralBacking - liquidatorReward`, any account can call `liquidateSwap()`. The buyer then receives its collateral, bounty and the pool backing left after the keeper reward. A voluntary early exit also prices the remaining term, which can make its payout much lower. Therefore, once pool liquidation is available, the lower early-exit quote no longer represents the value that can be realized for the buyer.

`KairosBuyerAdapter._realAssets()` includes this lower value in Morpho Vault V2's reported assets. If the Vault is open for deposits, a new depositor can receive too many shares before a keeper liquidates the swap. The higher liquidation payout then enters the adapter and is shared with those new shares, transferring part of a claim that belonged to existing shareholders. This requires an open BuyerAdapter Vault, a live early-exitable `BUY_FIXED` swap whose accrued claim has crossed the pool liquidation threshold and a remaining-tenor quote that makes voluntary early exit worth less than liquidation. The state is public. After it exists, the depositor needs no privileged role or oracle write. The amount transferred depends on the adapter allocation, available LP backing and the difference between the two payouts.

The production-path PoC uses a 30-day cumulative `BUY_FIXED` swap. After two days, a realized rate increase makes the pool liquidatable while the remaining-tenor quote falls against the buyer. From the identical state:

H-09 Buyer NAV ignores executable liquidation value

CONTINUED

- the buyer paid `482,951.019587` USDC at entry;
- immediate early exit and the value reported to the buyer adapter, is `487,812.625515` USDC;
- liquidation by an independent keeper pays the buyer `893,909.923698` USDC;
- the omitted executable value is `406,097.298183` USDC, while the separate `21,629.416005` USDC keeper reward is excluded.

Against unmodified official Morpho Vault V2 bytecode at commit

`f698b9ea78c528497d3d259cf7271ec25b60dca6`, an entrant deposits `1,004,861.605928` USDC at the understated NAV. After the independent keeper liquidates and Vault V2's maximum allowed NAV-growth limit catches up, the entrant redeems with `203,048.649091` USDC of profit. The incumbent loses `203,048.649092` USDC. The one-atom difference is token rounding.

Example:

1. Incumbent vault shareholders fund a buyer-adapter `BUY_FIXED` position.
2. Realized floating accrual moves above the pool-side liquidation threshold while the remaining forward curve is adverse to the buyer.
3. `realAssets()` reports the lower early-exit closeout even though liquidation is already executable at the higher accrued payout.
4. The attacker deposits into the parent vault at that understated NAV.
5. An independent keeper calls `SwapCore.liquidateSwap()`; the larger buyer payout lands in the adapter.
6. After Vault V2 recognizes the asset increase, the attacker redeems its share of the pre-deposit claim, diluting incumbents by the same amount.

— RECOMMENDATION

Make buyer NAV select by executable state rather than taking an unconditional minimum. At one fresh reference-index snapshot, first determine whether either side is liquidatable. If the pool is liquidatable, report the exact buyer payout that `liquidateSwap()` would realize: buyer collateral plus bounty plus `poolCollateralBacking - liquidatorReward`. If the buyer is liquidatable, report zero. Only compare accrued value with voluntary early-exit closeout while neither liquidation is executable.

If a per-swap liquidation preview cannot be made cheap enough, block parent-vault deposits and mints while any tracked swap has divergent executable liquidation and reported closeout values. Do not merely delay recognition with Vault V2's `maxRate`; that spreads the already-realized uplift over the diluted share supply.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_POCs/LiquidationBypassesRemainingTerm.t.sol

H-09 Buyer NAV ignores executable liquidation value

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1043>

MEDIUM **M-01**

ACKNOWLEDGED

Stale Morpho target underquotes BUY_FIXED swaps

CATEGORY Math

LOCATION contracts/oracles/BaseRateAverageIndex/MorphoBaseRateV1.sol:559-576

REVIEW Main Review

DESCRIPTION

`MorphoBaseRateV1._apyAtTarget()` reads `AdaptiveCurveIrm.rateAtTarget(marketId)` directly from storage. Morpho updates that mapping only when the market accrues, so the value omits target-rate movement over `block.timestamp - market.lastUpdate`.

The paired `CumulativeIndexBorrowMorpho` reference oracle uses `borrowRateView()` instead. That call projects the borrow rate and index through the same idle interval with the time-evolved `AdaptiveCurve` target. The fixed-rate anchor can consequently remain at an old target while the floating index already reflects the projected target. Permissionless Kairo pokes keep the 24-hour oracle rings fresh but do not accrue Morpho or update its stored target.

A buyer can hold the external market above target utilization, wait through an otherwise idle interval and open a scoped 90-day `BUY_FIXED` swap before another Morpho state change stores the new target. The understated entry rate remains fixed in the swap; subsequent settlement uses the higher floating index and transfers the difference from LP collateral.

The Foundry PoC starts with a 5% annual target, keeps utilization at 100% for eight days and pokes both Kairo base oracles every four hours. The projected target reaches 14.9592% while the stored target remains 5%. The stale 90-day base quote is 10.7426% instead of 19.7761%. Under the standard 2x leverage and utilization-fee parameters, a maximum swap against 1,000,000 USDC of LP backing realizes a 559,867.675122 USDC LP loss. Replacing only the stale quote with the synchronized quote prevents buyer profit in the same scenario.

Impact is material LP loss, but exploitation requires enough capital to sustain above-target Morpho utilization for days, no intervening state-changing Morpho interaction and sufficient Kairo liquidity to outweigh the manipulation cost.

RECOMMENDATION

Do not derive the long-end anchor or clamp from the raw `rateAtTarget` mapping. Project the current target from Morpho market state with the canonical `AdaptiveCurve` equation or

M-01 Stale Morpho target underquotes BUY_FIXED swaps

CONTINUED

derive an equivalent live target from `borrowRateView()` and current utilization. Use the same projected state for both `_apyAtTarget()` and the clamp envelope.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/MorphoIdleTargetStalenessDrainsLp.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1029>

MEDIUM **M-02**

ACKNOWLEDGED

Cumulative oracle upgrades reprice live swaps

CATEGORY Validation

LOCATION contracts/oracles/wrappers/UpgradableCumulativeRateOracle.sol:41-46

REVIEW Main Review

DESCRIPTION

`_validateDownstreamHealth` accepts any downstream that returns `isValid = true`. It ignores the returned cumulative index. The shared activation logic checks ERC-165 support and metadata decimals, but neither check establishes that the replacement uses the same cumulative-index epoch as the active source.

`SwapCore` stores its rate index under the wrapper address, so changing the downstream does not reset that accounting history. A replacement whose first index is higher than the previous value is therefore treated as yield earned by every existing swap. A public buyer can observe the queued replacement, open a `BUY_FIXED` swap before activation and receive LP collateral when the artificial increase makes the pool liquidatable. The PoC changes a zero-rate source from `1e27` to `2e27`; activation succeeds and 99% of the supplied LP collateral is paid to the buyer after three days even though no interest accrued.

For example:

- Governance queues a replacement cumulative oracle that reports `1.26e27`, while the active oracle reports `1.20e27`. Both are healthy and use the same decimals; they only have different index baselines.
- During the timelock, a buyer calls `SwapCore.buySwap()` to open a 10,000,000 USDC `BUY_FIXED` swap. `SwapCore` records `1.20e27` as its entry floating index and locks LP backing.
- After the timelock, governance calls `activateDownstream()`. The wrapper accepts the replacement because it checks `isValid` but never compares the old and new indices.
- Any account then calls `SwapCore.liquidateSwap()`. `RateIndexLib.update()` reads `1.26e27` through the unchanged wrapper address, so the 5% baseline jump is counted as floating interest earned by the buyer.
- After three days at a 5% fixed rate, the artificial floating payment is about 500,000 USDC while the fixed payment is only about 4,110 USDC. The buyer can therefore receive roughly 495,890 USDC from locked LP backing even though the market earned no such interest.

M-02 Cumulative oracle upgrades reprice live swaps

CONTINUED

A lower replacement index also passes activation. `RateIndexLib.update` then clamps every reading to the old high-water at lines 89-98. Accrual remains frozen until the new source catches up and it can remain frozen permanently. A downstream returning `(0, true)` is accepted as healthy as well, but every subsequent `SwapCore` index update reverts.

The oracle owner does not need to act maliciously. The failure occurs when a normal vendor migration or adapter replacement uses a different baseline, which cumulative indices commonly do. The timelock exposes the change but does not protect collateral already locked in live swaps.

— RECOMMENDATION

Make activation prove index continuity, not only interface compatibility. Read both the active and candidate cumulative indices, require both to be valid and nonzero and reject a candidate outside a deliberately small relative tolerance. If migrations between different epochs are required, store a timelocked conversion factor or offset in the wrapper and normalize the candidate so the wrapper's first post-upgrade value exactly continues from its last pre-upgrade value.

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1030>

MEDIUM **M-03**

ACKNOWLEDGED

Just-in-time LPs capture accelerated swap fees

CATEGORY Unexpected Behavior
LOCATION contracts/lib/Utils.sol:814-843
REVIEW Main Review

DESCRIPTION

`calculateEarlyExitSettlement` immediately includes the entire remaining fixed or floating payment when that payment favors the pool. `_calculatePayments` includes the swap's utilization fee and risk premium in that remaining payment. Consequently, an early exit credits fees for the unelapsed contract term to `pool.totalCollateral` at once.

LP minting values the same live swap differently. `calculateBucketUnrealizedPnL` removes the unaccrued utilization fee and risk premium from active-pool NAV and `supplyCollateral` mints shares at that lower price. This is sensible while the swap remains open because those fees have not been earned yet. It becomes unsafe when a pending early exit makes the excluded fees immediately realizable. There is no fee-entitlement checkpoint or settlement lock, so shares minted immediately before the exit receive the same fraction of the accelerated fees as shares that backed the swap before the transaction.

An attacker can observe a buyer's public `exitSwapEarly` transaction, front-run it with `supplyCollateral`, then let the buyer crystallize the remaining-term fees. `withdrawCollateral` initially caps the attacker at its entry price, but the cap expires after `lpProfitVestSeconds`. The profit remains in the pool after the swap is gone, so waiting out the default 12-hour window unlocks it. The attacker bears the risk that another swap uses its liquidity during that interval, but it does not need to control the exiting buyer or manipulate an oracle.

The PoC sets the configured early-exit fee to zero to isolate the remaining-term fees. With 1,000,000 USDC from the incumbent, a 100,000,000 USDC swap and a 9,000,000 USDC front-running deposit, early exit crystallizes 41,546.256332 USDC that was absent from the mint price. After 12 hours, the attacker withdraws 9,037,293.376054 USDC. Its 37,293.376054 USDC profit exactly equals the incumbent's loss relative to an otherwise identical exit without the attacker. A notional sweep reaches 245,235 USDC of profit on the same 9,000,000 USDC deposit.

M-03 Just-in-time LPs capture accelerated swap fees

CONTINUED

— RECOMMENDATION

Do not credit the unaccrued part of the utilization fee and risk premium directly to ordinary LP collateral when a swap exits early. Track that amount separately and vest it into LP NAV over the canceled remaining term, so a depositor cannot acquire the full remaining-term fee by supplying immediately before settlement and waiting only for `lpProfitVestSeconds`.

Alternatively, track fee entitlement by LP deposit epoch and distribute accelerated fees only to shares that were eligible before the exit became executable.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/EarlyExitFeeJitLpCapture.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1018>

MEDIUM **M-04**

ACKNOWLEDGED

Opposing swap masks the liquidation guard

CATEGORY Unexpected Behavior
LOCATION contracts/adapters/KairosMarketAdapter.sol:788-791
REVIEW Main Review

DESCRIPTION

`KairosMarketAdapter._liquidationGapOpen()` decides whether permissionless `forceDeallocate()` must be blocked by comparing the market's accrued-only LP share price with its fair LP share price. Both prices are market-wide marks. `VirtualShareLib` and `Utils.calculateTotalUnrealizedPnL()` add the PnL of every active bucket before producing them.

Liquidation is decided per swap. `Utils.isSwapLiquidatable()` compares one swap's accrued obligation with that swap's collateral. `Utils.liquidateSwap()` then settles only the selected position. Consequently, an opposite projected remaining leg from a second position can make `accruedOnly <= fair + LIQ_GAP_TOLERANCE` while another swap remains independently buyer-liquidatable. The guard then permits `forceDeallocate()` even though liquidating that swap would immediately increase the pool value.

A vault shareholder who also owns Kairos LP shares can exploit this cancellation. The attacker maintains an offsetting swap, calls Morpho Vault V2 `forceDeallocate()` before the underwater buyer is liquidated and burns vault-owned Kairos shares at the lower pre-liquidation price. The later liquidation distributes the buyer's collateral over fewer outstanding shares. This transfers part of the vault's forfeited recovery to the attacker's remaining LP position.

The shared PoC observed an accrued-only price of `1.229976675994e18` and a fair price of `1.4488073380955e18`, so the guard was closed even though the targeted swap was independently liquidatable. With a `900,000 USDC` forced withdrawal and identical liquidation timestamps in the control and attack executions, the vault lost `23,836.410634 USDC`. The attacker's co-LP position gained the same amount. The test also included the eventual loss on the masking position and showed that the co-LP gain exceeded the maximum `18,000 USDC` Morpho force-deallocation penalty for that withdrawal.

The attack requires capital for the masking swap, enough vault shares to absorb the Morpho penalty, a material co-LP position and execution before liquidation. These requirements reduce likelihood, but the value transfer scales with the forced withdrawal and available liquidation recovery. This is a bypass of the liquidation guard introduced for the earlier buyer-side force-deallocation issue. The earlier defect had no liquidation-aware

M-04 Opposing swap masks the liquidation guard

CONTINUED

protection; the current defect is that the remediation uses a net market-wide comparison for a per-swap state transition.

— RECOMMENDATION

Do not use a market-wide net PnL comparison to decide whether per-swap liquidation recovery exists. Block permissionless `forceDeallocate()` whenever any live swap is independently buyer-liquidatable and its immediate liquidation would increase the withdrawal value. Evaluate positions independently so projected gains and losses from different swaps cannot cancel.

If an exact per-swap check is too expensive, maintain conservative liquidation-gap metadata or a market-level counter as swaps are opened, updated, settled and liquidated. A simpler fail-closed alternative is to disable permissionless partial withdrawals while the market has active swaps and require allocator-controlled deallocation after executable liquidations are processed.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/ForceDeallocateAggregateLiquidationGapBypass.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1084>

MEDIUM **M-05**

ACKNOWLEDGED

Morpho target conversion underquotes growth

CATEGORY Math

LOCATION contracts/oracles/BaseRateAverageIndex/MorphoBaseRateV1.sol:559-576

REVIEW Main Review

DESCRIPTION

`MorphoBaseRateV1._apyAtTarget()` converts the annualized continuously compounded `rateAtTarget` into an APY with Morpho's three-term Taylor approximation. `_apyToTenorRate()` later treats that truncated value as a full-year APY and converts it into the fixed rate for the requested tenor.

This conversion is lower than the growth that `CumulativeIndexBorrowMorpho` can report from the same constant rate. Morpho applies its Taylor factor to the updated borrow principal on every accrual. The product of several accrual factors exceeds the single full-year Taylor polynomial assumed by the base oracle. Permissionless `accrueInterest` calls can select that cadence without changing the target or utilization.

A `BUY_FIXED` buyer therefore pays the lower stored fixed leg and receives the higher cumulative floating leg. An isolated Foundry test holds the target and reference rate at the AdaptiveCurve IRM's valid 200% annual target bound, uses a 90-day market with the protocol-wide maximum 18x leverage and accrues Morpho once per day. `MorphoBaseRateV1` quotes 235.8774% annualized while the reference realizes 258.5284%. On a 30.917M USDC notional, the uncapped payment difference is 1.726M USDC and settlement consumes all 999,000 USDC of reserved LP backing.

A second test reproduces the rate mismatch against the deployed Morpho Blue core and `AdaptiveCurveIrm` on a pinned Base fork. After canonical state transitions raise `rateAtTarget` to 200% annually, the test holds utilization at the 90% IRM target so the target, trailing spot and live borrow rate are the same value. The test creates a fresh 18x Kairo market, quotes 235.8774%, realizes 258.5284% under daily accrual and creates a 1.711M USDC uncapped payment gap. Settlement drains all 990,000 USDC of reserved backing in that maximum-leverage test market. This removes stale-target and utilization-prediction effects from the comparison, but it does not show that a planned or deployed Morpho Kairo market uses 18x leverage.

The explicit Morpho deployment plan uses 3.5x leverage for 30 days, 2.5x for 60 days and 2x for 90 days. At a constant 200% target with daily accrual, the same fixed/reference difference can consume approximately 23.17%, 21.10% and 19.21% of the affected pool backing at

M-05 Morpho target conversion underquotes growth

CONTINUED

maximum capacity. At a 100% target, those values fall to approximately 5.38%, 4.64% and 4.04%. These figures follow from the protocol's capacity and collateral formulas and show that the defect remains material under the planned configurations, but the full-drain result depends on the separate 18x test configuration.

Exploitation requires a stressed Morpho market whose target rate has risen materially, enough Kairo liquidity to open a near-capacity position and the borrow rate remaining near the target during the swap. The canonical test reaches the maximum target only after 30 days above 99% utilization and uses test-provisioned USDC to keep utilization at 90% afterward. The Kairo buyer does not need to hold the Morpho debt once that external state exists and permissionless accrual can realize the basis. However, the elevated target and stable utilization are specific market conditions that the buyer cannot create cheaply through Kairo alone.

— RECOMMENDATION

Do not replace the shared `_apyAtTarget()` result with an exact APY while `_blendAndClamp()` still operates in APY space. `RawMorphoBaseRateV1` inherits that calculation. A standalone replacement would raise its `BUY_FLOATING` quote and increase the separate APY-blend/log Jensen overquote.

Refactor the common model so it blends and clamps continuously compounded rates before applying either side's final quote transform:

```
spotCc = ln(1 + spotApy)
targetCc = ratePerYearWad
ccBlend = clamp(w(T) * spotCc + (1 - w(T)) * targetCc,
targetCc / CURVE_STEEPNESS,
targetCc * CURVE_STEEPNESS)
```

For `MorphoBaseRateV1` on `BUY_FIXED`, apply the full-tenor exponential bump to `ccBlend`. The resulting fixed leg is an upper bound on any partition of Morpho's three-term Taylor accrual. For `RawMorphoBaseRateV1` on `BUY_FLOATING`, return `ccBlend` directly so the raw quote does not exceed the modeled mean-reverting reference rate.

If the target-conversion correction must be deployed before the common rate-space refactor, make it fixed-side-specific through a separate hook or override. Otherwise, disable `RawMorphoBaseRateV1` until the APY-space blend is corrected. Do not expose the exact-target APY through the shared raw implementation as a standalone fix.

PROOF OF CONCEPT <https://github.com/Guardian0rg/protocol-team1-1783698575429/commit/69c87ba52c19957b8779e0e73b703e3fa40c6db3>

M-05 Morpho target conversion underquotes growth

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1031/>

MEDIUM **M-06**

ACKNOWLEDGED

Conservative NAV lets entrants capture gains

CATEGORY Unexpected Behavior
LOCATION contracts/adapters/KairosMarketAdapter.sol:722-726
REVIEW Main Review

DESCRIPTION

`KairosMarketAdapter._realAssets()` reports the vault's LP position at `getPoolSharePriceVirtualConservative()`. That price is `min(fair, accrued-only)`, so whenever a healthy live book has a positive remaining projection, the adapter omits value that the same LP shares can realize immediately through `SwapCore.withdrawCollateral()` at the fair price. The liquidation defense applies the discount market-wide even when no swap is liquidatable.

Morpho Vault V2 uses `realAssets()` to price deposits. A newcomer can therefore deposit after a favorable rate move and mint vault shares against the accrued-only value instead of the already-current fair value. Once the entrant owns vault shares, it can call permissionless `forceDeallocate()` on behalf of itself. The adapter allows this direction because `_liquidationGapOpen()` blocks only when `accruedOnly > fair`; `SwapCore.withdrawCollateral()` then burns the vault's Kairos shares at the higher fair price and moves the proceeds to idle vault cash. Morpho's `maxRate` delays recognition of the resulting uplift, but it does not assign that pre-deposit value back to incumbent shares. The entrant waits for the rate cap to catch up and redeems its share of the uplift.

The PoC opens a healthy 30-day `BUY_FIXED` swap against a 2,000,000 USDC pool, advances two days on a self-consistent 5% cumulative curve and changes only the remaining-tenor forecast from 5% to 3%. `liquidateSwap()` reverts with `E507`, proving no liquidation is available. The adapter nevertheless reports `1,000,866.559342` USDC while its fair LP value is `1,077,573.624773` USDC. A permissionless force-deallocation of the adapter's full fair value therefore realizes `76,707.065431` USDC that was omitted from deposit NAV, exceeding Morpho's maximum `21,551.472496` USDC penalty.

M-06 Conservative NAV lets entrants capture gains

CONTINUED

The canonical Vault V2 integration has an entrant deposit `1,000,866.559342` USDC, pay the full 2% force-deallocation penalty and later redeem `1,027,909.653303` USDC. Its `27,043.093961` USDC profit equals the incumbent's `27,043.093962` USDC loss within one token unit. No oracle manipulation, liquidatable swap, co-LP ownership or privileged role is required; the attacker only needs an open vault, a positive projection large enough to cover the configured penalty, capital for the deposit and time for `maxRate` to recognize the already-idle uplift. The value transfer scales with the adapter allocation and the fair/accrued-only gap.

— RECOMMENDATION

Do not use a market-wide accrued-only floor for parent-vault NAV. Preserve the liquidation defense with a per-swap liquidation-aware valuation: use the liquidation floor only for independently pool-side-liquidatable positions, while healthy positions remain at fair value. Use the same conditional mark in both storage and virtual-settlement paths.

If an exact on-chain liquidation-aware mark is too expensive, block vault deposits/mints whenever the conservative price is below fair or account for the difference in a reserve whose entitlement remains with shares outstanding before the gap opened. Merely blocking `forceDeallocate()` delays the extraction but does not remove the deposit dilution as the projection accrues.

PROOF OF CONCEPT <https://github.com/GuardianOrg/protocol-team1-1783698575429/commit/c34a5076a9188445ad10f94bc032a3df2b4cc7f2>

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1018>

MEDIUM **M-07**

ACKNOWLEDGED

Oracle upgrades enable a paired-swap LP drain

CATEGORY Upgradeability

LOCATION `contracts/SwapCore.sol:758-902`, `contracts/oracles/wrappers/BaseUpgradableOracleWrapper.sol:201-260`

REVIEW Main Review

DESCRIPTION

Each scoped market pair shares one reference-rate oracle but uses separate fixed-side and floating-side `UpgradableTenorRateOracle` wrappers. `queueDownstream()` publicly identifies a replacement for three days before `activateDownstream()` changes the active quote. `SwapCore.buySwap()` remains open during this interval. It reads whichever base quote is active for the selected side and stores that quote with the current shared reference index. It does not record a base-oracle epoch or prevent the corresponding market from accepting an opposite swap under another epoch.

Consequently, a searcher can open equal-notional opposite swaps around a known activation. For an upward update, the searcher opens `BUY_FIXED` immediately before activation and `BUY_FLOATING` immediately afterward. The first position keeps the outgoing fixed quote; activation does not reprice it. The second position stores the replacement floating-side quote. Both positions can have the same timestamp and reference index even when governance activates both wrappers atomically.

The upgrade is exploitable in this direction only when all of the following conditions hold. It must create a discrete increase between the outgoing fixed-side base quote and the replacement floating-side base quote that is larger than the utilization and risk markups charged to both positions. In rate terms, the condition is:

```
replacement floating base - outgoing fixed base
> fixed utilization fee + fixed risk premium
+ floating utilization fee + floating risk premium
```

`buySwap()` must remain enabled before and after activation and the first position must keep its outgoing quote after the pointer changes. The corresponding markets must continue to use the same reference-rate oracle, rate convention and term so equal-notional positions can store the same reference index and cancel the realized reference leg. The candidate quote and activation must also be observable early enough for the attacker to order one transaction before activation and the opposite transaction afterward. Finally, both sides

M-07 Oracle upgrades enable a paired-swap LP drain

CONTINUED

need enough available LP collateral and the attacker needs enough buyer collateral and liquidation bounty to open both positions at the required notional. The public three-day queue, unchanged market pair and same-block ordering satisfy these conditions in the PoC.

The 24-hour-to-12-hour TWAR change is only one concrete example. Any honest implementation or parameter migration can trigger the issue if it creates a sufficiently large discrete quote change while preserving the conditions above. Conversely, this trade is not profitable as shown if the effective quote change is no larger than the combined markups, new entries stop before the candidate becomes public, old and new positions are separated into different market epochs, the two sides no longer share a cancellable reference leg, or liquidity and buyer capital are insufficient. No compromised signer, malicious owner, stale quote, or invalid oracle value is required.

The exact scoped PoC uses a healthy 24-hour TWAR pair as the outgoing deployment and the repository-configured 12-hour TWAR pair as the replacement. All four models observe the same Aave V3 USDC borrow cumulative index and the same Aave R0 floor. During the last 24 hours of the timelock, the source realizes 12 hours near 0% followed by 12 hours near 20%. The outgoing 24-hour fixed model therefore reports an annualized base quote of `9.99999999999926%`, while the replacement 12-hour raw floating model reports `19.997260774276275%`. These are annualized oracle quotes, not one-day returns. The replacement is slightly below exactly 20% because it derives and annualizes a rate from discrete cumulative-index growth with integer rounding. Neither oracle is false or stale; each correctly reports its own averaging window.

The pre-activation `BUY_FIXED` adds its `0.2000273960094284%` utilization fee and `0.5%` upper risk premium to the outgoing quote. It therefore locks an all-in fixed rate of `10.7000273960093544%`. The post-activation `BUY_FLOATING` locks the replacement base quote as the fixed leg it receives, while its realized floating payment carries a `0.4%` utilization fee and `0.5%` lower risk premium. If `R` is the annualized Aave rate realized over the common one-day term, the two uncapped buyer payoffs are:

$$\text{BUY_FIXED} = R - 10.7000273960093544\%$$

$$\text{BUY_FLOATING} = 19.9972607742762750\% - R - 0.4000000000000000\% - 0.5000000000000000\%$$

$$\text{combined} = 8.3972333782669206\%$$

The shared reference rate `R` cancels because the positions have equal notional, entry time and entry index. The public activation has therefore been converted into a fixed annualized spread after both utilization fees and both signed risk premiums. Atomic activation does not remove the opportunity because the two attacker transactions surround the complete activation transaction. A downward update creates the symmetric trade: `BUY_FLOATING` before activation and `BUY_FIXED` afterward.

M-07 Oracle upgrades enable a paired-swap LP drain

CONTINUED

The validation supplies 1,000,000 USDC to each LP pool and uses the documented 1-day term, 3.5x leverage, 1% utilization slope, 85% kink, 20% maximum kink fee, 25% early-exit fee, 2% liquidation incentive and signed 50 bp premiums on both sides. The attacker sizes both positions to 80% of the replacement floating market's calculated capacity, below the 85% utilization kink. This produces equal notionals of 5,110,699,968.040934 USDC and requires approximately 1.289 million USDC across buyer collateral and liquidation bounties.

Over one day, the 8.397233378% annualized spread would produce approximately 1,175,773.708436 USDC before settlement caps. The floating position locks 800,000 USDC of LP backing, so normal maturity settlement caps the transfer at exactly 800,000 USDC. The floating LP pool falls from 1,000,000 USDC to 200,000 USDC, the fixed LP pool remains whole and the attacker retains 800,000 USDC as net profit after both buyer margins are returned. An intended, non-malicious oracle migration can therefore remove most of an affected LP pool.

The validated 24-hour-to-12-hour migration is plausible and scope-compliant, but the test does not establish that it is currently queued on a live deployment. Only the external Aave Pool index surface is deterministic so the required 72-hour history can be reproduced locally. The Kairo source, floor, base models, wrappers, risk oracle, market entry, collateral calculations, token transfers and maturity settlement all use production implementations.

Example of attack or failure scenario:

M-07 Oracle upgrades enable a paired-swap LP drain

CONTINUED

1. Governance queues healthy 12-hour bumped/raw base models to replace an outgoing 24-hour pair over the same Aave source and R0 floor.
2. During the timelock, a sharp source-rate increase makes the outgoing 24-hour fixed quote approximately 10% and the replacement 12-hour raw floating quote approximately 19.997%.
3. Immediately before activation, a searcher opens `BUY_FIXED` at the outgoing quote. After utilization and risk fees, this position locks a 10.700027% all-in fixed rate.
4. Governance activates both replacement wrappers in one transaction.
5. The searcher back-runs with equal-notional `BUY_FLOATING` at the 19.997261% replacement quote. Both positions store the same timestamp and Aave entry index.
6. At maturity, the common realized Aave leg cancels across the pair. After the floating position's 0.4% utilization fee and 0.5% risk premium, the pair retains an 8.397233% annualized spread.
7. The PoC sets the realized Aave rate to the first position's 10.700027% all-in fixed rate. This makes `BUY_FIXED` settle at zero and places the full paired gain on `BUY_FLOATING`; it does not create the gain because the realized rate already cancels algebraically before collateral caps.
8. The uncapped one-day gain exceeds the 800,000 USDC backing reserved for `BUY_FLOATING`. Settlement transfers the full backing to the attacker and leaves 200,000 USDC in that LP pool.

— RECOMMENDATION

Do not reveal a discrete replacement quote while new swaps can still lock the outgoing quote. Stop new entries in both corresponding markets before publishing a replacement. Keep the old wrappers available for existing positions and route new trading to a new market pair that uses the replacement models.

If a live migration is required, implement one coordinated pair-level quote epoch. `buySwap()` should reject entries in both markets while a replacement is pending. Activation should then blend old and new curves over a bounded transition or otherwise prevent two near-simultaneous positions from preserving both sides of a discrete quote change. Merely activating both wrapper pointers in one transaction is insufficient.

PROOF OF CONCEPT <https://github.com/GuardianOrg/protocol-team1-1783698575429/commit/477f4f42f2e7114b51713efb330b164d603e44a8>

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-08**

ACKNOWLEDGED

Dead shares drain fresh LP refills

CATEGORY Validation
LOCATION contracts/SwapCore.sol:593-610
REVIEW Main Review

DESCRIPTION

`supplyCollateral` accepts an idle pool at `MIN_SHARE_PRICE` even when old shares remain outstanding. It mints the refill at the synthetic floor without cancelling the old claims. `withdrawCollateral` later values every share at the same floor, so an old holder can redeem against collateral supplied by a newer LP.

A healthy-price share pile initially retains only about one millionth of its old principal. That limit does not hold after another wipe. An attacker can refill the wiped pool at the floor, receive floor-priced shares and own the buyer position that makes the pool pay its full backing. If the attacker also calls `liquidateSwap`, the LP backing, buyer collateral and liquidation reward all return to the same wallet while the floor-priced shares survive. Those shares retain a claim equal to the whole refill.

The PoC repeats this sequence with the protocol fee disabled, as configured by the deployment script and the recorded Base `Admin`. The attacker starts with 5M USDC, self-hedges two 1M-USDC wipes and still holds 5M USDC plus `1e18` dead share atoms. A new LP then supplies 1M USDC. The attacker redeems `1e18` shares first, finishes with 6M USDC and leaves the new LP with no collateral.

`KairosMarketAdapter.allocate` rejects this state, but a direct `SwapCore.supplyCollateral` call remains available.

RECOMMENDATION

Reject every deposit priced at `MIN_SHARE_PRICE` while `totalShares` is nonzero, including idle pools. Recovery should use a new market or an explicit loss-epoch design that prevents pre-recovery shares from claiming post-recovery collateral. Do not rely on callers to reproduce the adapter guard.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test/audit/HYP_015_DeadShareFloorRecovery.t.sol

RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

ACKNOWLEDGED

Uncapped oracle call blocks emergency fallback

CATEGORY DoS

LOCATION contracts/SwapCore.sol:1489-1500

REVIEW Main Review

DESCRIPTION

When an expired swap is settled, `_settleSwaps` first calls `updateMarketRateIndex` without a gas limit. Only after that call returns does it enter `resolveExpiryIndex`:

```
try this.updateMarketRateIndex(marketId) {} catch {}
expiryIndex = Utils.resolveExpiryIndex(...);
```

This order can make Kairo's emergency fallback unreachable. The `try/catch` handles a failed oracle call, but it does not restore the gas that the call consumed.

After the settlement grace period, `resolveExpiryIndex` is supposed to retry the oracle with a fixed 2,500,000-gas allowance. If that protected retry fails, Kairo extrapolates the last known index and settles the swap. Before making the retry, `resolveExpiryIndex` requires at least 3,039,682 gas. This check prevents a caller from submitting too little gas and forcing extrapolation without giving the oracle a fair retry.

The uncapped call runs before this protection. Its three optional oracle pokes are each limited to 500,000 gas, but the final reference-oracle read in `RateIndexLib.update` has no limit. A reference oracle, adapter or downstream protocol can consume almost all forwarded gas before reverting or returning an invalid value. EIP-150 leaves a small amount of gas in each calling frame, so `try/catch` can continue. However, the remaining gas can be far below the 3,039,682 required for fallback. `resolveExpiryIndex` then reverts `E451` before the protected retry is attempted.

The current [Base transaction limit](https://docs.base.org/base-chain/network-information/troubleshooting-transactions#gas-limit-exceeds-maximum) is 16,777,216 gas. At that limit, the deterministic trace recorded the following values:

Reference topology	First reference-read forwarding	Gas after <code>catch</code>	<code>resolveExpiryIndex</code> entry	Gas at <code>E451</code> check	Result
--- --- --- --- --- ---					
direct reference	14,767,995	496,447	488,302	485,455	<code>E451</code>

M-09 Uncapped oracle call blocks emergency fallback

CONTINUED

```
| wrapper -\> adapter | 14,775,624 -\> 14,543,774 | 727,377 | 715,623 | 712,776 | E451 |
| wrapper -\> adapter -\> external protocol | 14,775,626 -\> 14,543,776 -\> 14,315,977 | 953,815 | 938,523 |
935,676 | E451 |
```

All three executions had 16,773,157 gas before the first self-call and forwarded 16,510,978 gas into it. Each transaction reverted with selector `0xd3b08d30`. The swap remained unsettled, no `ExpiryIndexExtrapolated` event was emitted and the market was not terminated.

The test configures the reference-rate, base-rate and risk-premium slots as three distinct pokeable oracles and makes all three due. Each poke receives the production 500,000-gas limit before the final uncapped reference read consumes the remaining gas. This reproduces the worst case that the 2,500,000-gas protected retry is designed to handle.

The minimum successful gas limits are far above Base's maximum:

```
| Reference topology | Highest observed failure | First observed success |
|---|---|---|
| direct reference | 100,439,950 | 100,470,034 |
| wrapper -\> adapter | 67,979,651 | 68,009,735 |
| wrapper -\> adapter -\> external protocol | 51,764,544 | 51,794,628 |
```

Control calls above those thresholds reached the protected retry, emitted `ExpiryIndexExtrapolated`, settled the swap and terminated the market.

An expired unsettled swap makes both `supplyCollateral` and `withdrawCollateral` revert `E415`. Therefore, a persistently gas-burning reference dependency can prevent settlement and block LP deposits and withdrawals for as long as the dependency remains in that state. The fallback was added to prevent exactly this type of dead-oracle lock, but the earlier uncapped call stops it from running.

The failure requires the oracle or one of its dependencies to consume almost all forwarded gas rather than reverting promptly. The settlement caller does not need to underfund the transaction: the PoC fails even at the chain's maximum permitted gas limit.

— RECOMMENDATION

Do not make the uncapped best-effort update after the grace period. Keep it before the grace period so Kairo can still collect a real post-maturity snapshot. Once the grace period ends, enter `resolveExpiryIndex` directly and let its fixed-budget retry decide whether to use the oracle or extrapolate:

```
if (block.timestamp < actualExpiry + admin.settlementGracePeriodSec()) {
try this.updateMarketRateIndex(marketId) {} catch {}
}
```

M-09 Uncapped oracle call blocks emergency fallback

CONTINUED

```
expiryIndex = Utils.resolveExpiryIndex(rateIndex, markets, marketId, actualExpiry, swapId, admin);
```

Alternatively, cap the first attempt so it must leave more than `MIN_FALLBACK_ADJUDICATION_GAS` plus the gas needed to finish settlement.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCs/HYP012FallbackGasStarvation.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1042>

MEDIUM **M-10**

ACKNOWLEDGED

Temporary liquidity underprices paired swaps

CATEGORY Unexpected Behavior
LOCATION `contracts/lib/Utils.sol:1051-1063`
REVIEW Main Review

DESCRIPTION

`seasonedUtilizationFee` prices a new swap against the pool's aggregate seasoned collateral. The calculation does not remember which LP supplied the collateral that reduced the fee. `buySwap` later adds only the swap's required backing to aggregate `pool.lockedCollateral`. `withdrawCollateral` permits any withdrawal that fits within `pool.totalCollateral - pool.lockedCollateral`. Consequently, nothing ties fee-reducing LP shares to the exposure priced against them.

An attacker can supply 900,000 USDC beside 100,000 USDC from an honest LP and wait until the deposit is fully seasoned. One helper call can then open equal-notional `BUY_FIXED` and `BUY_FLOATING` swaps before withdrawing both attacker LP positions. The withdrawals succeed when the honest collateral covers each market's required LP backing. In the scoped 30-day Aave-borrow test, the fixed pool ended the helper call with 100,000 USDC total and 76,321.915041 USDC locked. The floating pool ended with 100,000 USDC total and 100,000 USDC locked. The attacker retained no LP shares or backing obligation.

This finding depends on the separately reported High issue that lets the intended independent fixed and raw oracle rings return valid but inverted quotes. The two intended markets share one cumulative reference index, so equal notionals cancel the realized reference payment without an external hedge. With synchronized histories, the pair remains loss-making and temporary liquidity is not a standalone arbitrage.

The PoC restores one snapshot for its control and attack executions. Without temporary liquidity, the inverted pair loses 42,789.287774 USDC. With temporary liquidity, utilization payments fall by 62,376.951620 USDC and the pair earns 19,587.663845 USDC on-chain. It retains 18,551.346191 USDC after both protocol fees, risk premiums, liquidation bounties, LP and buyer funding costs and conservative Ethereum L1 gas. The honest pools lose 33,587.663845 USDC at maturity.

The 33,587.663845-USDC pool loss is the same terminal loss produced by the combined oracle-inversion exploit. It must not be added to the impact of the High oracle finding. This Medium finding captures the separate pricing defect: temporary capital removes 62,376.951620 USDC of compensation and changes the exact paired position from unprofitable to

M-10 Temporary liquidity underprices paired swaps

CONTINUED

profitable. Exploitation requires an open-LP real-value market, enough capital to season both deposits, honest collateral sufficient to cover the stored backing and the independent-ring inversion. The intended Aave-borrow configuration satisfies the contract and oracle topology, but the checked-in mainnet market row still has unresolved oracle addresses.

— RECOMMENDATION

Bind the liquidity used for utilization pricing to the resulting exposure. Track the LP shares or deposit epoch that contributed to the seasoned collateral and prevent the corresponding value from being withdrawn until the swaps priced against it are closed. An alternative is to charge a withdrawal adjustment equal to the utilization-fee increase that the departing liquidity would have caused and credit that amount to the remaining LPs.

Do not rely on the one-block deposit lock or the EMA reduction after withdrawal. Both occur too late to correct a fee already stored in a swap.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/CAND001TemporaryLiquidityScopedComposition.ts

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1053>

MEDIUM **M-11**

ACKNOWLEDGED

Morpho APY blend overquotes BUY_FLOATING

CATEGORY Math

LOCATION `contracts/oracles/BaseRateAverageIndex/MorphoBaseRateV1.sol:614-622`, `contracts/oracles/BaseRateAverageIndex/RawMorphoBaseRateV1.sol:57-62`

REVIEW Main Review

DESCRIPTION

`MorphoBaseRateV1` calculates the tenor weight for an exponentially mean-reverting rate, but `_blendAndClamp()` applies that weight after transforming the spot and target rates into APYs. `RawMorphoBaseRateV1` then takes `ln(1 + blendedApy)` and presents the result as the lower-bound continuously compounded rate for `BUY_FLOATING`.

Those operations are in the wrong order. Because `ln` is concave,

$$\ln(1 + w * \text{spotApy} + (1 - w) * \text{targetApy}) > w * \ln(1 + \text{spotApy}) + (1 - w) * \ln(1 + \text{targetApy})$$

whenever the two endpoints differ. The left side is the raw production quote. The right side is the average continuously compounded rate along the same exponential mean-reversion path that defines `w(T)`. Skipping the final tenor bump only removes fixed-leg compounding; it does not make the APY-space blend a lower bound on a time-varying reference rate. This directly contradicts the contract's stated safety property that an un-bumped `BUY_FLOATING` quote cannot exceed reference-index growth.

The canonical Base-fork proof isolates this transform-order error. It first uses ordinary deployed Morpho Blue and `AdaptiveCurveIrm` transitions at 100% utilization to reach the IRM's 200% annual target cap and to build the scoped one-day spot window. It then opens the planned 30-day, 3.5x KAIRO `BUY_FLOATING` market and supplies the deployed Morpho market toward

$$\text{utilization}(t) = 90\% + 10\% * \exp(-\text{lambda} * t).$$

At a capped target, the deployed IRM therefore returns the exact corresponding rate-space path

$$\text{rate}(t) = \text{targetRate} * (1 + 3 * \exp(-\text{lambda} * t)),$$

without target staleness, an IRM mock or an out-of-range utilization. The production raw quote is `6.576852449216882735`, while the rate-space control is `3.436034802458983200` and the canonical cumulative index realizes `3.970317501621355592` after compounding.

M-11 Morpho APY blend overquotes BUY_FLOATING

CONTINUED

On a 6,409,981.115665 USDC position, the entry includes the configured utilization curve and a 50 bp lower-side risk premium. The uncapped fixed-minus-floating payment is 1,363,730.388720 USDC, so canonical settlement transfers all 990,000 USDC of reserved LP collateral to the buyer. A separate assertion replaces the actual utilization charge with the configuration's full 20% annual cap; the remaining 1,265,243.314743 USDC gap still exceeds the reserve.

This is not the existing stored-target issue: `rateAtTarget` remains synchronized and pinned throughout the swap. It is also not the existing target-Taylor issue: that issue makes a `BUY_FIXED` quote too low under repeated constant-rate accrual, while this issue makes the scoped raw `BUY_FLOATING` quote too high under the oracle's own varying-rate mean-reversion model. Using an exact exponential target APY would increase, rather than remove, the APY-space Jensen gap.

Therefore, an unprivileged buyer can wait for a stressed Morpho state with a large spot/target spread, open a near-capacity `BUY_FLOATING` swap at the overstated fixed receipt and profit as utilization normalizes. Under the explicitly planned 30-day/3.5x configuration, the validated gap exceeds every token reserved for the position and removes 99% of a fresh 1,000,000-USDC LP pool.

The strongest validated case requires Morpho's target to have reached its documented cap and a one-day high-utilization spot window. The fork reaches that condition after 30 days at canonical 100% utilization using test-provisioned cbBTC/USDC, so it proves the accounting and loss but does not claim that deliberately creating the external state is cheap. Once such a stressed state occurs organically, however, the KAIRO buyer needs no Morpho debt, oracle permission, transaction ordering or privileged role. The profitable future path stays between 90% and 100% utilization and supplying toward it is recoverable capital that also earns Morpho supply yield.

Example:

1. Morpho cbBTC/USDC remains highly utilized long enough for `rateAtTarget` to approach its cap and the trailing one-day borrow rate is near the IRM's upper curve bound.
2. The attacker opens a near-capacity KAIRO `BUY_FLOATING` swap. `RawMorphoBaseRateV1` locks `ln(1 + average(APY))` as the fixed receipt.
3. Morpho utilization relaxes from 100% toward its 90% target, naturally or through additional supply. The cumulative reference follows the lower `average(log(1 + APY))` rate path.
4. At maturity, the fixed receipt exceeds canonical floating growth plus the stored utilization fee and risk premium.
5. `SwapCore` caps the buyer's gain at `poolCollateralBacking`, transferring the entire reserved LP amount to the buyer.

M-11 Morpho APY blend overquotes BUY_FLOATING

CONTINUED

RECOMMENDATION

Perform the mean-reversion blend and clamp in continuously compounded rate space:

```
spotCc = ln(1 + spotApy)
targetCc = annualized rateAtTarget
cc(T) = clamp(w(T) * spotCc + (1 - w(T)) * targetCc,
targetCc / CURVE_STEEPNESS,
targetCc * CURVE_STEEPNESS)
```

Return `cc(T)` directly from `RawMorphoBaseRateV1`. For the bumped fixed-side oracle, apply the tenor compounding transform to this rate-space blend. Do not blend endpoint APYs and then take one logarithm.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/MorphoRawApyBlendCanonicalJensenDrain.t.sol

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1031/>

MEDIUM **M-12**

ACKNOWLEDGED

Dust deposit consumes utilization bootstrap

CATEGORY Unexpected Behavior
 LOCATION contracts/SwapCore.sol:682-698
 REVIEW Main Review

DESCRIPTION

KAIRO tracks two liquidity amounts. Actual pool collateral determines whether the pool can support a swap. Seasoned collateral determines the utilization fee charged to the buyer. A new LP deposit increases actual collateral immediately, but normally takes one hour to become fully seasoned. This delay prevents an LP from depositing temporary liquidity solely to lower the utilization fee for a swap.

A new market would otherwise begin with no seasoned collateral and charge the maximum fee. To avoid this cold start, `supplyCollateral()` fully seasons the first successful deposit whenever `utilAcc[marketId].lastUpdate == 0`. However, the function rejects only a zero deposit. It does not authenticate the first LP or require a meaningful bootstrap amount.

An unprivileged account can therefore deposit one token atom before the intended launch LP. For USDC, one atom is `0.000001 USDC`. This dust deposit receives the complete first-deposit privilege and permanently makes `lastUpdate` nonzero. If the intended LP then deposits 1,000,000 USDC in the same block, no time has elapsed, so none of that deposit enters `avgTotalCollateral`.

The pool now has enough actual collateral to accept a large swap, but only the attacker's atom counts when KAIRO calculates the utilization fee. The validation produced the following result for a 1,000,000-USDC `BUY_FIXED` swap in the intended 30-day Aave V3 USDC borrow market:

```
| Result | Clean market | Poisoned market |
|---|---|---|
| Utilization fee | 0.0004877% | 20.85% |
| All-in fixed rate | 4.6539% | 25.5034% |
| Required buyer collateral | 1,092.89 USDC | 5,989.06 USDC |
```

A buyer using limits from the clean quote reverts because the poisoned rate, markup and total input exceed those limits. A buyer who accepts the poisoned quote posts much more collateral and loses 5,577.701221 USDC more when both swaps settle at the same fair floating rate.

M-12 Dust deposit consumes utilization bootstrap

CONTINUED

This requires an open-LP market that has not received its first deposit, separate market-creation and LP-seeding transactions and a buyer entering before the intended liquidity finishes seasoning. The attacker can withdraw the atom after the one-block withdrawal lock, but receives no victim funds. Withdrawing the atom does not change a fee already stored in a swap.

An immediate full pool exit and refill does not restore the bootstrap because `lastUpdate` remains nonzero. If no further updates occur, the intended liquidity becomes fully seasoned after one hour and normal pricing returns. The issue is therefore launch grief or buyer overpayment rather than direct theft.

— RECOMMENDATION

Do not let the first arbitrary public deposit initialize the utilization accumulator. Initialize the accumulator when the market is created so every public deposit follows normal seasoning. If immediate launch liquidity is required, allow only the market owner to perform an atomic bootstrap deposit and enforce a meaningful configured minimum before enabling public deposits and swaps. Do not automatically restore the privilege after a full pool exit.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/P1705UtilizationBootstrapPoisoning.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1056>

MEDIUM **M-13**

ACKNOWLEDGED

Queued tenor upgrade lets LPs capture new mark

CATEGORY MEV

LOCATION contracts/oracles/wrappers/UpgradableTenorRateOracle.sol:144-225

REVIEW Main Review

DESCRIPTION

`UpgradableTenorRateOracle._validateDownstreamHealth()` proves required-tenor coverage, single/batch consistency, positional ordering, decimals and raw/compounded quote shape. It never compares the candidate curve with the active curve. Runtime reads also ignore `pendingDownstream` until activation.

Consequently, a public queue can reveal a healthy but economically discontinuous replacement while `SwapCore.supplyCollateral()` continues to mint only against the outgoing mark. An ordinary LP can deposit before activation, receive excess shares, let the exact three-day wrapper delay outlast the default 12-hour profit vest and burn after activation at the replacement mark. The finite vest delays a persistent step but does not preserve the step for incumbent shares.

With one 10,000,000-USDC, one-year cumulative `BUY_FIXED` swap halfway to maturity, a 10% to 5% continuously compounded replacement raised the share price after a 2,200,000-USDC attacker deposit. The attacker withdrew 2,336,433.835955 USDC and gained 136,433.835955 USDC. Activating before the same cloned deposit returned principal within one atom. The incumbent lost the attacker's gain within one atom. The reverse 5% to 10% transition produced 127,911.837276 USDC of ordering-specific `BUY_FLOATING` gain after subtracting ordinary control carry.

Both old and new downstreams remained nonnegative, valid at every requested tenor, same-interface, 18-decimal, batch-consistent and quote-shape-attested. Therefore this is separate from semantic-invalidity finding P17-06. Exploitation requires a trusted owner to queue and activate a normal discontinuous migration, an open or legitimately whitelisted LP market, enough available collateral, attacker capital and a favorable step that persists through vest. No live queued deployment was established.

RECOMMENDATION

M-13 Queued tenor upgrade lets LPs capture new mark

CONTINUED

At activation, compare active and candidate quotes at one timestamp for every live required and remaining tenor. Reject replacements outside an explicit small economic-continuity tolerance. For intentional larger migrations, freeze LP mint/burn before publishing the candidate and route new liquidity to a new market/oracle epoch, blend the curves continuously or reserve the mark delta for shares outstanding before the transition. Extending the finite profit vest alone is not sufficient because a persistent step becomes withdrawable when the vest ends.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/P1708QueuedTenorUpgradeLpTiming.t.sol

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-14**

ACKNOWLEDGED

Bucket averages break compounded accrued NAV

CATEGORY Math
LOCATION `contracts/lib/SwapFormulas.sol:727-793`
REVIEW Main Review

DESCRIPTION

`calculateBucketUnrealizedPnL` first replaces every swap in a bucket with one `avgFixedRate` and one `avgEntryTime`. It then passes those averages to `accruedFixedLeg` when `compoundAccruedBase` is enabled. This is not a valid aggregation. Compounded fixed accrual is exponential in both the stored rate and elapsed time, so compounding the averages does not equal the sum of each swap's compounded accrual.

The affected calculation supplies the accrued-only price used by `KairosMarketAdapter` for cumulative `BUY_FIXED` markets. A downward error makes the adapter report a false loss. A new vault depositor can mint shares at that depressed value and retain part of the correction when the swaps settle from their individual rates and entry indices.

The production PoC places two one-year swaps in the same permitted bucket at 15% and 10% continuous rates. Their exact summed accrued PnL is zero. Production nevertheless marks a \$15,000,000 adapter position at \$13,761,412, understating it by \$1,238,587. After exact settlement removes the error, a timed depositor redeems \$460,816 more than their deposit and the incumbent loses the same amount. The depositor keeps \$348,415 after the vault's maximum 2% force-deallocation penalty and a 5% annual capital-cost benchmark. No collateral cap binds and both swaps settle at zero independently.

RECOMMENDATION

Do not compound a bucket-wide average fixed rate over a bucket-wide average elapsed time. Calculate accrued fixed payments per swap or partition positions into cohorts that preserve the rate and entry-time inputs required by the nonlinear formula. Sum those accrued payments before applying the adapter share-price calculation and collateral limits. Add a differential invariant requiring bucket accrued-only PnL to equal the sum of per-swap accrued PnL for every cumulative `BUY_FIXED` bucket.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/CompoundedBucketAverageNavDilution.t.sol

M-14 Bucket averages break compounded accrued NAV

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1048>

MEDIUM **M-15**

ACKNOWLEDGED

BUY_FLOATING averages invent accrued losses

CATEGORY Math

LOCATION `contracts/lib/Utils.sol:1624-1641, contracts/lib/SwapFormulas.sol:727-833`

REVIEW Main Review

— DESCRIPTION

For cumulative `BUY_FLOATING`, each swap's accrued fixed payment is `notional * baseRate * (now - entryTime) / YEAR`. The bucket does not preserve the `baseRate * notional * entryTime` cross-moment needed to sum that expression. It stores `weightedLpRate` and `weightedEntryTime` independently, then `calculateBucketUnrealizedPnL` multiplies their averages:

```
totalNotional * avgBaseRate * (now - avgEntryTime) / YEAR.
```

This differs from `sum(notional_i * baseRate_i * (now - entryTime_i) / YEAR)` whenever base rates and entry times are correlated. The code already stores `weightedFeeTime` specifically to eliminate the identical fee/time covariance error, but has no corresponding `weightedLpRateTime` term. The cumulative floating-index leg is harmonically aggregated and remains exact, so it does not cancel the fixed-leg error.

The error becomes a whole-NAV defect in the accrued-only branch introduced for `KairosMarketAdapter`. That branch drops the remaining fixed leg that would otherwise cancel the covariance in fair pricing. `getPoolSharePriceVirtualConservative()` can therefore select a synthetic loss that no individual swap has. Vault V2 mints new shares against the understated adapter value and the new shareholder retains part of the correction when individual settlement removes the bucket error.

The production PoC places two equal-notional, 90-day cumulative `BUY_FLOATING` swaps in the same maximum legal bucket at 5% and 15%. Each swap has exactly zero accrued PnL when reconstructed from its own stored base rate, entry time and cumulative entry index. Production nevertheless reports a 2,054,800.862508 USDC loss on a 15,000,000 USDC adapter. That amount matches the omitted covariance term, all collateral limits remain slack and both swaps later settle at zero independently. A timed Vault V2 depositor gains 863,597.604003 USDC at the incumbent's expense and retains 668,422.106757 USDC after the maximum 2% force-deallocation penalty and a 5% annual capital-cost benchmark.

The three authoritative `BUY_FLOATING` configurations all use cumulative references and raw/simple base-rate quotes, including the allowlisted `RawMorphoBaseRateV1`. They therefore reach the straight-line accrued basis used in the PoC; compounding is not required.

M-15 BUY_FLOATING averages invent accrued losses

CONTINUED

— RECOMMENDATION

Preserve `sum(baseRate_i * notional_i * entryTime_i)` for simple accrued fixed legs and calculate accrued fixed payment as `(weightedLpRate * now - weightedLpRateTime) / (YEAR * WAD)`, with signed arithmetic and deliberate rounding. Alternatively, value accrued payments per swap or partition positions into cohorts that retain the required cross-moment. Apply the same representation to storage and virtual bucket copies.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/BuyFloatingAccruedCovarianceNavDilution.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1048>

MEDIUM **M-16**

ACKNOWLEDGED

Accrued-only buckets ignore fee-time covariance

CATEGORY Math

LOCATION `contracts/lib/SwapFormulas.sol:727-754,775-797,835-865`

REVIEW Main Review

DESCRIPTION

`Utils.addSwapToBucket` records `weightedFeeTime`, which contains the information needed to calculate how much fee income each swap has earned since it was opened. The normal fair-value calculation uses this value correctly when it removes fees that have not yet accrued.

The accrued-only calculation does not. It sets `timeRemaining` to zero and therefore skips the later `weightedFeeTime` adjustment. Instead, it calculates accrued fees using the bucket's average fee and average entry time:

```
totalNotional * averageFee * (now - averageEntryTime) / (YEAR * WAD) .
```

This is not equivalent to summing the fees earned by the individual swaps:

```
sum(notional_i * fee_i * (now - entryTime_i)) / (YEAR * WAD) .
```

The difference depends on how each swap's fee relates to its entry time. For example, an earlier high-fee swap combined with a later low-fee swap makes the accrued-only value too low. Reversing their order can make it too high. The error grows with the bucket's total notional, the difference between the fees and the time between the entries. It is not limited to token rounding.

This issue occurs when at least two swaps share a bucket but have different utilization fees or risk premiums and different entry times. All six scoped cumulative configurations can reach this state because utilization and risk-premium values can change between swap entries. A user does not need to control an oracle. They can observe an existing bucket and deposit or redeem while its accrued-only value is wrong. Creating the condition deliberately would require funding the relevant buyer collateral.

M-16 Accrued-only buckets ignore fee-time covariance

CONTINUED

`KairosMarketAdapter._realAssets` reports the lower of the fair value and the accrued-only value to Vault V2. If the accrued-only value is understated, a new depositor receives too many vault shares. Exact settlement later restores the missing value, allowing the new depositor to capture part of the incumbent shareholders' assets. If the accrued-only value is overstated, a user can redeem too much and leave the later correction to the remaining shareholders.

The production PoC uses two equal-notional, 90-day cumulative `BUY_FLOATING` swaps in one legal bucket. Both store the same 25% base rate. The earlier swap pays a 20% risk premium and realizes a 5% floating reference rate; the later swap pays no premium and realizes 25%. Each swap therefore has approximately zero accrued PnL and settles at approximately zero independently. Despite that, production reports adapter assets of 35,890,423.642821 USDC instead of 40,000,019.025876 USDC, inventing a 4,109,595.383055 USDC loss. All collateral caps remain slack.

The canonical Vault V2 mints a timed depositor shares at that understated value. After exact settlement and the configured maximum 2% force-deallocation penalty, the depositor earns 1,627,121.140923 USDC gross and 1,085,999.200158 USDC after a 5% annual capital-cost benchmark. The incumbent loses 1,627,140.166800 USDC.

— RECOMMENDATION

Use `weightedFeeTime` when calculating accrued-only value. Calculate the base legs without the averaged utilization fee or risk premium. Then add the exact accrued fee amount from `weightedFees * currentTime - weightedFeeTime`, using full-precision arithmetic and an explicit rounding rule. The normal fair-value calculation and the accrued-only calculation should use the same helper for fee accrual.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/AccruedFeeTimeCovarianceNavDilution.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1048>

MEDIUM **M-17**

ACKNOWLEDGED

Live Aave floor cuts lock paired swap profit

CATEGORY Unexpected Behavior

LOCATION contracts/oracles/BaseRateAverageIndex/AaveBaseBorrowRateMinimum.sol:42-46

REVIEW Main Review

DESCRIPTION

`getMinimumWad()` reads Aave's current `baseVariableBorrowRate` on every quote. Both scoped Aave USDC borrow models then use that spot value as the minimum for their historical rate. `SwapCore.buySwap()` stores the resulting base quote without recording which Aave rate configuration produced it.

Aave V3 can update this reserve parameter in place. Its provider address and pool-wide strategy address do not change and neither KAIRO `UpgradableTenorRateOracle` has a pending activation. Consequently, a searcher can open `BUY_FLOATING` immediately before an announced floor reduction and equal-notional `BUY_FIXED` immediately after it. Both positions can store the same timestamp and shared Aave reference index. Their future realized reference legs cancel:

$$\begin{aligned} &\text{combined annual payoff} \\ &= \text{old floating base} - \text{new fixed all-in rate} \\ &- \text{floating utilization fee} - \text{floating risk premium} \end{aligned}$$

The trade locks a profit whenever the floor reduction is larger than both positions' markups. It does not require the searcher to predict the rate realized over the swap term.

The scoped one-day PoC reduces Aave's rate data from 5% to 2% without changing the Aave strategy address, either KAIRO model or either KAIRO wrapper. With 1,000,000 USDC in each LP pool, a 20,440,000,000 USDC pair locks a 1.4399408208% annual spread after utilization and risk markups. An arbitrary reachable 3% maturity rate produces 806,366.859668 USDC of aggregate LP loss and 582,366.859669 USDC of attacker profit after 224,000 USDC of protocol fees. The floating pool loses 616,000.000001 USDC; the fixed pool loses 190,366.859668 USDC.

The required Aave reduction must exceed the configured markups. The attacker also needs two-sided KAIRO liquidity and enough temporary capital for both positions.

M-17 Live Aave floor cuts lock paired swap profit

CONTINUED

— RECOMMENDATION

Do not consume Aave's live rate parameter as an instantly executable floor. Cache it in a KAIRO-controlled pair-level configuration. Before adopting a changed floor, permanently stop new entries in both old markets and create a fresh paired market epoch for new swaps. Existing positions can continue settling against the old market pair.

If markets must remain reusable, enforce a bounded continuous transition whose maximum cross-transaction quote movement cannot exceed the combined markups. Merely updating both sides atomically is insufficient because searchers can place opposite swaps before and after that transaction.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/AaveMinimumStrategyUpdatePairedDrain.t.sol

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-18**

ACKNOWLEDGED

Morpho supply index can be clamped

CATEGORY	Oracle
LOCATION	CumulativeIndexSupplyMorpho.sol#L46-L47 RateIndexLib.sol#L89-L98
REVIEW	Main Review

DESCRIPTION

`CumulativeIndexSupplyMorpho.getCumulativeIndex()` exposes the Morpho supply share exchange rate as an `ICumulativeRateOracle` index. This lets it be used as the `referenceRateOracle` for a `RateConvention.Cumulative` market.

The problem is that the Morpho supply share price is not strictly cumulative. When Morpho bad debt is realized, the loss is socialized to suppliers by reducing market supply assets while supply shares remain outstanding. As a result, the value returned by `CumulativeIndexSupplyMorpho.getCumulativeIndex()` can materially decrease.

Kairos does not reject this decrease on the settlement path. `RateIndexLib.update()` assumes a `RateConvention.Cumulative` source should be monotonic, so if the live value is below the last recorded index, it clamps the live value back up to that floor.

For example, if `RateIndexLib.update()` records a Morpho supply index `X`, bad debt later reduces the live index to `Y < X`, and a swap settles afterward, settlement still uses `X`. The Morpho supplier loss is therefore treated as flat index growth instead of a negative move.

This can misprice already-open swaps. The decreased Morpho supply index is not reflected in the floating leg because it is clamped away. For `BUY_FIXED`, this can benefit the buyer because the buyer receives an overstated floating leg. For `BUY_FLOATING`, this can benefit LPs because the buyer pays an overstated floating leg instead of receiving the negative floating performance that the real index decrease would imply.

The matching base-rate oracle does not fully prevent this. `IndexBaseRateV1` and `RawIndexBaseRateV1` can return invalid if they personally observed the higher pre-loss value through `poke()`. However, they keep their own high-water, separate from `RateIndexLib`. If `RateIndexLib.update()` observed the higher value but the base oracle missed it, the base oracle can continue quoting while settlement remains clamped to the higher reference floor.

Although current planned deployments use `CumulativeIndexBorrowMorpho`, `CumulativeIndexSupplyMorpho` is still an in-scope adapter and can currently pass market creation checks as a `RateConvention.Cumulative` `referenceRateOracle`.

M-18 Morpho supply index can be clamped

CONTINUED

— RECOMMENDATION

Reconsider how `CumulativeIndexSupplyMorpho` is intended to be used in `RateConvention.Cumulative` markets, especially when it is used as the settlement `referenceRateOracle`.

The final design should explicitly handle material Morpho supply losses instead of letting `RateIndexLib.update()` silently clamp them as flat cumulative growth. Depending on the intended product behavior, this could mean restricting `CumulativeIndexSupplyMorpho` from settlement usage, adding material-loss detection, pausing impaired markets for governance resolution, or implementing loss-aware settlement semantics.

Once the intended usage is decided, market creation should enforce the chosen compatibility rules instead of relying on operator discipline.

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1095>

MEDIUM **M-19**

ACKNOWLEDGED

Realized pool gains are retroactively seasoned

CATEGORY Logical Error
LOCATION contracts/lib/Utils.sol:630-706,1392-1404
REVIEW Main Review

DESCRIPTION

The utilization accumulator must be accrued with pre-mutation pool collateral so newly added liquidity receives no residence credit for time before it entered the pool. `accrueUtilAvg()` documents this ordering explicitly and LP supply/withdrawal follow it. Buyer-owes settlement and buyer liquidation do not: they add settlement collateral to `pool.totalCollateral` without first accruing the accumulator.

If `utilAcc.lastUpdate` is at least one hour old, the next same-block buy calls `projectUtilAvg()` with the post-settlement pool value. The full-window branch returns that value directly, treating the entire newly realized gain as though it had resided in the pool throughout the preceding hour. This increases seasoned capacity and lowers the target buyer's utilization fee. The Core comment classifies the credit as minor and non-attacker-controllable, but a buyer can early-exit its own losing swap and immediately backrun the close with another buy.

The intended 30-day Aave-borrow parameters reproduce material extraction. Starting with a 1,000-USDC seasoned pool, a buyer opens a 220,880-USDC sacrificial `BUY_FIXED` swap, waits one hour and exits after the reference rate falls. The close pays 144.437826 USDC into the pool. An immediate 800,000-USDC buy is charged a 0.4103912928361287% utilization rate. A same-state control that performs the documented pre-gain accrual charges 0.6903068817739351%. At maturity, the production buyer retains 184.054086 USDC more and the LP pool receives exactly 184.054086 USDC less. The pricing advantage exceeds the sacrificial pool payment by 39.616260 USDC before gas and financing and all amounts scale with pool and trade size.

A buyer or searcher can therefore place a buyer-owes close immediately before a large buy and pay less for that buy. The undercharge becomes a matching terminal revenue loss for non-consenting LPs. At sufficient scale, the buyer's gain can exceed the cost of creating the realized pool gain.

M-19 Realized pool gains are retroactively seasoned

CONTINUED

The sequence requires approximately one hour without a utilization-accumulator update, a buyer-side close that adds material collateral and enough spot capacity for a fee-sensitive target buy. The buyer must fund both swaps and bears rate, financing, ordering and execution risk. These conditions limit how often the sequence is profitable, but they do not require privileged access: early exit is buyer-controlled in the authoritative market rows, while normal settlement and liquidation can also be permissionlessly ordered.

This finding starts from a correctly seasoned pool and is caused by missing hooks on settlement/liquidation PnL transitions.

Example:

1. Let a correctly initialized market go one hour without a utilization-accumulator update.
2. Close a losing attacker-owned swap, adding buyer collateral to the LP pool.
3. In the same block, buy a large swap that fits the enlarged spot capacity.
4. The fee path projects the post-gain pool through the entire stale interval and undercharges the target swap.
5. At target settlement, the buyer retains the undercharge and LPs receive less terminal value.

— RECOMMENDATION

Centralize every `pool.totalCollateral` mutation behind a helper that first calls `accrueUtilAvg(utilAcc[marketId], pool.totalCollateral, UTIL_AVG_WINDOW)` with the pre-mutation value, then applies the gain or loss. Pass the market accumulator into settlement and liquidation or perform the accrual in their `SwapCore` callers immediately before those functions mutate pool value. Preserve the existing post-loss `min(EMA, spot)` clamp.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/KAIR0H020PoolGainSeasoning.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1077>

MEDIUM **M-20**

ACKNOWLEDGED

Top-ups can shorten active LP profit vests

CATEGORY Unexpected Behavior
LOCATION contracts/SwapCore.sol:663-675
REVIEW Main Review

DESCRIPTION

`supplyCollateral()` stores one aggregate `LpPosition` for all of an LP's shares. Its comments say that `vestEndTimestamp` "always extends", but every successful supply unconditionally replaces the deadline with `block.timestamp + admin.lpProfitVestSeconds()`. It does not compare that timestamp with the active position's existing deadline.

This lets an LP shorten the vest protecting all of its old shares after the Admin legitimately reduces the global duration. The LP only needs to add enough collateral to mint one share. Once the new, shorter deadline passes, `withdrawCollateral()` no longer applies the old entry-price cap, so the LP can remove profit that should still be protected for the remaining pool.

The intended 30-day Aave-borrow configuration was tested with two LPs supplying 1,000,000 USDC each, a 500,000,000-USDC `BUY_FIXED` swap, an authorized seven-day-to-one-hour duration change and a one-USDC top-up. The top-up moved the deadline from `2201112400` to `2200522000`, 590,400 seconds earlier. After one hour, the production withdrawal paid 1,002,319.773210 USDC. Subtracting the top-up and comparing with the no-top-up branch's still-capped 1,000,000-USDC withdrawal gives 2,318.773210 USDC of captured protected profit. The other LP's claim fell by exactly the same 2,318.773210 USDC.

An ordinary LP can therefore transfer still-vested profit from co-LPs to itself after a routine, timelocked duration decrease. The amount taken from the remaining LPs scales with the attacker's existing position and the favorable mark available before the original deadline.

The sequence becomes available whenever governance reduces the duration while a large LP position remains under its old deadline. Extracting value also requires a favorable share-price move and enough immediately available pool collateral to withdraw. No Admin compromise is necessary: the owner performs the authorized policy change, then the LP uses public supply and withdrawal functions. Duration changes are expected to be infrequent, but every decrease can expose all still-vested aggregate positions.

Example:

M-20 Top-ups can shorten active LP profit vests

CONTINUED

1. An LP deposits under a seven-day vest while another LP supplies the same pool.
2. The Admin queues and later activates a one-hour duration.
3. Before the old deadline, the first LP supplies one USDC, replacing the entire position's deadline with `now + 1 hour`.
4. A favorable active-swap mark exists when the short deadline passes.
5. The LP withdraws all old and new shares uncapped, reducing the remaining LP's claim by the released protected profit.

— RECOMMENDATION

When a nonzero position is still vested, set the new deadline to `max(lpPos.vestEndTimestamp, block.timestamp + admin.lpProfitVestSeconds())`. If governance must allow old protection to be shortened, represent deposits as separate vest tranches and apply the new duration only to newly minted shares.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/KAIR0H015VestDurationShortening.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1076>

MEDIUM **M-21**

ACKNOWLEDGED

Direct settlement blocks adapter recovery

CATEGORY	DoS
LOCATION	contracts/SwapCore.sol:1420-1430; contracts/adapters/KairosBuyerAdapter.sol:396-425
REVIEW	Main Review

DESCRIPTION

`SwapCore.makePayment()` intentionally lets any account settle an expired swap. If an arbitrary account uses it for a tracked BuyerAdapter position, the resulting payout is transferred to the adapter and the Core position becomes settled.

The Vault's permissionless `forceDeallocate()` escape hatch can no longer recover that payout.

`KairosBuyerAdapter.deallocate()` detects the settled position but rejects every `ACTION_MAKE_PAYMENT` call carrying the `forceDeallocate` selector before forwarding any tokens. The restriction exists because the same branch also removes tracking, even though forwarding an already-recorded payout does not require that removal.

Any account can trigger this state once an adapter-owned swap is eligible for settlement. The Vault assets then remain unavailable to shareholders until an allocator or sentinel acts. The funds are still recoverable by those roles, so the practical duration of the lock depends on their availability, but a permissionless caller can remove the depositor recovery route precisely after the economic operation is complete.

The PoC opens a real adapter-owned swap, settles it through Core from an unrelated account and confirms that the payout is held by the adapter. A depositor's subsequent `forceDeallocate(... ACTION_MAKE_PAYMENT ...)` call reverts with `RemoveSettledNotAllowedViaForceDeallocate` without moving the payout. The same data succeeds only when a configured allocator calls ordinary `deallocate()`.

RECOMMENDATION

Consider adding a dedicated force-safe settled-payout selector that never mutates tracking or allocation membership beyond the exact cash transfer.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_POCS/HYP005BuyerAdapterStateMachine.t.sol

M-21 Direct settlement blocks adapter recovery

CONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1075>

LOW **L-01**

ACKNOWLEDGED

Market creation accepts wrong-tenor risk premium

CATEGORY Validation
LOCATION contracts/lib/Utils.sol:244-247
REVIEW Main Review

DESCRIPTION

`probeOraclesForMarketCreation` receives `swapTerm`, but it only calls the risk premium oracle's parameterless `getRate()`. The configured `UpgradableSidedPremiumOracle` implements that call by reading its downstream curve at the wrapper's immutable `defaultTenorSeconds`. Neither `createMarket` nor the oracle probe requires that default to equal the new market's term. `buySwap` later repeats the same parameterless read and stores the returned premium in the position.

Consequently, a healthy wrapper can silently price a market at the wrong point on the premium curve. The supported deployment tooling accepts wrapper tenor and market term independently, then forwards the two wrapper addresses without comparing their immutable defaults to `swapTerm`. For example, the proof of concept creates a 90-day pair with 30-day wrappers over a curve that quotes 1% at 30 days and 4% at 90 days. Market creation succeeds and both `BUY_FIXED` and `BUY_FLOATING` store the 1% value.

RECOMMENDATION

Make the premium lookup tenor-explicit. Add a tenor-aware function to the sided wrapper and pass `market.swapTerm` from both market creation and swap entry instead of relying on `getRate()`.

If backward compatibility requires the parameterless interface, require `defaultTenorSeconds() == swapTerm` when a market uses a sided premium wrapper. Also verify `isUpperSide() == true` for `BUY_FIXED` and `false` for `BUY_FLOATING`. Add the same checks to the deployment preflight and include a regression test that rejects a 30-day wrapper for a 90-day market.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Escrow cannot bypass a blocked recipient

CATEGORY DoS
LOCATION contracts/SwapCore.sol:503-520
REVIEW Main Review

DESCRIPTION

`claimEscrow()` derives the recipient only from `swap.userAddress`. It clears `escrowedCollateral[swapId]` and transfers the full amount to that same address. The caller cannot select another receiver.

This prevents recovery when settlement fails because the token rejects one recipient while still accepting transfers from `SwapCore` to other addresses. Settlement stores the buyer's full entitlement in `escrowedCollateral` and marks the swap as settled. Every later claim retries the rejected recipient. The failed transfer rolls the escrow clearing back, but no call can change the receiver.

The restriction is permanent for integrated positions. A wrapped swap keeps the wrapper as `swap.userAddress` even after its NFT changes owners. The core rejects `transferSwapPosition()` after settlement, and the wrapper rejects unwrapping. A buyer-adapter swap likewise keeps the adapter as its recipient. The adapter can forward funds to `parentVault` only after it first receives them, so recipient blocking prevents that forwarding step.

An arbitrary account can call `makePayment()` once settlement is available and crystallize this state. In the shared POC, a 100,000 USDC-like position stored the full 15.169432 USDC buyer entitlement in escrow. Neither the current NFT holder nor the parent vault could receive it while the wrapper or adapter remained blocked. Fork tests confirmed that canonical Base and Ethereum USDC can blacklist either integration address independently while allowing the same `SwapCore` sender to transfer to an ordinary holder or parent vault. The recorded Base KAIRO markets currently use development tokens, so exposure of a presently deployed real-value market was not established.

RECOMMENDATION

Add an authorized `claimEscrowTo(bytes32 swapId, address recipient)` function. Authenticate the caller against the existing `swap.userAddress`, reject the zero address, clear escrow before transferring, and rely on transaction rollback to restore escrow when the alternate transfer fails. Keep `claimEscrow()` as a backwards-compatible call that selects `swap.userAddress`.

L-02 Escrow cannot bypass a blocked recipient

CONTINUED

The wrapper should expose this function only to the current NFT owner and pass the owner's chosen receiver directly to `SwapCore`. The buyer adapter should always select its immutable `parentVault`; permissionless execution is safe when callers cannot change that destination.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_POCs/CAND006EscrowRecipientBlock.t.sol

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Aave upgrade can make index flash-controlled

CATEGORY Unexpected Behavior

LOCATION contracts/oracles/cumulativeIndex/CumulativeIndexSupplyAave.sol:29-52

REVIEW Main Review

DESCRIPTION

`CumulativeIndexSupplyAave` checks `FLASHLOAN_PREMIUM_TO_PROTOCOL()` only when it is constructed. `getCumulativeIndex()` later accepts every positive `getReserveNormalizedIncome()` result without confirming that the pool still routes all flash-loan premium to treasury.

This matters because the configured pool address is an upgradeable Aave proxy. Historical Aave V3 implementations allowed governance to route part of the premium to suppliers and cumulated that amount into the liquidity index after the borrower's callback. If a future non-malicious Aave upgrade restores that behavior, an ordinary borrower can pay for a precisely timed permanent index increment while the adapter continues to return `isValid = true`.

KAIRO reads cumulative indices again within the same block. Therefore the borrower can place swap entry, LP share changes, liquidation or maturity settlement on either side of the increment. The direct validation opened a production-code `BUY_FIXED` swap before the external condition changed, applied ordinary index growth to maturity, completed a 33,084,235.190976 USDC flash under the official historical Aave premium formula and then settled through production `SwapCore`. At 500,000,000 USDC notional, 10x leverage, a 200,000 USDC LP cap, 5 bp Aave fee and a 50/50 supplier split, the index-linked receipt was 22,594.882356 USDC. After 16,542.117595 USDC flash cost, 3,257.646838 USDC utilization/risk markup, 958.904109 USDC protocol fee and 46.287231 USDC collateral opportunity cost, the attacker retained 1,789.926583 USDC. The LP's withdrawal fell by the full 22,594.882356 USDC relative to the cloned no-flash control.

RECOMMENDATION

Recheck `FLASHLOAN_PREMIUM_TO_PROTOCOL()` in `getCumulativeIndex()` and return `(0, false)` unless it is exactly `10_000`. Also monitor the Aave pool implementation and halt or redeploy the affected markets before accepting a pool upgrade. The already deployed Base adapter predates the constructor check and must be replaced if this guard is intended to protect the live configuration.

L-03 Aave upgrade can make index flash-controlled

CONTINUED

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCs/HYP002AaveSupplyDirectEconomics.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1028>

LOW **L-04**

ACKNOWLEDGED

Bucket tenor averaging misprices LP shares

CATEGORY Math
LOCATION `contracts/lib/Utils.sol:1771-1820`
REVIEW Main Review

DESCRIPTION

`calculateTotalUnrealizedPnL` reduces each active bucket to its average entry time, requests one base-rate quote at the average remaining tenor and passes that quote to `calculateBucketUnrealizedPnL` together with the bucket's harmonic entry index and other averaged values. A harmonic entry index is exact for the accrued cumulative-index ratio by itself. It is not exact after that ratio is multiplied by one remaining-tenor projection for swaps that entered at different times and indices.

Consequently, the bucket PnL can differ from the sum of the same fair-PnL formula applied to every swap. The discrepancy does not require either collateral cap. A test with two equal `BUY_FIXED` swaps, 20,000,000 USDC total notional, a one-year term and the analytical boundary spread `d = term / 24` measured bucket losses of 212.049395 USDC at a 20% continuous rate and 11,798.524944 USDC at a 100% continuous rate. The exact per-swap sum was within three USDC atomic units of zero in both cases.

The 100% case also produces a positive LP-share timing sequence with the reachable same-bucket spread `term / 24 - 1`. An entrant supplied 100,000 USDC against 2,000,000 USDC of incumbent collateral while the aggregate mark set the share price to 0.9941011775815. Exact settlement then removed the averaging error. After the 12-hour profit vest, the entrant withdrew 100,564.966550 USDC. Supplying after settlement returned exactly 100,000 USDC. The entrant retained 356.633217 USDC after charging 208.333333 USDC for 15.208 days of capital at 5% APR or 345.647249 USDC after additionally charging 10.985968 USDC for the measured 392,356 production-call gas at a conservative 7 gwei and 4,000 USDC/ETH benchmark. Protocol costs were zero and incumbent LPs lost 564.966550 USDC. The profitable sequence is opportunistic: manufacturing both one-year swaps would lock 1,909,202.031622 USDC of buyer collateral and incur at least 95,460.101581 USDC of capital cost before buy gas or protocol fees. The sequence remains exposed to rate movement; its measured break-even annual rate shock was about 2.83 percentage points. These prerequisites limit the severity.

RECOMMENDATION

L-04 Bucket tenor averaging misprices LP shares

CONTINUED

Do not use one average remaining tenor and one projected rate to price LP share mutations. Preserve per-swap valuation inputs or partition active positions into cohorts whose entry time, entry index and remaining-tenor quote can be valued independently. Sum those uncapped raw PnLs before applying the protocol's intended realizability constraints.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/HYP008ResidualBucketAggregation.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1018>

LOW **L-05**

ACKNOWLEDGED

Unpriceable swap blocks unrelated cash recovery

CATEGORY Warning
LOCATION contracts/adapters/KairosBuyerAdapter.sol:742-751
REVIEW Main Review

DESCRIPTION

`_deallocateReturn` recomputes the value of every tracked swap before any deallocation can commit. This couples recovery of realized token balances to the oracle health of unrelated live swaps because

`_realAssets` calls `SwapCore.getBuyerSettlementValue` for every remaining ID.

For example, swap A can settle successfully and leave its payout in `KairosBuyerAdapter`. If swap B remains live and a required reference, base or risk oracle read reverts, recovering A first transfers its payout to the Vault, records `_payoutForwarded[A]` and removes A from tracking. The final aggregate revaluation then reads B and reverts. Transaction atomicity restores all of those earlier changes, so A's cash remains in the adapter.

The same coupling applies to escrow claims and settled-swap cleanup. Official Morpho Vault V2 has no independent token rescue call. Its allowance pull runs only after `IAdapter.deallocate`, while `withdraw` and `redeem` also reprice every adapter. Consequently, realized cash can remain inaccessible for the duration of an unrelated oracle failure. A permanent oracle failure can make the lock permanent.

RECOMMENDATION

Add an isolated settled-cash recovery action that verifies the target swap is resolved, marks its payout as forwarded, removes its tracking entry and transfers only that realized amount to `parentVault` without calling `_realAssets`. Return empty IDs and zero change so canonical Vault V2 can commit the custody recovery without touching cap accounting. Add a separate reconciliation action that recomputes the adapter value and updates `lastReportedAllocation` once pricing is healthy again.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/P17_11_BuyerAdapterOracleRollback.t.sol

L-05 Unpriceable swap blocks unrelated cash recoveryCONTINUED

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Stale market creation spends unbounded live fee

CATEGORY Validation
LOCATION contracts/SwapCore.sol:182-215
REVIEW Main Review

DESCRIPTION

`createMarket` reads `CREATE_MARKET_FEE_AMOUNT` and `CREATE_MARKET_FEE_TOKEN` from `Admin` when the transaction executes. The caller supplies no expected fee token, maximum fee amount, fee configuration version or deadline. Therefore, calldata prepared while the fee is zero or small can remain valid after a queued fee activation and spend the new amount from any fee token for which the creator has already granted SwapCore enough allowance.

The three-day Admin timelock makes every increase public before activation. This gives an attentive creator time to replace a pending EOA transaction, revoke the allowance or avoid broadcasting a signed transaction. Exact allowances and insufficient `msg.value` also prevent excess spending. However, these are off-chain or token-level mitigations rather than properties enforced by `createMarket`. Repository deployment tooling grants SwapCore allowances as large as one billion tokens or `type(uint256).max`, so persistent allowance is a realistic precondition.

The trusted Admin's authority to select and activate the fee is not the issue. The user-protection failure is that one unchanged transaction does not state which fee the creator accepts. Impact is limited to the newly activated fee and the creator's balance/allowance. Exploitation additionally requires a stale transaction to remain executable across the public three-day notice period, so the likelihood is low.

RECOMMENDATION

Add `expectedFeeToken`, `maxFeeAmount` and `deadline` parameters to `createMarket`. Revert when `block.timestamp > deadline`, the live fee token differs from `expectedFeeToken` or the live fee exceeds `maxFeeAmount`. Keep the Admin timelock.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/P17_16_MarketCreationFeeBounds.t.sol

L-06 Stale market creation spends unbounded live feeCONTINUED

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1059>

LOW **L-07**

ACKNOWLEDGED

Force-deallocation rounding leaks to co-LPs

CATEGORY	Unexpected Behavior
LOCATION	contracts/adapters/KairosMarketAdapter.sol:565-575
REVIEW	Main Review

DESCRIPTION

For an exact partial withdrawal, `SwapCore.withdrawCollateral()` returns the requested assets but rounds the shares burned up:

```
shares burned = ceil(assets * WAD / share price)
```

The value represented by the rounded-up fraction is not returned to the withdrawing LP. It remains in the pool and increases the value of the remaining LP shares.

This becomes exploitable through permissionless `forceDeallocate()`. A caller who also owns shares in the same KAIRO pool can choose a withdrawal amount just below the value of an integer number of shares. The Vault loses the rounded excess, while the caller recovers it through their remaining LP shares.

`KairosMarketAdapter.deallocate()` calculates how much of this excess belongs to other LPs. However, it permits the withdrawal whenever `leak * 1000 <= assets`. The check therefore allows up to 0.1% of every forced withdrawal to be transferred from the Vault to the remaining LPs.

For example, suppose one raw share is worth 1 USDC and the adapter owns 1,000 shares. A forced withdrawal of 999.001 USDC rounds up to a burn of all 1,000 shares. The Vault receives 999.001 USDC after giving up 1,000 USDC of share value. The remaining 0.999 USDC stays in the pool and is captured by a caller who owns the other LP shares. This is the first amount accepted by the current tolerance; requesting one token atom less reverts.

The caller must own a meaningful portion of the remaining KAIRO shares and enough Vault shares to pay the force-deallocation penalty. At the maximum 2% penalty, the official Vault burns shares representing 19.980020 USDC but keeps those tokens inside the Vault. The PoC deposits temporary capital immediately after `forceDeallocate()`, allowing the caller to own most of the Vault while that retained balance is recognized. The caller recovers 19.960079 USDC of the penalty and earns 0.954693 USDC before gas. Profit after the measured Base transaction cost is 0.949719 USDC.

L-07 Force-deallocation rounding leaks to co-LPs

CONTINUED

The transfer is limited to approximately 0.1% of the forced assets and is highly capital-intensive. The validated example requires about 999,000 USDC of co-LP value for 12 hours and 1,000,025 USDC of temporary Vault capital to earn less than 1 USDC. Nevertheless, the loss is permissionless, repeatable and increases with the value of each raw share.

— RECOMMENDATION

Do not permit a relative value transfer to other LPs during `forceDeallocate()`. The safest solution is a share-denominated withdrawal that returns the full value of every share burned.

If a tolerance is necessary, use a small absolute amount based on the collateral token's decimals, preferably one atomic unit. Return any larger excess to the Vault or send it to a reserve that the caller and remaining LPs cannot capture. Retain the share-burn limit and test the boundary amounts, supported token decimals, high share prices, repeated calls and maximum Vault penalties.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/P17_10_ForceDeallocateRoundingTolerance.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1101>

LOW **L-08**

ACKNOWLEDGED

Wrapper activation accepts unusable rates

CATEGORY	Validation
LOCATION	contracts/oracles/wrappers/UpgradableTenorRateOracle.sol:144-225
REVIEW	Main Review

DESCRIPTION

`UpgradableTenorRateOracle._validateDownstreamHealth()` checks that a replacement supports the required tenors, returns valid and consistent single and batch quotes and preserves the wrapper's raw or compounded quote type. `UpgradableSidedPremiumOracle` only checks that its selected premium returns `isValid = true`. Neither wrapper verifies that the returned number is acceptable to the KAIRO market using it.

A cumulative reference index cannot decrease, so KAIRO requires every projected base rate for that convention to be non-negative. The PoC activates a fully compatible raw tenor oracle that returns `(-0.05e18, true)` or -5%. After the three-day delay, the wrapper accepts and forwards the quote. Fair LP share prices, LP value, deposits, withdrawals, early exits, adapter NAV and buyer settlement-value views then revert with `E512`. Accrued-only views remain available because they do not use the projected rate.

`SwapCore.buySwap()` also applies the negative cumulative-rate check only to `BUY_FIXED`. A direct `BUY_FLOATING` swap can therefore open at -5% and reserve 432.588321 USDC of LP backing, even though its projected views and early exit immediately revert with `E512`. Liquidation and ordinary maturity can still release the backing, so it is not permanently locked, but normal LP operations may remain unavailable until then or until the oracle is replaced.

The same mismatch affects premiums. A selected lower premium of `(-0.01e18, true)` passes wrapper activation, but the next `BUY_FLOATING` entry reverts with `E612` when KAIRO rejects the negative premium.

This requires the trusted owner to activate a future or misconfigured, interface-compatible oracle that reports a negative value as valid. The six currently named downstream implementations already prevent such values. The emergency cutoff only changes the failure to `E611`; restoring normal operation requires a healthy replacement to complete another three-day delay. The result is temporary loss of pricing and position-management functionality, not direct theft.

L-08 Wrapper activation accepts unusable rates

CONTINUED

— RECOMMENDATION

Bind each wrapper to an immutable semantic profile in addition to its interface and decimals. The profile should state the market rate convention, base versus premium role, selected premium side, quote type and permitted numeric interval. For cumulative base wrappers, reject any negative single or batch probe at activation and apply the same check to every runtime forwarded read. For premium wrappers, reject a negative selected-side probe and enforce non-negativity on runtime reads. Keep signed values available for conventions whose domain legitimately permits them rather than adding a global sign restriction. Independently move `Utils.requireNonNegativeCumulativeRate(baseRate, convention)` before the `BUY_FIXED / BUY_FLOATING` branch in `buySwap()` so both cumulative entry directions enforce the same rule. Retain `E512` and `E612` at downstream consumers as defense in depth. Add recovery tests covering cutoff, replacement delay and existing positions.

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1088>

LOW **L-09**

ACKNOWLEDGED

Protocol fee event overstates revenue

CATEGORY Events
LOCATION Utils.sol#L1562-L1576
REVIEW Main Review

DESCRIPTION

`distributeProtocolFee()` subtracts `creatorAmount` from `protocolAmount` after a successful creator-fee transfer and emits `CreatorFeeCollected`. However, when the remaining protocol transfer succeeds, it emits `ProtocolFeeCollected` with the original gross `protocolFee` instead of the net `protocolAmount` actually received by `protocolMultisig`. When `CREATOR_FEE_SHARE` is enabled, indexers that sum `ProtocolFeeCollected.feeAmount` will overstate protocol revenue and double-count the creator share.

RECOMMENDATION

Emit `ProtocolFeeCollected` with `protocolAmount`, or rename/document the event as the gross fee charged and add a separate event for the net protocol treasury amount.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1055>

LOW **L-10**

ACKNOWLEDGED

Morpho index can change unexpectedly

CATEGORY Oracle
LOCATION CumulativeIndexBorrowMorpho.sol#L45-L46
REVIEW Main Review

DESCRIPTION

`CumulativeIndexBorrowMorpho.getCumulativeIndex()` exposes Morpho's current `totalBorrowAssets` / `totalBorrowShares` ratio as a `Types.RateConvention.Cumulative` index. Unlike Aave's protocol-level accumulator returned by `getReserveNormalizedVariableDebt()`, this ratio is not a lifetime monotone accumulator.

If the Morpho borrow side is fully repaid while `lastUpdate` remains nonzero, `totalBorrowAssets` and `totalBorrowShares` can both become zero, causing `CumulativeIndexBorrowMorpho.getCumulativeIndex()` to return the virtual-offset baseline instead of the prior index. The analogous reset can occur in `CumulativeIndexSupplyMorpho.getCumulativeIndex()` after all borrows are repaid and all supply shares are withdrawn: `totalSupplyAssets` and `totalSupplyShares` become zero and the supply index returns the same baseline.

This again results in the issue described in M-18

The reset also creates a stale high-water epoch for either adapter. When borrowing or supplying restarts from the lower baseline, Kairos continues flooring the index to the previous epoch's high-water and observes no new growth until the new share ratio exceeds that high-water.

The same empty-market condition can also create an upward discontinuity, not only a reset downward. In an initialized but empty Morpho supply market,

`CumulativeIndexSupplyMorpho.getCumulativeIndex()` returns the virtual-offset baseline near `1e21`. If one atomic asset/share is then supplied, the $(\text{totalSupplyAssets} + \text{VIRTUAL_ASSETS}) / (\text{totalSupplyShares} + \text{VIRTUAL_SHARES})$ ratio can jump close to `2e21` without any interest accrual.

A similar upward non-interest movement can occur before the source fully empties. Ordinary Morpho supply-side rounding is supplier-favorable: exact-share withdrawals burn the requested shares but round returned assets down, leaving residue in `totalSupplyAssets`. If the source is thin or winding down and one actor is the last or dominant supplier, split exact-share

L-10 Morpho index can change unexpectedly

CONTINUED

withdrawals can reduce `totalSupplyShares` toward dust while retaining asset residue. The adapter then reports the higher share ratio as valid cumulative growth even when `totalBorrowAssets == 0` and no interest accrued.

Because `RateIndexLib.update()` intentionally rereads cumulative sources intra-block, two same-maturity positions can observe different settlement indexes in one block: one side settles against the baseline, then the Morpho dust supply moves the index upward, and the opposite side settles against the higher value. This is another consequence of using a raw, dust-sensitive Morpho share ratio as a settlement cumulative index.

— RECOMMENDATION

Consider adding minimum asset/share floors, explicit epoch handling, and source-level exposure checks for Morpho cumulative adapters.

`CumulativeIndexBorrowMorpho.getCumulativeIndex()` and `CumulativeIndexSupplyMorpho.getCumulativeIndex()` should return `(0, false)` when the relevant Morpho side falls below configured `minAssets` or `minShares` thresholds. Full zero-side transitions should be treated as source epoch boundaries instead of continuing to reuse the old `RateIndexLib` high-water for new exposure. In addition, Kairos should cap or pause aggregate market exposure using a Morpho source when the total locked backing or notional supported by that source becomes too large relative to the underlying Morpho market depth.

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1093>

LOW **L-11**

ACKNOWLEDGED

Extreme tenor gaps break interpolation

CATEGORY	DoS
LOCATION	contracts/oracles/BaseTimelockedOracle.sol:259–262; contracts/oracles/BaseTimelockedOracle.sol:1058–1063
REVIEW	Main Review

DESCRIPTION

The oracle constructor requires tenor buckets to be ordered but does not limit their magnitude or the size of adjacent gaps. Interpolation subtracts two `uint256` buckets and casts the result to `int256`. A gap above `type(int256).max` therefore becomes negative, so an ordinary positive query offset is divided by a negative range. This turns what should be interpolation between two bounded rates into signed extrapolation.

If such a curve is deployed or activated, an ordinary in-range market tenor can return a rate far outside the immutable oracle bounds while `isValid` remains true. Entry, NAV, liquidation, early-exit and settlement calculations may then consume the escaped rate and materially misprice a position or reduce LP backing. Creating the unsafe bucket geometry requires a privileged configuration mistake, but no further privilege is needed to trigger it once active.

The focused PoC uses the production oracle logic with buckets `[1, type(uint256).max]` and stored rates `[0, 1e18]`, both within the configured `[-1e18, 1e18]` bounds. An ordinary eight-hour query returns `-14,399.5e18` with `isValid = true` because the computed result is never checked against its endpoints or the active bounds.

RECOMMENDATION

Reject every bucket and adjacent difference that cannot be represented as a positive `int256`, both at construction and during bucket activation. Prefer unsigned interpolation with an explicit rate-difference sign branch and recheck the computed value is between its two endpoints and inside the active bounds before returning `isValid = true`.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/CurrentLedgerUniqueValidation.t.sol

RESOLUTION

Status: Acknowledged.

L-11 Extreme tenor gaps break interpolation

CONTINUED

<https://github.com/kairosswap/protocol/pull/1081>

LOW **L-12**

ACKNOWLEDGED

High-price mints enrich incumbent LPs

CATEGORY	Rounding
LOCATION	contracts/SwapCore.sol:610-617; contracts/adapters/KairosMarketAdapter.sol:381-402
REVIEW	Main Review

— DESCRIPTION

`SwapCore.supplyCollateral()` rounds `collateralAmount * 1e18 / sharePrice` down and rejects a deposit only when the result is zero. If one raw LP share represents a large amount of collateral, a deposit slightly below the value of two shares therefore receives only one share. The full deposit still enters the pool, so the existing shareholders immediately own their share of the rounding remainder.

An exact `minSharesOut` quote does not protect the depositor because that quote is already rounded down to one share. `KairosMarketAdapter.allocate()` makes the same loss possible through a Vault: it calls Core with `minSharesOut = 0` and checks the caller's minimum only after the deposit. A thin pool whose share price has been raised through ordinary buyer-loss settlements can therefore transfer a material part of a later deposit to an incumbent LP. The attacker must fund and season that pool and wait for a depositor to accept the coarse quote, but the loss can approach one raw share value as that value grows.

The Foundry PoC reproduces the original ledger witness: an incumbent leaves 10,002 token units behind one LP share through an ordinary buyer-to-pool settlement, after which a victim deposits 20,003 units and receives one share. The incumbent then withdraws 15,002 units, realizing 5,000 units taken from the victim's rounding remainder. A scalable case raises one raw share to 1,000 USDC through successful `supplyCollateral()`, `buySwap()` and `makePayment()` calls. The adapter then accepts 1,999.999999 USDC for one share, immediately reports only 1,499.999999 USDC and lets the incumbent realize 499.999999 USDC of profit. No storage mutation, transfer-tax token, post-entry oracle change or omitted slippage parameter is involved.

— RECOMMENDATION

Reject a deposit when its rounding remainder exceeds a small, explicit fraction of `collateralAmount` or require deposits to be a sufficiently large multiple of the current raw-share value. Apply the same check in `KairosMarketAdapter.allocate()` before Core pulls Vault assets. Keep `minSharesOut` as price-slippage protection, but do not rely on it as the only granularity check.

L-12 High-price mints enrich incumbent LPs

CONTINUED

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/CurrentLedgerUniqueValidation.t.sol

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1086>

LOW **L-13**

ACKNOWLEDGED

Core operators lose rights after wrapping

CATEGORY	Access Control
LOCATION	SwapPositionWrapper.sol#L17-L22
REVIEW	Main Review

DESCRIPTION

`SwapPositionWrapper` takes custody of the swap in `SwapCore`, so addresses authorized through `SwapCore.setAuthorization()` for the original `userAddress` no longer control that position after `wrap()`. This can break integrations that rely on `SwapCore.isAuthorized` operators continuing to manage positions, because wrapped lifecycle actions such as `unwrap()`, `exitEarly()`, and `redeem()` are gated by NFT ownership instead.

RECOMMENDATION

Document this permission-domain change explicitly in the user/integrator docs and UI. If delegated management is expected after wrapping, consider supporting ERC721-approved operators in wrapper lifecycle functions or adding wrapper-specific delegation.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1083>

LOW **L-14**

ACKNOWLEDGED

Rounded index ratio can brick settlement

CATEGORY DoS

LOCATION `<mention-page url="https://app.notion.com/p/e098bda5828c823e8a89819f586bc4e0"/>`
`[External.sol#L48-L59<br>](https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/e415e373265a60cbd76662cab78cee9cfedf80a4/packages/hardhat/contracts/interfaces/External.sol#L48-L59)`
`[ChainlinkAdapter.sol#L35-L52<br>](https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/e415e373265a60cbd76662cab78cee9cfedf80a4/packages/hardhat/contracts/oracles/adapters/ChainlinkAdapter.sol#L35-L52)`
`[PythAdapter.sol#L105-L142](https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/e415e373265a60cbd76662cab78cee9cfedf80a4/packages/hardhat/contracts/oracles/adapters/PythAdapter.sol#L105-L142)`

REVIEW Main Review

DESCRIPTION

`deriveRateFromIndex()` first converts the ratio between `closingFloatingIndex` and `entryFloatingIndex` to WAD precision:

```
int256 indexRatio =
safeCast(Math.mulDiv(closingFloatingIndex, WAD, entryFloatingIndex));
```

For the `Types.RateConvention.SpotCompoundRate` convention, it then passes this rounded ratio to `FixedPointMathLib.lnWad()`.

If a sufficiently negative cumulative rate moves the closing index close to `MIN_INDEX`, the WAD-scaled ratio can truncate to zero even though both indices remain strictly positive. For example, with an entry index of `1e27` and a closing index of `1e6`, the calculation is:

$$1e6 \times 1e18 / 1e27 = 0$$

Consequently, `lnWad(0)` reverts.

The documentation of `IRateOracle` explicitly requires adapters backing `Types.RateConvention.SpotRate` or `Types.RateConvention.SpotCompoundRate` markets to clamp readings below a feed-specific negative floor. However, this requirement is not enforced.

`ChainlinkAdapter.getPriceWithMetadata()` scales and returns the signed Chainlink answer without applying a minimum rate. Similarly, `PythAdapter.getRate()` deliberately accepts negative prices and returns `scaledPrice` without applying the documented negative floor. Therefore, both adapters can return a valid but sufficiently negative rate that drives the index into the unsafe range.

An affected swap cannot be settled through `makePayment()`. The same calculation also prevents liquidation and mark-to-market operations. Once the unsafe index is stored as the swap's expiry snapshot, subsequent oracle recovery does not replace that historical value. The swap remains at the expiry queue cursor, and the expired-unsettled gate blocks collateral supply and withdrawal for the market.

L-14 Rounded index ratio can brick settlement

CONTINUED

The impact is a potentially permanent market-level denial of service and locked LP collateral. The issue is rated Medium because reaching the unsafe range requires an extreme accumulated negative rate or a severely incorrect oracle reading.

— RECOMMENDATION

For `Types.RateConvention.SpotCompoundRate`, calculate the logarithmic return by subtracting the logarithms of the two positive indices instead of constructing a rounded WAD ratio:

```
int256 lnRatio =  
FixedPointMathLib.lnWad(safeCast(closingFloatingIndex))  
- FixedPointMathLib.lnWad(safeCast(entryFloatingIndex));  
return (lnRatio * int256(SECONDS_IN_YEAR)) / int256(timeElapsed);
```

Although the indices are RAY-scaled, this is valid because both use the same scale. `lnWad()` interprets each input using the same denominator, which cancels when the logarithms are subtracted.

Additionally, make the documented oracle requirement enforceable by adding configurable negative-rate floors to `ChainlinkAdapter` and `PythAdapter`. Validate the configured floor when creating a `Types.RateConvention.SpotRate` or `Types.RateConvention.SpotCompoundRate` market.

For complete accounting below `MIN_INDEX`, consider storing historical cumulative rate-time values and deriving the rate directly from their difference.

PROOF OF CONCEPT <https://github.com/GuardianOrg/protocol-team1-1783698575429/commit/7ca36122713dc9d71292ad16aa305033b61c656d#diff-b66c21c0ee72469efc98b15e2e9223db57e849ace70284ddd872d478a6496f4d>

— RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

Settled event can emit before credit

CATEGORY Events

LOCATION SwapPositionWrapper.sol#L158-L161
SwapPositionWrapper.sol#L192-L193

REVIEW Main Review

— DESCRIPTION

`settle()` and `settleBatch()` emit `PositionSettled` after calling `_creditKnownSettled()`, but `_creditKnownSettled()` intentionally returns without crediting while `SwapCore.isLockedd()` is true. During a callback-capable token transfer from a direct `SwapCore.makePayment()` call, reentering either wrapper settlement path can therefore emit `PositionSettled` with stale or zero `redeemable[tokenId]` before the final payout is recorded.

— RECOMMENDATION

Emit `PositionSettled` only when `_credited[tokenId]` is true after `_creditKnownSettled()` completes in both `settle()` and `settleBatch()`.

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1085>

LOW **L-16**

ACKNOWLEDGED

Floating NFTs show zero rate

CATEGORY Unexpected Behavior
LOCATION SwapPositionWrapper.sol#L433-L434
REVIEW Main Review

DESCRIPTION

`tokenURI()` always renders `swapRate` as the NFT's `Swap Rate`, but `SwapCore.buySwap()` intentionally stores `swapRate == 0` for `BUY_FLOATING` positions because their floating leg is derived at settlement. As a result, wrapped `BUY_FLOATING` NFTs report a zero rate even though their fixed/base rate is stored in `baseRate`, misleading users and metadata consumers.

RECOMMENDATION

Include enough market context to distinguish `BUY_FIXED` from `BUY_FLOATING`, and render `swapRate` for `BUY_FIXED` positions but `baseRate` for `BUY_FLOATING` positions.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1078>

INFORMATIONAL **I-01**

ACKNOWLEDGED

Natspec for minCollateral is incomplete

CATEGORY Documentation
LOCATION SwapCore.sol#L177
REVIEW Main Review

DESCRIPTION

The `createMarket()` NatSpec describes `minCollateral` as the minimum buyer collateral required per swap. However, `buySwap()` enforces `minCollateral` against both the required buyer collateral and the required LP collateral backing, and the surrounding type comments describe it as applying to both sides.

RECOMMENDATION

Update the `minCollateral` NatSpec to state that it applies to both buyer collateral and LP collateral backing per swap.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1083>

INFORMATIONAL I-01.2

ACKNOWLEDGED

Short TWAR windows can misprice long swaps

CATEGORY Informational

LOCATION <https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/580875df571a96f4568a30c3510ec217767c2054/packages/hardhat/contracts/oracles/BaseRateAverageIndex/IndexBaseRateV1.sol#L396-L410>

REVIEW Main Review

DESCRIPTION

`IndexBaseRateV1` and `RawIndexBaseRateV1` derive fixed quotes from a trailing realized window through `_realizedTwar()` and `getRateByTenor()`, but the utilization of the referenced lending market can be influenced by traders. If `twarWindowSec` is short relative to the supported `swapTerm`, a trader can pay to move utilization during the TWAR window and cause the resulting fixed quote to reflect that short-lived regime.

The observed source interest is real, not artificial, and any profit still depends on future floating accrual. However, the market configuration should account for the cost of moving the underlying utilization. The original Team2 example used `CumulativeIndexBorrowCompound`, but the configuration concern is not Compound-specific.

RECOMMENDATION

Calibrate `twarWindowSec`, maximum `swapTerm`, and Kairos exposure caps against the capital and time required to move the referenced lending market's utilization. Consider longer TWAR windows, source-depth-based exposure caps, or deviation checks for long-tenor markets that reference utilization-sensitive lending sources.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Lib ignore pattern hides source folders

CATEGORY Informational
LOCATION .gitignore#L22
REVIEW Main Review

DESCRIPTION

The bare `lib` pattern ignores every nested `lib` directory under `packages/hardhat`, including source folders such as `contracts/lib` and `contracts/dev/lib`. Existing tracked files remain tracked, but newly added files in these folders will be suppressed from `git status` by default and can be excluded from editor search, quick-open, file watchers, and gitignore-aware review tooling. This creates a real risk that new source files are missed during development or review.

RECOMMENDATION

Anchor the Foundry dependency ignore to the package root by replacing `lib` with `/lib/`.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/commit/fc844b2d225c509d080267465139b5f61d762dee>

INFORMATIONAL **I-03**

ACKNOWLEDGED

Unused owner on SwapCore

CATEGORY Informational
LOCATION SwapCore.sol#L39
REVIEW Main Review

DESCRIPTION

SwapCore inherits Ownable2Step, but none of its protocol logic uses onlyOwner, _checkOwner(), or owner(). As a result, the inherited ownership functions such as transferOwnership() and acceptOwnership() only manage an owner field that has no authority over SwapCore, which can mislead integrators or reviewers about the contract's admin model.

RECOMMENDATION

Consider removing the unused Ownable2Step inheritance from SwapCore.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1060/changes>

INFORMATIONAL **I-04**

ACKNOWLEDGED

Zero owner transfer bypasses cancel event

CATEGORY Informational
LOCATION Admin.sol#L146-L149
REVIEW Main Review

DESCRIPTION

`Admin.transferOwnership()` allows `newOwner` to be `address(0)` because the inherited `Ownable2Step.transferOwnership()` permits zero-address pending owners. Calling `transferOwnership(address(0))` effectively cancels the pending owner without using `cancelOwnershipTransfer()`, skips the `OwnershipTransferCancelled` event, and leaves `pendingOwnerActivationTime` set to a stale nonzero value.

RECOMMENDATION

Reject `address(0)` in `transferOwnership()` and require callers to use `cancelOwnershipTransfer()` for cancellations.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1059>

INFORMATIONAL **I-05**

ACKNOWLEDGED

Pending fee updates cleared silently

CATEGORY Informational
LOCATION Admin.sol#L202-L208
Admin.sol#L300-L311
REVIEW Main Review

DESCRIPTION

`disableProtocolFee()` clears `pendingCreatorFeeShare` and `pendingCreatorFeeShareActivationTime`, but only emits a creator-fee event when `CREATOR_FEE_SHARE` was already nonzero. Similarly, `disableMarketCreationFee()` clears `pendingCreateMarketFeeToken`, `pendingCreateMarketFeeAmount`, and `pendingCreateMarketFeeActivationTime`, but only emits market-creation fee events based on the active fee state. If either fee update was queued while the active value was already disabled, the disable function cancels the pending update without an explicit event. Event-only monitors may continue to believe the queued fee update is pending.

RECOMMENDATION

Emit explicit cancellation events when `disableProtocolFee()` clears a pending creator-fee share update and when `disableMarketCreationFee()` clears a pending market-creation fee update, or emit corresponding zero-value queued events before clearing the pending state.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1085>

INFORMATIONAL **I-06**

ACKNOWLEDGED

The admin variable can be marked immutable

CATEGORY Informational
LOCATION SwapCore.sol#L49
REVIEW Main Review

DESCRIPTION

`SwapCore.admin` is assigned once in the constructor and there is no function to update it afterward. Keeping it as a regular storage variable makes the admin binding look mutable and requires storage reads for each access, even though the value is intended to be fixed for the lifetime of the contract.

RECOMMENDATION

Mark `admin` as `immutable`.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1060/changes>

INFORMATIONAL **I-07**

ACKNOWLEDGED

Creation fee delivery can fail open

CATEGORY Documentation
LOCATION Utils.sol#L1605-L1607
REVIEW Main Review

DESCRIPTION

`deliverCreationFeeOrRefund()` treats failed delivery of the market creation fee to `protocolMultisig` as a foregone fee and refunds the payer. This is intentional for cases where the multisig cannot receive the token, but it also means market creation can succeed without collecting the configured fee. For example, a non-standard token with unusually expensive `transfer()` logic may allow a caller to use the 63/64 gas forwarding rule to make `tryTransferToken()` fail while leaving enough gas for the refund path. Similarly, a token pause or blacklist authority that can block transfers to `protocolMultisig` can cause the fee to be refunded instead of collected.

RECOMMENDATION

Document the assumptions for `feeToken`, including that it should have predictable-gas ERC20 transfer behavior and that token-level pause or blacklist controls can cause market creation fees to be foregone.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1083>

INFORMATIONAL I-08

ACKNOWLEDGED

Creation fee foregone event is incomplete

CATEGORY Events
LOCATION SwapCore.sol#L294-L296
REVIEW Main Review

DESCRIPTION

`createMarket()` emits `MarketCreationFeeCollected()` with both `fixedMarketId` and `floatingMarketId`, as well as `feeToken`, when the market creation fee is delivered. However, when delivery fails and the fee is foregone, it reuses `ProtocolFeeForegone()` and emits only `fixedMarketId`, `msg.sender`, and `feeAmount`. This makes the failure path less informative than the success path, even though the fee applies to the creation of both paired markets and may be denominated in either native ETH or an ERC20 token.

RECOMMENDATION

Emit a dedicated market-creation-fee foregone event that includes both `fixedMarketId` and `floatingMarketId`, the creator, `feeToken`, and `feeAmount`, matching the scope of `MarketCreationFeeCollected()`.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1079>

INFORMATIONAL **I-09**

ACKNOWLEDGED

Market ID omits two config flags

CATEGORY Documentation
LOCATION Utils.sol#L265-L287
REVIEW Main Review

DESCRIPTION

`generateMarketId()` hashes most market creation parameters together with `creator`, `timestamp`, and `nonce`, but it does not include `earlyExitAllowed` or `lpWhitelistEnabled`. Market IDs remain unique because `globalNonce` is included, but the ID is not a complete commitment to all market configuration.

RECOMMENDATION

Consider adding these fields as well.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1083>

INFORMATIONAL **I-10**

ACKNOWLEDGED

Global nonce comment is stale

CATEGORY Documentation
LOCATION SwapCore.sol#L70-L71
REVIEW Main Review

DESCRIPTION

The comment above `globalNonce` says it is used for generating unique swap IDs, but `globalNonce` is also used when generating both fixed and floating market IDs in `createMarket()`.

RECOMMENDATION

Update the comment to state that `globalNonce` is used for both market ID and swap ID generation.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1083>

INFORMATIONAL **I-11**

ACKNOWLEDGED

No-op bucket updates bypass rate checks

CATEGORY Documentation
LOCATION BaseTimelockedOracle.sol#L541-L554
REVIEW Main Review

DESCRIPTION

`activateTenorBucketsChange()` clears stored forecasts and resets `lastObservedAt` when a pending `tenorBuckets` update is activated. As a result, the next `postForecasts()` for the new bucket set is treated as a first post: `_validateRateVector()` skips the `maxRateChangeWad` check because the previous `updatedAt` is zero, and `_validateTenorsAndCadence()` skips the `minUpdateIntervalSec` cadence check because `lastObservedAt` is zero. For example, if a no-op `tenorBuckets` update is queued and activated, the same bucket set can receive any in-bounds forecast values without the normal continuity checks.

RECOMMENDATION

Reject no-op `tenorBuckets` updates so the reset path cannot be used without changing the bucket set. Note that this does not fully prevent continuity checks from being bypassed, because real bucket changes still reset forecast history and make the first post for the new bucket set a fresh initialization.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1083>

INFORMATIONAL **I-12**

ACKNOWLEDGED

First snapshot equality is not short-circuited

CATEGORY Gas Optimization
LOCATION RateIndexLib.sol#L245
REVIEW Main Review

DESCRIPTION

`getIndexAt()` only short-circuits when `targetTime` is lower than the first stored snapshot timestamp. When `targetTime == snaps[0].timestamp`, the function later reaches the generic exact-match path and returns the same `snaps[0].indexValue`, doing unnecessary branching or binary-search work.

RECOMMENDATION

Change the first-snapshot check to `targetTime <= snaps[0].timestamp` so equality with the first snapshot returns immediately.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1085>

INFORMATIONAL I-13

ACKNOWLEDGED

Aave supply closeouts assume reserve activity

CATEGORY Math
LOCATION contracts/lib/Utils.sol:755-807
REVIEW Main Review

DESCRIPTION

`calculateEarlyExitSettlement` uses the remaining-tenor quote from `IndexBaseRateV1` to estimate the floating leg of an active Aave supply `BUY_FIXED` swap. The oracle converts the trailing continuously compounded return into an exponential tenor quote. Settlement therefore assumes that the Aave supply index will continue compounding at roughly the rate observed during the trailing window.

Aave accrues supply interest linearly between reserve updates. Each interaction that updates the USDC reserve stores that interval's growth in the liquidity index and starts a new interval. Repeated intervals produce:

$$(1 + r * dt[0]) * (1 + r * dt[1]) * \dots$$

As interactions become more frequent, this approaches $\exp(r * T)$. The exponential quote is therefore a reasonable fair-value estimate for an active reserve. It can overstate future growth if the USDC reserve becomes unusually inactive.

The original High-severity analysis treated the entire remaining 83-day tenor as one uninterrupted linear interval. That assumes no further USDC reserve interactions and does not establish a general settlement loss. With the PoC parameters, only four evenly spaced USDC reserve interactions are enough for the projected floating leg to become buyer-favorable, matching production's settlement classification. The reported 42,566.511092-USDC difference is therefore specific to the dormant-reserve endpoint.

Only interactions that update the USDC reserve affect this calculation. Activity in unrelated Aave reserves does not. No unprivileged user can ensure that the public USDC reserve remains inactive for the remaining tenor, so a practical fund-loss scenario was not established.

RECOMMENDATION

I-13 Aave supply closeouts assume reserve activity

CONTINUED

Document that the exponential closeout estimate assumes continued activity in the relevant Aave reserve. Monitor prolonged reserve inactivity if this assumption is operationally important.

If the protocol wants to account for dormant reserves, use an activity-aware estimate or an explicit bound between the idle linear case and the active exponential case. Do not replace the projection with one unconditional linear interval, because that can overcharge buyers under normal reserve activity. Add tests for dormant, sparse and frequently updated reserves.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/BumpedFixedEarlyExitUsesExponentialAaveSupplyProjection.t.sol

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-14**

ACKNOWLEDGED

Misbehaving Aave data provider note

CATEGORY Documentation

LOCATION <mention-page url="https://app.notion.com/p/f4a8bda5828c8249ac14016729a4b7fb"/>

REVIEW Main Review

DESCRIPTION

The NatSpec of `AaveBaseBorrowRateMinimum` says:

```
* @dev Failure modes are returned as '(0, false)' – never revert – so a misbehaving Aave
```

However, this `(0, false)` is being propagated from the oracle all the way up to `SwapCore`, causing the transaction to revert, which effectively still results in blocked operations because of misbehaving Aave data provider.

RECOMMENDATION

Consider strengthening the comment.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairoswap/protocol/pull/1083>

INFORMATIONAL **I-15**

ACKNOWLEDGED

Risk premium invalidity uses different error

CATEGORY Error
LOCATION Utils.sol#L1203-L1205
REVIEW Main Review

DESCRIPTION

`getRiskPremiumOracleRate()` handles the configured `riskPremiumOracle`, but when `IRateOracle.getRate()` returns `isValid == false`, it reverts with `Errors.E610()`. In `Errors.sol`, `E605()` is the risk-premium-specific invalid-rate error, while `E610()` is the generic stale/invalid oracle-answer code. Market creation already uses `E605()` for this same `riskPremiumOracle.getRate()` invalid flag, so runtime paths such as `buySwap()` and `Views.calculateSwapRate()` report an inconsistent selector for the same oracle failure class.

RECOMMENDATION

Consider changing the invalid risk premium branch in `getRiskPremiumOracleRate()` to revert with `Errors.E605()`, while keeping negative risk premium values on `Errors.E612()`.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1085>

INFORMATIONAL **I-16**

ACKNOWLEDGED

Redundant rate duration in settlement

CATEGORY Best Practices
LOCATION Utils.sol#L557
REVIEW Main Review

— DESCRIPTION

`settleSwap()` first normalizes `swapDuration` to either `market.swapTerm` for maturity settlement or `block.timestamp - swap.entryTimestamp` for early exit. The later `rateDuration` variable then duplicates the same value in both branches before being passed to `deriveRateFromIndex()`. This adds minor code complexity without changing behavior.

— RECOMMENDATION

Remove `rateDuration` and pass `swapDuration` directly to `deriveRateFromIndex()`.

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1085>

INFORMATIONAL I-17

ACKNOWLEDGED

Liquidation lacks maintenance buffer

CATEGORY Math
LOCATION Utils.sol#L1304-L1318
REVIEW Main Review

DESCRIPTION

`isSwapLiquidatable()` marks a swap liquidatable only when the owing side's collateral is fully consumed. Buyer-side liquidation requires `netObligation >= swapCollateral`, while pool-side liquidation requires `netObligation >= swapPoolCollateralBacking - liquidatorReward`, where the remaining backing is reserved for the liquidator reward rather than as counterparty protection. As a result, `liquidateSwap()` cannot be called while the owing side still has a maintenance buffer, so an adverse reference-rate move can take a swap directly from healthy to undercollateralized before liquidation becomes executable. The counterparty then receives only the posted collateral or backing, leaving any excess obligation unrecoverable. This also conflicts with the liquidation bot guide, which says liquidations close underwater positions before the obligation exceeds posted collateral: <https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/580875df571a96f4568a30c3510ec217767c2054/research/liquidation-bot-guide.md#L291>.

RECOMMENDATION

Add a maintenance threshold so `isSwapLiquidatable()` can trigger before the owing side's collateral is fully consumed, for example when `netObligation >= collateral * maintenanceRatioWad / WAD` with `maintenanceRatioWad < WAD`. For pool-side liquidation, reserve the liquidator reward in addition to this buffer. Update the liquidation documentation to match the implemented solvency guarantees.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-18**

ACKNOWLEDGED

Reuse cached market id

CATEGORY Gas Optimization
LOCATION SwapCore.sol#L1106-L1108
REVIEW Main Review

DESCRIPTION

`liquidateSwap()` caches `swap.marketId` in `marketId`, but then loads the market with `markets[swap.marketId]`. This repeats a storage field read that the function already cached locally.

RECOMMENDATION

Use the cached `marketId` when loading the market: `Types.Market storage market = markets[marketId];`

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1085>

INFORMATIONAL **I-19**

ACKNOWLEDGED

Document live gap availability risk

CATEGORY Documentation
LOCATION IndexBaseRateV1.sol#L296-L302
REVIEW Main Review

DESCRIPTION

`IndexBaseRateV1.setMaxLiveGapSec()` documents the upper freshness bound, but does not clearly state the availability impact of tightening `maxLiveGapSec` below `minSpacingSec`. Because `poke()` can only advance a new ring slot after `minSpacingSec`, a smaller `maxLiveGapSec` can make regularly scheduled pokes insufficient to keep the left TWAR anchor valid. This causes reads to return `isValid=false`, which can block new swaps, early exits, and oracle-priced LP valuation paths when the oracle is used as a base-rate feed. The same operational warning is documented more explicitly in `MorphoBaseRateV1.setMaxLiveGapSec()`, but not consistently across equivalent contracts.

RECOMMENDATION

Expand the `IndexBaseRateV1.setMaxLiveGapSec()` NatSpec to match the stronger warning in `MorphoBaseRateV1.setMaxLiveGapSec()`. Explicitly document that `maxLiveGapSec` has no lower bound, that values below `minSpacingSec` can cause reads to return `isValid=false` even under regular poke cadence, and that this may block consumers that require a live base-rate quote.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1083>

INFORMATIONAL **I-20**

ACKNOWLEDGED

Comment misstates early-exit duplicate handling

CATEGORY Documentation
LOCATION SwapCore.sol#L1459-L1462
REVIEW Main Review

— DESCRIPTION

The comment above the `swap.settled` guard in `_settleSwaps()` states that only `makePayment()` is affected by the idempotent skip because `exitSwapEarly()` pre-checks `settled`. However, duplicate swap IDs within the same `exitSwapEarly()` call all pass validation before settlement begins. The first occurrence settles the swap, while subsequent occurrences reach this guard and are skipped. The behavior is safe, but the comment inaccurately describes it.

— RECOMMENDATION

Either update the comment to acknowledge that duplicate IDs within the same `exitSwapEarly()` call are skipped by `_settleSwaps()`, or explicitly reject such duplicates during preprocessing. This can be done by checking `swap.isEarlyExit` before setting it and reverting with an appropriate protocol error when it is already `true`.

— RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1083>

INFORMATIONAL **I-21**

ACKNOWLEDGED

Renunciation can freeze a paused oracle

CATEGORY DoS
LOCATION contracts/oracles/BaseTimelockedOracle.sol:479-489
REVIEW Main Review

DESCRIPTION

`BaseTimelockedOracle` inherits OpenZeppelin's immediate `renounceOwnership()` without overriding it. If the owner renounces while `isPaused` is true, ownership becomes the zero address and the only `unpause()` path is permanently unavailable. Every rate read continues returning `isValid = false`.

The contract already treats removal of emergency controls as a timelocked operation:

`queueRevokePausability()` and `activateRevokePausability()` take three days and activation clears `isPaused`. Immediate ownership renunciation bypasses that lifecycle and can also strand pending governance changes.

This sequence requires a mistake by the trusted oracle owner rather than an unprivileged attacker. The resulting state is nevertheless irreversible: markets that rely on the oracle permanently lose rate availability and may be unable to enter positions, calculate NAV, exit early, liquidate or settle, depending on the oracle slot and available fallback.

The PoC deploys the production base implementation, posts a healthy rate, pauses it and renounces ownership. The owner becomes the zero address, the same rate remains invalid and the former owner can no longer call `unpause()`.

RECOMMENDATION

Override `renounceOwnership()` and make it revert, as `BaseUpgradableOracleWrapper` already does. If permanent administrative removal is required, implement a dedicated timelocked flow that requires the oracle to be unpaused, clears or resolves pending changes and emits an explicit terminal-state event.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/CurrentLedgerUniqueValidation.t.sol

RESOLUTION

Status: Acknowledged.

I-21 Renunciation can freeze a paused oracle

CONTINUED

<https://github.com/kairosswap/protocol/pull/1085>

INFORMATIONAL I-22

ACKNOWLEDGED

Morpho adapters misreport index decimals

CATEGORY	Warning
LOCATION	contracts/oracles/cumulativeIndex/AbstractCumulativeIndexMorpho.sol:59-63; contracts/oracles/cumulativeIndex/CumulativeIndexBorrowMorpho.sol:42-47; contracts/oracles/cumulativeIndex/CumulativeIndexSupplyMorpho.sol:43-48
REVIEW	Main Review

DESCRIPTION

Both Morpho cumulative-index adapters encode their asset-to-share ratio with an explicit `1e27` multiplier, but their shared `decimals()` function returns 18. The empty-market value near `1e21` does not make the encoding 18-decimal; it is `1e27 / 1e6`, caused by Morpho's virtual-share denominator.

Core's cumulative-rate math divides one index value by another, so the constant multiplier cancels and existing KAIRO settlements are not directly mispriced. A consumer that scales the raw index according to `decimals()`, however, interprets it by a factor of `1e9`. This can produce incorrect monitoring, migration or admission decisions wherever declared decimals are trusted. An upgradable wrapper also caches 18 and would reject a corrected 27-decimal replacement, so a live deployment would need an explicit migration rather than an in-place metadata change.

The PoC covers both production adapters. Equal large asset and share totals return an index near `1e27` while metadata reports 18 and the empty state returns exactly `1e21`, matching `1e27 / 1e6`.

RECOMMENDATION

Return 27 for new deployments and document the virtual-share effect separately. Do not change live oracle metadata in place: deploy a correctly described adapter and migrate wrappers/markets through an explicit new epoch so cached decimals and index continuity are handled deliberately.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/CurrentLedgerUniqueValidation.t.sol

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1091>

INFORMATIONAL **I-23**

ACKNOWLEDGED

Unused credit helper remains

CATEGORY Superfluous Code
LOCATION SwapPositionWrapper.sol#L282
REVIEW Main Review

DESCRIPTION

`_creditIfNeeded()` is no longer called by `settle()`, `settleBatch()`, `exitEarly()`, or `redeem()`, which now use `_creditKnownSettled()` instead. Keeping the unused helper increases maintenance overhead and can confuse future changes around wrapper settlement crediting.

RECOMMENDATION

Remove `_creditIfNeeded()` and update stale comments that still refer to it.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1080>

INFORMATIONAL **I-24**

ACKNOWLEDGED

Factory NatSpec is imprecise

CATEGORY Documentation

LOCATION KairosBuyerAdapterFactory.sol#L12-L16
KairosMarketAdapterFactory.sol#L12-L16

REVIEW Main Review

DESCRIPTION

Both factory comments state “Multiple adapters can exist per vault, one per Kairos market” and say the vault curator calls `createAdapter()`, but the code enforces one adapter per `(vault, marketId)` and allows either the vault `curator()` or `owner()` to create it.

RECOMMENDATION

Reword the NatSpec to make the scoping explicit: multiple adapters can exist per vault, with at most one adapter per Kairos market for that vault, and multiple vaults may each deploy their own adapter for the same Kairos market. Also update the deployment pattern to say either the vault `curator()` or `owner()` may call `createAdapter()`.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1083>

INFORMATIONAL I-25

ACKNOWLEDGED

Adapter list can be spammed

CATEGORY	DoS
LOCATION	KairosBuyerAdapterFactory.sol#L74-L99 KairosMarketAdapterFactory.sol#L70-L89
REVIEW	Main Review

DESCRIPTION

`createMarket()` is permissionless, and `createAdapter()` only requires a valid `marketId` plus an `IVaultV2`-compatible `vault` controlled by `msg.sender` through `owner()` or `curator()`. An attacker can therefore create many markets and/or fake vaults, repeatedly append adapters to the unbounded `adapters` array, and eventually make `getAllAdapters()` revert or exceed RPC return limits.

RECOMMENDATION

Avoid relying on unbounded `getAllAdapters()` for discovery. Add paginated adapter reads and use `AdapterCreated` events for indexing, or gate `createAdapter()` to trusted vaults if the factories are intended to list only legitimate integrations.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1085>

INFORMATIONAL I-26

ACKNOWLEDGED

Supply rounding comment is misleading

CATEGORY Documentation
LOCATION CumulativeIndexSupplyMorpho.sol#L15-L20
REVIEW Main Review

DESCRIPTION

The rounding limitation comment in `CumulativeIndexSupplyMorpho` appears to have been copied from the borrow-side adapter and describes the wrong direction of drift for ordinary supply-side operations.

The comment says Morpho supply-share rounding can cause the share-price index to decrease by sub-wei amounts during deposit/withdraw churn. However, ordinary Morpho supply-side rounding is generally supplier-favorable: `supply()` mints shares rounded down, `withdraw()` burns shares rounded up or returns assets rounded down, and residue remains in `totalSupplyAssets` for the remaining suppliers. This biases the global `totalSupplyAssets` / `totalSupplyShares` ratio upward, not downward.

RECOMMENDATION

Update the `CumulativeIndexSupplyMorpho` comment to describe the supply-side rounding direction separately from `CumulativeIndexBorrowMorpho`.

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1083>

HIGH **H-01**

PARTIALLY RESOLVED

Force-deallocation forfeits settlement reserves

CATEGORY	Unexpected Behavior
LOCATION	contracts/adapters/KairosMarketAdapter.sol:556-568
REVIEW	Remediation Review

DESCRIPTION

`KairosMarketAdapter.deallocate` does not block permissionless `forceDeallocate` while the market-wide early-exit or liquidation vesting reserve is live. Its two guards cover only the adapter position's deposit-profit vest and gains from a liquidation that is still executable. Both guards are false after a swap has already exited and the pool is idle.

This matters because `PoolInternalsLib._bookVesting` keeps a realized settlement gain in `pool.totalCollateral`, while `VirtualShareLib.deductVesting` subtracts the outstanding amount from every burn price. `SwapCore.withdrawCollateral` therefore burns the adapter's shares at a discount during the drip period. A caller can encode the current discounted position value as a partial withdrawal, which burns the adapter's entire position despite the `fullAmount` restriction. The rounding guard also accepts the withdrawal because it values the burned shares at the same discounted price.

A vault shareholder who also owns the remaining Kairos LP shares can invoke this behavior after an early exit or liquidation once the adapter's unrelated deposit-profit vest has expired. As the reserve drains, the forfeited vault entitlement accrues to the caller's direct LP position.

The PoC creates a 90-day market at the permitted 12x leverage, realizes a buyer-favorable early exit after 30 days, then forces a 1,000,000 USDC vault adapter out beside a 1,000,000 USDC attacker co-LP. The adapter receives 537,671.232876 USDC instead of its 1,078,741.033408 USDC value after the reserve drains. The co-LP gains the exact 541,069.800532 USDC difference. Through official Morpho Vault V2 bytecode with its maximum 2% force-deallocation penalty, the attacker's net portfolio profit is 509,513.308408 USDC. The victim depositor loses the same amount.

The attack requires a gain-producing early exit or liquidation, a live settlement reserve, direct co-LP capital and enough vault shares to pay the force-deallocation penalty. The reserve state is public. No privileged role, oracle manipulation, pending liquidation or non-standard token is required. The transfer scales with leverage, reserve size and the

H-01 Force-deallocation forfeits settlement reserves

CONTINUED

fraction of shares forcibly removed. A single tested withdrawal takes more than half of the adapter's eventual value, so this is a material loss of vault funds under realistic protocol limits.

— RECOMMENDATION

Expose the outstanding market vesting reserve through `ISwapCore` and reject permissionless `forceDeallocate` whenever the adapter owns shares and that reserve is nonzero. Allocator-controlled deallocation can remain available for an intentional early unwind.

A more flexible fix can transfer the burned shares' pro-rata reserve entitlement into a vault-owned claim instead of forfeiting it. In either design, do not measure conservation only at the vesting-discounted burn price.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/ForceDeallocateVestingReserveBypass.t.sol

— RESOLUTION

Status: Partially resolved.

<https://github.com/kairosswap/protocol/pull/1111>

HIGH **H-02**

RESOLVED

Force-deallocation transfers value to co-LPs

CATEGORY Math

LOCATION `contracts/adapters/KairosMarketAdapter.sol:593-594,632-644,789-794`

REVIEW Remediation Review

DESCRIPTION

`KairosMarketAdapter._realAssets` reports the vault's LP stake at `getVirtualSharePrice(Conservative)`. For an active pool, that view is the lower of the fair and accrued exact-netted marks. An actual withdrawal uses `getPoolBurnPrice` instead, whose fair and accrued legs are conservative per-bucket lower bounds. These prices can differ materially even when no position is liquidatable.

Permissionless `forceDeallocate` does not conserve the value reported by `_realAssets`. It records `getPoolBurnPrice` before withdrawal and uses that same lower price to value the shares burned by `SwapCore.withdrawCollateral`. Consequently, `burnValue` is approximately equal to the assets returned and the value-leak guard accepts the call, although extinguishing the vault's shares removes a much larger exact-netted claim. The omitted claim remains behind the same pool shares and accrues to the other LPs.

The PoC creates a supported ten-year cumulative market at 2x leverage. A 10,000,000 USDC attacker co-LP and a 90,000,000 USDC vault adapter back two 500,000,000 USDC swaps. Both swaps occupy the open low-rate band, one at 0.1 percent and one at 1.5 percent. After two years, the adapter's deposit vest is expired, no buyer-side cap binds and accrued is below fair, so every existing force-deallocation guard is open. The exact-netted price is 1.13169988598843 while the conservative burn price is 0.6.

The attacker requests 53,999,999.999999 USDC, which burns all but one raw adapter share. Before the call, the adapter reports 101,852,989.738958 USDC. The vault therefore loses 47,852,989.738954 USDC of already reported NAV immediately and the co-LP gains the exact same amount. Settling both swaps at maturity produces a 69,749,999.999993 USDC cash loss for the vault and the same cash gain for the co-LP.

A companion test executes through official Morpho Vault V2 bytecode from commit `f698b9ea78c528497d3d259cf7271ec25b60dca6`. The attacker holds direct Kairos LP shares and enough vault shares to pay the maximum 2 percent penalty. After the penalty, the attacker's net mature-cash profit is 66,851,890.844767 USDC, exactly equal to the victim vault shareholder's loss.

H-02 Force-deallocation transfers value to co-LPs

CONTINUED

The attack requires a market whose bucket bounds remain wider than its exact netted mark, direct co-LP capital and enough vault shares to absorb the force penalty. It needs no privileged role, liquidatable swap, outstanding settlement reserve, oracle manipulation or rounding-sized request. A caller can wait for ordinary base-rate movement within an open band before forcing the vault out.

— RECOMMENDATION

Measure permissionless withdrawal conservation on the same basis the adapter exposes to Vault V2. Capture the adapter's exact-netted conservative value before and after withdrawal and revert when the pre-withdrawal value exceeds `assets + postWithdrawalValue` by more than a small absolute token-atom tolerance. Equivalently, block `forceDeallocate` whenever `getPoolBurnPrice` is materially below `getVirtualSharePrice(Conservative)` while the adapter owns shares.

Do not reuse the lower burn price to establish the value of the claim being destroyed. Keep the allocator-controlled path available for an intentional conservative unwind.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/ForceDeallocateBoundedPriceMismatch.t.sol

— RESOLUTION

Status: Resolved.

<https://github.com/kairosswap/protocol/pull/1111>

HIGH **H-03**

ACKNOWLEDGED

Late vault deposits capture settlement reserves

CATEGORY	Unexpected Behavior
LOCATION	contracts/adapters/KairosMarketAdapter.sol:789-794
REVIEW	Remediation Review

DESCRIPTION

`KairosMarketAdapter._realAssets()` values the adapter's LP shares with the conservative virtual share price. `VirtualShareLib.deductVesting()` subtracts the outstanding early-exit or liquidation reserve from that price. The deduction represents a temporary accounting holdback: the tokens remain in `pool.totalCollateral` and the adapter's existing LP shares retain their pro-rata claim as the reserve drips into the share price.

Direct Kairos deposits do not dilute this claim because `supplyCollateral()` charges the reserve-undeducted mint price. A deposit into the parent Morpho Vault does not mint Kairos LP shares. Morpho instead prices the deposit from the adapter's reserve-deducted `realAssets()`. Consequently, a late depositor receives vault shares against the discounted value and acquires part of the realized reserve that belonged to incumbent vault shares.

The production PoC settles a 90-day swap after 30 days and leaves the pool idle with no active swaps. Fair and accrued-only prices are equal, no bucket cap binds and the only omitted value is the live settlement reserve. A 587,671.232876 USDC deposit into the pinned official Morpho Vault V2 captures 270,534.900265 USDC gross as the reserve drips. The depositor then makes the adapter liquid and pays the maximum configured 2 percent force-deallocation penalty. Its net profit is 254,486.917382 USDC, exactly equal to the incumbent vault-share loss.

The attacker needs an open parent vault, capital for a deposit and a material settlement reserve. The reserve state is public and can be produced by ordinary early exit or liquidation after the vault already owns Kairos LP shares. The adapter then reports only reserve-deducted value, so the attacker's deposit acquires part of the deferred claim at the understated NAV. No oracle write, privileged role, direct Kairos LP position, active liquidation or malformed token is required. After the reserve releases, an ordinary allocator unwind can realize the acquired value. Force-deallocation is not the source of the gain; it only makes the claim liquid sooner.

H-03 Late vault deposits capture settlement reserves

CONTINUED

— RECOMMENDATION

Include the adapter's pro-rata claim on `vestingOutstanding` in the value reported to the parent vault. This value is illiquid until it drips, but it is still owned by the adapter's existing Kairos LP shares. Expose the outstanding reserve and total pool shares through `ISwapCore`, then add `adapterShares * vestingOutstanding / totalPoolShares` to the reserve-deducted position value with conservative rounding.

If the parent integration must report only immediately realizable value, block parent-vault deposits and mints while a reserve is outstanding or track the deferred claim for the vault shares that existed when it was created. Blocking only force-deallocation does not prevent the dilution because the ownership transfer happens at deposit time.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/SettlementReserveOuterVaultDepositCapture.t.sol

— RESOLUTION

Status: Acknowledged.

HIGH **H-04**

RESOLVED

Failed base quotes bypass liquidation reserves

CATEGORY Unexpected Behavior

LOCATION contracts/lib/Utils.sol:1356-1376; contracts/lib/Utils.sol:1695-1719

REVIEW Remediation Review

DESCRIPTION

`Utils.liquidateSwap` values the fair side of a buyer-liquidation gain with `tryBaseRateOracle`. When the current remaining-tenor base quote is invalid, reverting, malformed, out of bounds or negative in a Cumulative market, that helper silently substitutes the swap's stored entry rate.

`PoolInternalsLib.vestSettlementJump` then subtracts the resulting marginal fair PnL from the buyer collateral just realized into `pool.totalCollateral` and creates a reserve only if the difference is positive. The historical entry rate is not a conservative bound on the current marginal mark. After rates move, it can make the seized collateral appear fully reflected in pre-liquidation NAV, produce `jump <= 0` and skip the reserve entirely.

This violates the settlement invariant that a forced removal must not publish more value to current shares than the position contributed to the immediately preceding fair NAV. The intended production wiring makes the failure mode reachable without failure of the reference index used to authorize liquidation.

`UpgradableTenorRateOracle` returns `(0, false)` under its independent emergency cutoff and `IndexBaseRateV1` also returns invalid when its observation ring cannot tightly bracket the TWAR boundary, while a Cumulative market's bare reference-index adapter can continue to return a healthy index. Liquidation therefore succeeds, seizes all buyer collateral, removes the swap from the active bucket and leaves the pool idle with the unreserved gain immediately visible in its raw share price.

Because the oracle's observation times are public, a depositor can see when the base-rate quote is about to become stale. They can deposit into the parent Morpho Vault just before that happens, while the active swap still holds down the adapter's NAV. After anyone liquidates the swap, the depositor receives part of the sudden, unreserved increase in pool value. The PoC uses the current production `SwapCore`, `Utils`, `PoolInternalsLib`, `KairosMarketAdapter`, both paired `UpgradableTenorRateOracle` wrappers and the vault NAV flow at the allowed 12x leverage. Deterministic downstream mocks supply only the rate trajectory; the vulnerable `(0, false)` response comes from the real wrapper's emergency-cutoff implementation. With a healthy live remaining-tenor quote of 0.1%, liquidation books

H-04 Failed base quotes bypass liquidation reserves

CONTINUED

1,811,910.771253 USDC and the late entrant's claim is 142,945.974887 USDC. With the same reference index and liquidation state but an invalid base quote, the 5% entry-rate fallback books no reserve and the entrant's claim becomes 1,018,702.148620 USDC. The outage alone transfers 875,756.173733 USDC more to the entrant than the healthy-quote control. Relative to its 112,746.762994 USDC deposit, the entrant earns 905,955.385626 USDC and the incumbent loses the same amount to within one token atom.

The attacker does not need to manipulate the reference index, cross the liquidation threshold, call an admin function or control the liquidator. The required conditions are a material difference between entry and current remaining-tenor base rates, a buyer-side liquidation whose realized collateral exceeds its current marginal fair mark, a base-only invalidity window and an open parent vault. Stale observation windows are timestamp-visible and liquidation is permissionless. Because one transaction transfers most of a million-USDC incumbent position under protocol-permitted leverage and the loss scales with pool size and the rate move, the consequence is material loss of vault funds under realistic oracle-outage conditions.

— RECOMMENDATION

Make `tryBaseRateOracle` return both the quote and whether it is live and do not use the entry rate as a valuation substitute. If the base quote is unavailable, preserve liquidation liveness but fail closed on gain recognition: conservatively book the full positive `realizedNavChange` into the vesting reserve for the canceled remaining term or use a formally proven lower bound on marginal fair PnL that cannot understate the reserve. A later healthy valuation may release any excess conservatism; the liquidation transaction must not publish an unpriced positive jump to share NAV.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/LiquidationBaseOracleFallbackReserveBypass.ts

— RESOLUTION

Status: Resolved.

<https://github.com/kairosswap/protocol/pull/1116>

HIGH **H-05**

RESOLVED

Accrued minting exposes projected exit gains

CATEGORY	Logical Error
LOCATION	contracts/lib/PoolInternalsLib.sol:77-82, contracts/lib/VirtualShareLib.sol:592-603
REVIEW	Remediation Review

— DESCRIPTION

Direct LP deposits into an active market are priced from `mintUpper`, which includes accrued PnL but excludes the projected remaining swap payment. The early-exit vesting hook uses a different basis: it subtracts the removed swap's marginal fair PnL from the value realized at settlement and reserves only the excess. When the remaining payment favors the pool, the buyer's early exit collects that projected payment, but `vestSettlementJump` can compute zero because the collected amount is already represented by the fair mark. A new LP therefore pays the lower accrued-only price immediately before the exit and receives a pro-rata share of value it did not purchase.

An unprivileged LP can observe a buyer's early-exit transaction, deposit before it, wait through the ordinary 12-hour LP profit vest and withdraw after the projected payment becomes idle pool collateral. The buyer can rationally close a maximum-loss position before liquidation to recover the separately prefunded liquidation bounty, so the sequence does not require cooperation or oracle control by the LP. In the validated 90-day Cumulative BUY_FIXED case, a 5 percentage-point rate move caused the buyer's 704,500.978474 USDC collateral balance to be realized by the pool while the hook booked no vesting reserve. A 1,000,000 USDC just-in-time deposit then withdrew 1,343,270.744233 USDC, whereas the same deposit made immediately after the exit withdrew 999,999.999999 USDC. The 343,270.744234 USDC difference was an equal loss to the incumbent LP. This is a scalable and reliable material loss under a legitimate rate trajectory, without liquidation, protocol fees, risk premiums or an early-exit fee.

— RECOMMENDATION

H-05 Accrued minting exposes projected exit gains

CONTINUED

Size the early-exit and buyer-side-liquidation reserve against the same conservative value basis used to mint direct LP shares. For example, calculate the pre-settlement marginal accrued-only PnL as well as marginal fair PnL, subtract the lower of those two values from `realizedNavChange` and vest any positive remainder over the canceled term. This preserves the existing protection when buyer-favorable projected value is discarded while also reserving pool-favorable projected value that accrued-only entrants did not pay for. Keep the collateral clamp in `_bookVesting` and apply the same basis rule to both forced-settlement callers.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/AccruedMintProjectedExitGain.t.sol

— RESOLUTION

Status: Resolved.

<https://github.com/kairosswap/protocol/pull/1115>

HIGH **H-06**

RESOLVED

Mixed maturities misprice forced-exit vesting

CATEGORY	Math
LOCATION	packages/hardhat/contracts/lib/PoolInternalsLib.sol:122-195; packages/hardhat/contracts/lib/SwapFormulas.sol:833-1049
REVIEW	Remediation Review

DESCRIPTION

`vestSettlementJump` measures a forced exit against the marginal mark of the swap's aggregate bucket. `_swapMarginalMtmPnL` reconstructs the removed swap inside the live stored bucket and `_markActiveBucket` prices both versions as active at one current index and projected rate. A bucket can nevertheless contain members on both sides of maturity, either within a wide ordinary interval or after circular bucket reuse while an expired swap remains unsettled. The expired member is then valued with the live member's remaining-tenor assumptions, so the marginal delta can materially under- or over-book the reserve.

This breaks the invariant that forced-exit value released into LP collateral is withheld only to the extent attributable to the departing position. An attacker who is or becomes a co-LP can capture an under-booked release, while over-booking depresses adapter NAV and lets a later vault depositor dilute incumbents before the false reserve drips back. The current-code rollover PoC books 8,639,495.656542 USDC instead of the 3,000,045.181979-USDC control and lets a 10,000,000-USDC entrant redeem 11,593,057.760651 USDC. The ordinary maturity-straddle proof demonstrates the opposite under-booking direction without requiring modulo reuse.

RECOMMENDATION

Do not value mixed live/expired members as one active bucket. Prevent physical bucket reuse until it is empty and, for every forced exit, virtually settle expired members and compute the departing swap's pre/post delta using its own maturity/index semantics. If homogeneity cannot be certified, reserve the full positive realized jump conservatively.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/HelixMixedMaturityForcedExitVesting.t.sol

RESOLUTION

Status: Resolved.

H-06 Mixed maturities misprice forced-exit vesting

CONTINUED

<https://github.com/kairosswap/protocol/pull/1121>

HIGH **H-07**

RESOLVED

Delayed risk report pairs with newer base rate

CATEGORY	Oracle
LOCATION	packages/hardhat/contracts/oracles/RiskRateOracle.sol:198-245; packages/hardhat/contracts/SwapCore.sol:836-875; packages/hardhat/contracts/lib/Utils.sol:1423-1437
REVIEW	Remediation Review

— DESCRIPTION

Risk-premium reports bind their own observedAt timestamp and freshness window, but they do not bind the base/reference observation or ring state against which the premium was modeled. buySwap reads the current base quote and the independently fresh risk curve, then permanently stores both. An untrusted relayer can therefore withhold a still-valid signed risk report until the base-rate ring has moved and trade against a cross-vintage combination the trusted reporters never evaluated together.

Both inputs remain individually fresh and valid, so ordinary stale-data checks do not prevent the mismatch. The current-code PoC changes only the risk-report vintage between two otherwise identical branches. The delayed 0.5% premium consumes 0.9 USDC of a 1-USDC LP deposit, whereas a contemporaneous 2.5% premium at the same base rate and settlement path increases pool collateral to 11.195111 USDC. Exploitation requires a material base move during the report's accepted freshness window and access to the signed bearer payload, but it can consume material LP backing.

— RECOMMENDATION

Bind signed risk reports to the relevant market/oracle identities and to the base/reference observation timestamp, ring sequence, or block used by the pricing model. At entry, require a tight maximum cross-feed skew in addition to each feed's independent maximum age.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/HelixRiskReportBaseRateVintage.t.sol

— RESOLUTION

Status: Resolved.

<https://github.com/kairosswap/protocol/pull/1140>

MEDIUM **M-01**

RESOLVED

Five-axis bucket pricing can exceed the gas cap

CATEGORY DoS
LOCATION `contracts/lib/BucketBoundsLib.sol:556-573`
REVIEW Remediation Review

DESCRIPTION

`_boundsFallback` permits 32 corner combinations. This equals the full 2^5 product of the stored rate, entry-index, utilization-fee, risk-premium and entry-time ranges. Consequently, a bucket that is heterogeneous on every axis never uses the aggregate-collateral fallback. `SwapFormulas._dualPnlRanges` evaluates both risk-premium endpoints inside the fee and time loops for every such bucket.

Every LP deposit and withdrawal from an active pool calls `BucketBoundsLib.poolBounds`. The repository's maximum-occupancy gas test fills 63 economically significant sub-buckets but leaves the risk-premium range constant. A companion test also varies that fifth range. With the production-derived 30-day term, five-day interval, seven time buckets, nine rate bands and 3.5x leverage, withdrawal consumes 21,029,379 gas. A low-level call supplied with 16,777,216 gas fails with empty return data. The same withdrawal succeeds without the cap and returns collateral.

If a live market reaches this state, LP deposits and withdrawals cannot execute until positions settle or the book becomes cheaper to value. Adapter operations that depend on the same share pricing can also fail. This is a temporary but market-wide availability failure.

The test does not prove that an unprivileged buyer can force the state in a production market. It directly controls mock base-rate oracles to place swaps in all nine bands during every active time window. A buyer can decide when to trade around legitimate oracle observations, but cannot make a trusted base-rate source traverse those bands. Reaching the measured state therefore requires an extreme external rate trajectory, a permissive market configuration or control of the market's oracle. This constraint lowers the finding to Medium despite the severe effect once the state exists.

M-01 Five-axis bucket pricing can exceed the gas cap

CONTINUED

This is a post-remediation manifestation of the gas-growth family described by the earlier Low finding "Bucket saturation inflates LP pricing gas." That report covered the former 1,000-bucket scan, measured 14,031,500 gas and did not establish a transaction-cap failure. The present implementation has a different cost trigger: five-axis corner enumeration in the bounded 63-sub-bucket design.

— RECOMMENDATION

Reduce `MAX_CORNERS` below 32 so five-axis buckets use the aggregate-collateral fallback, then measure the resulting mint/burn spread. Alternatively, lower the occupancy ceiling until a 32-corner book has substantial margin below the transaction cap.

The robust fix is to remove the complete occupied-book scan from share mutations. Maintain bounded incremental valuation state or use a permissionless checkpoint that cannot block withdrawals. Extend the gas gate to vary risk premium as well as rate, index, utilization fee and entry time. Test every supported term and rate convention through an exact 16,777,216-gas call.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/MaxOccupancyFiveDimGas.t.sol

— RESOLUTION

Status: Resolved.

<https://github.com/kairosswap/protocol/pull/1110>

MEDIUM **M-02**

ACKNOWLEDGED

Zero cap allocation blocks recovery reporting

CATEGORY	Unexpected Behavior
LOCATION	contracts/adapters/KairosBuyerAdapter.sol:321-327
REVIEW	Remediation Review

DESCRIPTION

`KairosBuyerAdapter.allocate()` reports `_allocationDelta(_realAssets())` directly after recording a newly purchased swap. Unlike `_deallocateReturn()`, this calculation does not floor a zero value to one asset unit while `_swapIds` is nonempty. A swap whose immediate mark-to-market value is zero can therefore leave both `lastReportedAllocation` and the parent Vault V2's cap allocation at zero even though the adapter still tracks the position.

This state is reachable with market parameters accepted by `SwapCore`, including one-times leverage, no liquidation incentive and an early-exit fee that consumes the entry-block closeout value. In a focused test, a 100,000 USDC BUY_FIXED swap entered with zero reported value and later recovered to 43.287673 USDC after the reference index increased. When the adapter tried to report that recovery, `_deallocateReturn()` returned the nonempty adapter ID and the positive change, but Morpho Vault V2 checked that the existing allocation was nonzero before applying the change and reverted with `ZeroAllocation`. The revert rolled back the adapter's mirror update, leaving the Vault's cap exposure understated and preventing deallocation operations that retain tracked exposure from committing.

A complete cleanup that removes every tracked swap can still use the adapter's empty-ID handling and a later positively valued allocation may restore the mirror while the cap remains open. The defect can nevertheless require special allocator sequencing and delay reporting or recovery during mixed-position lifecycles or wind-down, when a new allocation may no longer be permitted. Exploitation also requires a curator-approved, loss-heavy market configuration and allocator activity; it is not a permissionless attack.

RECOMMENDATION

Use a single allocation-reporting function for both `allocate()` and `deallocate()`. Whenever `_swapIds` is nonempty, floor a zero reported value to one asset unit before calling `_allocationDelta()`. Keep the empty-ID result limited to the state where `lastReportedAllocation` is zero and no swaps remain tracked.

M-02 Zero cap allocation blocks recovery reporting

CONTINUED

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/KairosC002BuyerZeroMirror.t.sol

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-03**

ACKNOWLEDGED

Raw cumulative exits can skip compounding

CATEGORY Math
LOCATION SwapFormulas.sol#L377-L390
REVIEW Remediation Review

DESCRIPTION

`cumulativeRemainingCorrection()` silently skips the exponential compounding correction whenever the computed `u` is greater than or equal to `MAX_EXP_INPUT_WAD`. This helper is used by `calculateEarlyExitSettlement()` to price the remaining floating leg for `Cumulative` `BUY_FLOATING` early exits.

For `BUY_FLOATING` markets, the projected base quote is raw and must be compounded over the remaining tenor before settlement. However, if `u = projectedRate * timeRemaining / SECONDS_IN_YEAR` is outside the `expWad()` input domain, the function does not revert or cap the value. Instead, it leaves `compounding` as zero and returns only the linear base plus `cumulativeRemainingCrossTerm()`.

As a result, a buyer can early-exit a `Cumulative` `BUY_FLOATING` swap while the projected rate is above the exponential domain and avoid paying the remaining compounded floating-leg amount. The PoC compares two exits from the same pre-exit snapshot: one with `u` just below `MAX_EXP_INPUT_WAD`, which consumes the buyer collateral, and one with `u` just above the threshold, which skips compounding and releases more value to the buyer.

RECOMMENDATION

Do not silently fall back to linear pricing when `u >= MAX_EXP_INPUT_WAD` in `cumulativeRemainingCorrection()`. For state-changing settlement paths such as `exitSwapEarly()`, revert when the projected cumulative exponent is outside the supported domain, or otherwise enforce an explicit conservative cap that cannot undercharge the pool. The same domain handling should be applied consistently to any view/NAV path that relies on `cumulativeRemainingCorrection()`.

PROOF OF CONCEPT <https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/8a571632e3335a689cd67d3bcaacb00af9323408/packages/hardhat/test-foundry/EarlyExitRawQuoteCompounding.t.sol#L186-L234>

RESOLUTION

Status: Acknowledged.

MEDIUM **M-04**

RESOLVED

Departed premium extrema transfer LP value

CATEGORY Unexpected Behavior
LOCATION contracts/lib/PoolInternalsLib.sol:265-324
REVIEW Remediation Review

DESCRIPTION

`PoolInternalsLib.certifiedBoxTighten` recalculates the surviving bucket's base-rate, entry-index and entry-time bounds after a swap leaves. It never recalculates `minUtilFee`, `maxUtilFee`, `minRiskPremium` or `maxRiskPremium`. Therefore, a fee or premium extreme remains in the bucket until every other swap in that bucket leaves, even though the corresponding weighted moment and collateral were removed.

`BucketBoundsLib` still feeds those historical extrema into the corner valuation used for LP mint and burn prices. A buyer can exploit this during an ordinary premium change. The buyer first opens a dust position while the premium is high. After the premium falls, large positions can enter the same time and base-rate bucket. When the dust position expires first, settling it removes its notional and premium moment but leaves its high premium in `maxRiskPremium`. A co-LP can then withdraw at the depressed burn price. The unpaid value remains in the pool and becomes claimable by an LP who stays until the live swaps settle.

The proof uses a 90-day, 4x Cumulative market. A 1 USDC notional position entered at a 3% premium posts 4,932 atomic USDC as collateral. Two 1,000,000 USDC positions enter the same bucket one day later at a zero premium. After the dust position expires and is settled, both the stale and clean books have the same two live swaps, aggregate moments, collateral bounds and certified base-rate, index and time ranges. The pricing-bound difference is that the stale book retains `maxRiskPremium = 3%`. It also holds 0.004932 USDC more pool collateral from settling the dust swap. That tiny balance difference would raise the stale price, so it works against the demonstrated haircut.

The stale burn price is `0.9990465763024`, compared with `0.9999999999746` in the clean control. A 2,500,000-share LP receives 2,497,616.440756 USDC instead of 2,499,999.999936 USDC. The 2,383.559180 USDC haircut remains in the pool. After the two live swaps settle, the remaining LP receives 2,383.564112 USDC more than the clean control. An actor who controls the dust buyer and the remaining LP can therefore transfer value from a withdrawing LP for a cost that is orders of magnitude smaller than the captured amount. The attack requires a sufficiently large legitimate fee or premium decrease while positions continue to share a bucket, plus enough co-LP capital to benefit from another LP's withdrawal.

M-04 Departed premium extrema transfer LP value

CONTINUED

RECOMMENDATION

Remove fee and premium extrema when their last live member leaves the bucket. One option is to maintain second moments for `utilFee` and `riskPremium` and include both dimensions in the same survivor-bound certification used for rate, index and time. If exact extrema or certification cannot be maintained, do not use historical fee or premium bounds to set transactable LP prices.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_P0Cs/StaleFeePremiumBoundsImpact.t.sol

RESOLUTION

Status: Resolved.

<https://github.com/kairosswap/protocol/pull/1127>

MEDIUM **M-05**

ACKNOWLEDGED

Transient Morpho floor forces stale settlement

CATEGORY Unexpected Behavior

LOCATION `contracts/lib/Utils.sol:583-649`, `contracts/oracles/cumulativeIndex/CumulativeIndexBorrowMorpho.sol:36-45`

REVIEW Remediation Review

DESCRIPTION

`resolveExpiryIndex()` treats one failed reference-oracle update after the settlement grace period as proof that the oracle is permanently unusable. It immediately settles the swap from extrapolated historical snapshots and permanently terminates both markets in the pair. It does not distinguish a lasting outage from a source that is invalid only while the settlement call is running.

This is exploitable with the intended Morpho borrow index.

`CumulativeIndexBorrowMorpho.getCumulativeIndex()` returns `(0, false)` whenever the underlying borrow assets or shares fall below its configured exposure floor. The deployed USDC configuration uses floors of 1 USDC and `1e12` shares. A borrower who controls most of the market's borrow shares can temporarily repay enough debt to leave both totals just below those floors, call `makePayment()` after grace and borrow the same assets again before the transaction ends. Morpho permits repayment on behalf of a borrower without authorization, while borrowing again only requires the borrower's normal authorization, collateral and market liquidity. The guaranteed retry therefore sees an invalid source and commits the stale fallback even though the source is valid again when the attack transaction returns.

A `BUY_FLOATING` owner can use this when the stale history understates the floating amount owed at maturity. The deterministic PoC follows an 89-day 5% history with one day at a bounded 200% rate. For a 1 billion USDC notional, healthy settlement pays the buyer 8,217,262.052922 USDC. The transient floor crossing instead pays 13,626,532.084562 USDC, transferring an extra 5,409,270.031640 USDC from the LP pool and terminates both markets. The repayment leaves 0.508869 USDC and `5e11` shares rather than emptying the market and Morpho's exact `toAssetsUp` and `toSharesUp` conversions restore the source index to within one part per million in the same transaction.

The attack requires a missed post-maturity snapshot throughout the grace period, a favorable difference between the recorded history and the real maturity index, control of enough borrow shares to cross the floors, sufficient collateral and Morpho liquidity to reborrow and temporary repayment capital. Those conditions reduce the likelihood, but the capital

M-05 Transient Morpho floor forces stale settlement

CONTINUED

is returned in the same transaction and the demonstrated loss can consume millions of dollars of reserved LP collateral without oracle or Kairos privileges.

— RECOMMENDATION

Do not make irreversible fallback and termination decisions from one invalid read. Record the first failed adjudication without settling, then require the source to remain invalid across separate blocks and for a minimum persistence period before enabling fallback. A successful observation must clear that pending verdict. For Morpho exposure-floor failures, settlement should remain pending long enough for keepers to record a valid post-maturity snapshot after exposure returns. Keep the existing bounded emergency path for genuinely dead sources, but require evidence that cannot be created and removed within one transaction.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/TransientMorphoFloorFallback.t.sol

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-06**

ACKNOWLEDGED

Repeated pricing scans lock LP deallocation

CATEGORY DoS

LOCATION contracts/adapters/KairosMarketAdapter.sol:556-594,602-604,686-692

REVIEW Remediation Review

DESCRIPTION

`KairosMarketAdapter.deallocate` combines several full occupied-book pricing reads in one transaction without budgeting their aggregate gas cost. A permissionless force-deallocation checks `anyBucketCapBinds`, reads the accrued-only and fair virtual prices, reads the burn price, calls `withdrawCollateral`, then calls `_realAssets()` to report the new allocation. Each operation walks the same live bucket book. An allocator deallocation performs fewer checks, but still combines the withdrawal scan with a second full valuation through `_realAssets()`.

The individual `SwapCore` operations can remain below the transaction gas cap while their adapter composition exceeds it. The PoC creates a 90-day market with five active 15-day windows and four adjacent rate bands from 2.9% to 11.5%, for 20 occupied sub-buckets. Every force-deallocation safety check is open. A direct `withdrawCollateral` from that state consumes 5,365,526 gas, but the production adapter call consumes 17,849,153 gas. Calling the adapter with exactly 16,777,216 gas exhausts gas and returns no revert payload. Calling it without the cap succeeds.

The same problem affects allocator recovery at the protocol occupancy ceiling. With 63 calm sub-buckets, an isolated core withdrawal consumes 14,807,990 gas, while `vault.deallocate` consumes 19,212,061 gas. An exact-cap call fails and the uncapped control returns the requested collateral. The adapter owns the LP shares and `onlyParentVault` prevents a depositor or allocator from bypassing it with a direct core withdrawal. Consequently, the permissionless escape hatch can become unusable at only 20 occupied sub-buckets. At higher occupancy, even the trusted allocator can be unable to recover the vault's position until swaps settle and the book becomes cheaper to value.

Buyers can populate time windows by trading normally, but they cannot force a trusted production oracle through the four tested rate bands. The deterministic test controls mock rate oracles to create that history. This reachability constraint lowers the likelihood, while the concrete consequence remains a temporary lock of vault liquidity under an allowed market shape.

M-06 Repeated pricing scans lock LP deallocation

CONTINUED

RECOMMENDATION

Replace the adapter's sequence of pricing calls and `withdrawCollateral` with one atomic, adapter-oriented `SwapCore` operation. After refreshing the reference index once, that operation should run one `BucketBoundsLib.poolBounds` fold and reuse its `accruedLower`, `fairLower` and `capBinds` outputs to enforce the vesting, liquidation-gap, burn-price and value-leak checks before burning shares. It should return the withdrawn amount, shares burned and the adapter's exact post-withdraw value. Because a successful withdrawal cannot have expired unsettled swaps and does not modify the swap buckets, the post-withdraw conservative value can be calculated from the same lower-bound PnL and the updated collateral and share totals; it does not require another occupied-book scan. `KairosMarketAdapter` should pass that returned value directly to `_allocationDelta` instead of calling `_realAssets()`.

Keep the quote, guards, state mutation and returned valuation in the same `SwapCore` call so no state or oracle change can invalidate a separately computed quote. After this consolidation, calibrate the occupied-sub-bucket ceiling against the worst-case complete `vault.deallocate` and `vault.forceDeallocate` transactions, including vault overhead, oracle work, five-axis corner pricing, token transfers and allocation accounting. Set the ceiling so both calls retain a documented safety margin below the target chain's transaction gas limit.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/P0Cs/packages/hardhat/test-foundry/zz_POCs/MarketAdapterCompositeGasLock.t.sol

RESOLUTION

Status: Acknowledged.

MEDIUM **M-07**

RESOLVED

Paired collateral omission underprices LP burns

CATEGORY Math
LOCATION `contracts/lib/SwapFormulas.sol:2437-2450`
REVIEW Remediation Review

DESCRIPTION

`SwapCore.buySwap` sizes each buyer's collateral from the larger-magnitude base quote across the two paired markets. This protects a canonical Cumulative pair when the `BUY_FIXED` market uses a bumped quote and the `BUY_FLOATING` market uses the smaller raw quote. `SwapFormulas._dualBoxMinCaps` later reconstructs the bucket's buyer-collateral caps from the selected market's local base-rate box only. For the lower-quoted side, the reconstructed cap is therefore smaller than the collateral every real position in that bucket actually posted.

The understated cap feeds `accGain0vershoot`, the conservative LP burn bound and `anyBucketCapBinds`. A position can consequently be reported as buyer-liquidatable and its accrued LP claim can be discounted even when the actual accrued obligation remains below `swap.collateralBalance`. An LP who withdraws at that lower bound burns too many shares for the collateral received. The omitted claim remains in the pool and is paid to the LPs who keep their shares through settlement.

The PoC uses the intended Cumulative bumped/raw oracle topology with a 90-day market. A `BUY_FLOATING` buyer posts 20,740,318.703050 USDC because the paired bumped quote is 258.528382%, while the local raw quote is 200%. Halfway through the term, the accrued payment owed to the LP is 18,630,000 USDC. `liquidateSwap` correctly reverts because the buyer still has more collateral than the obligation, but `anyBucketCapBinds` returns true and the conservative burn price is 1.16410821917805 instead of the exact accrued price of 1.18629999999998.

One of two equal LPs then withdraws. It receives 58,205,410.958902 USDC rather than the 59,315,000 USDC it receives by holding the identical control position through settlement. The maturity reference return is chosen so that the final obligation is the same 18,630,000 USDC already reflected by the exact accrued mark, isolating the burn-bound error from later rate exposure. The 1,109,589.041098 USDC difference is received by the remaining LP in the attack branch. A buyer or searcher who is also the remaining LP can wait for an ordinary reference-rate state and a third-party LP to leave while this false cap window is open.

M-07 Paired collateral omission underprices LP burns

CONTINUED

The finding requires the paired quote to be materially larger than the selected market's quote, a still-solvent accrued obligation between the two collateral caps and another LP to withdraw; those conditions make exploitation state-dependent, but the resulting transfer can be material without oracle failure or privileged access.

— RECOMMENDATION

Preserve the pair-level collateral basis in bucket accounting and use it wherever buyer caps are reconstructed. The minimal design is to store a per-swap collateral base rate or the actual buyer collateral per notional in a sound bucket range and maintain its extrema through add and removal.

`_dualBoxMinCaps`, the coupled overshoot fold and the cap-bind signal should derive buyer caps from that same stored basis instead of `lpRate` whenever paired collateral differs from the local quote.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/PairedCollateralBoundMismatch.t.sol

— RESOLUTION

Status: Resolved.

<https://github.com/kairoswap/protocol/pull/1129>

MEDIUM **M-08**

ACKNOWLEDGED

Tiny exits can prolong vesting

CATEGORY DoS
LOCATION PoolInternalsLib.sol#L106-L119
REVIEW Remediation Review

DESCRIPTION

`_bookVesting()` merges every new vesting booking into a single global `VestingReserve`. When a new positive `jump` is booked, the function first computes the current `live` outstanding amount, then sets `v.amount = live + booked`, resets `v.dripStart = now`, and sets `v.dripEnd` to the later of the existing end and the new booking horizon.

This allows a small later `exitSwapEarly()` or buyer-side `liquidateSwap()` with a positive `jump` and a longer remaining term to re-drip the entire existing `live` reserve, not only the newly booked amount. An attacker can repeatedly open and early-exit small positions before the reserve fully drips to keep pushing `v.dripEnd` forward. This does not let the attacker directly steal the reserve because minting uses the undeducted `mintPrice`, but it can indefinitely delay LP NAV recognition and keep full last-LP withdrawals blocked by the outstanding vesting guard.

RECOMMENDATION

Consider vesting design alternatives that don't prolong the vesting period on each interaction

PROOF OF CONCEPT <https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/b47f42928ead083ec290cadd38cdb3f1e886b4/packages/hardhat/test-foundry/VestingReserveExtensionGrief.t.sol#L99-L140>

RESOLUTION

Status: Acknowledged.

MEDIUM **M-09**

RESOLVED

Stale minimum notional depresses burns

CATEGORY	Business logic errors
LOCATION	PoolInternalsLib.sol#L271-L288 PoolInternalsLib.sol#L297-L304 Utils.sol#L2091-L2128 VirtualShareLib.sol#L592-L603
REVIEW	Remediation Review

— DESCRIPTION

`certifiedBoxTighten()` relies on `minMemberNotional` to derive survivor-containing bucket bounds after a swap is removed. However, `minMemberNotional` is intentionally left stale-low when the minimum-notional member exits. If the removed member was a tiny rate/index/time extreme, the certifier can continue assuming that a tiny far-away survivor exists and fail to tighten `minLpRate`, `maxLpRate`, or related boxes.

After removal clears `E415`, LP withdrawals are live again. `getPoolBurnPrice()` routes through `poolMintAndAnchorPrices()`, which consumes `BucketBoundsLib.poolBounds()` lower bounds. A stale-wide survivor box can therefore depress the burn price for exiting LPs, transferring value to remaining LPs. The PoC shows the same old settlement and same live survivor swaps producing a stale burn price of `1.000172603687` versus a clean burn price of `1.001208220126`, about `10 bps` worse for the exiting LP.

— RECOMMENDATION

Reconsider the usage of a stale-low `minMemberNotional`.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/e7ebcf5429e877167f901915f66ac94e0d89020c/packages/hardhat/test-foundry/zz_POCs/RolloverStaleBucketImpact.t.sol#L79-L135

— RESOLUTION

Status: Resolved.

<https://github.com/kairosswap/protocol/pull/1127>

MEDIUM **M-10**

PARTIALLY RESOLVED

Dust swap poisons its bucket's LP burn price

CATEGORY	Logical Error
LOCATION	packages/hardhat/contracts/lib/PoolInternalsLib.sol:157-204, packages/hardhat/contracts/lib/BucketBoundsLib.sol:472-577
REVIEW	Remediation Review

— DESCRIPTION

Bucket extrema are updated pointwise when a swap enters, without weighting the extreme by that swap's notional. Conservative active-bucket pricing then applies the stored rate, index, fee, premium, and time corners to the bucket's entire aggregate notional. A one-token active swap at an extreme rate can therefore make a hundreds-of-millions-notional incumbent position appear to share that extreme, even though the weighted fair mark barely changes.

The current-code differential PoC creates identical 90-day books. Both contain a 240,000,000-token incumbent swap and 1,000,000 tokens of LP collateral; one branch adds a one-token swap at 7.9% beside the 6.1% incumbent. That active dust member lowers the executable burn price from `1.640587292898` to `1.103473051244` WAD. A victim LP receives `55,173.652562` tokens instead of `82,029.364644`. After both swaps settle and the surviving LP exits, the attacker's terminal gain increases by exactly the victim's `26,855.712082`-token shortfall. A strict share-burn or output bound prevents extraction only by making the withdrawal unavailable.

This is distinct from departed-extrema and stale-minimum findings: the outlier remains live, the recorded box accurately contains both members, and no removal-time recertification runs. Fixing stale state after a member exits therefore does not address this path. The demonstrated attack requires the adversary to fund the large incumbent exposure and remain the surviving LP, which limits likelihood and supports Medium severity despite the material executable haircut.

— RECOMMENDATION

Make active-bucket bounds notional-aware. Preserve enough distribution information to apply each extreme only to a conservatively bounded portion of aggregate notional, or segregate economically small/outlier members into separate pricing cohorts. A relative minimum-notional rule can be defense in depth, but it should not replace a bound that remains conservative without letting one atom determine the valuation of the entire bucket.

M-10 Dust swap poisons its bucket's LP burn price

CONTINUED

— RESOLUTION

Status: Partially resolved.

<https://github.com/kairosswap/protocol/pull/1130>

MEDIUM **M-11**

ACKNOWLEDGED

Bearer update permits obsolete risk premium

CATEGORY	Frontrunning
LOCATION	packages/hardhat/contracts/oracles/RiskRateOracle.sol:198-245; packages/hardhat/contracts/SwapCore.sol:836-875; packages/hardhat/contracts/SwapCore.sol:923-948
REVIEW	Remediation Review

— DESCRIPTION

Signed risk-premium updates are permissionlessly relayable and take effect only when `postRiskPremiums` writes storage. A bearer or searcher who sees a signed increase can buy scarce LP capacity immediately before relaying it. `buySwap` permanently snapshots the old lower premium, swap rate and buyer collateral; publication cannot reprice the position afterward.

The bearer receives a free timing option against the risk model's newer observation and externalizes the omitted protection to LPs if the modeled adverse path occurs. Helix's production-parameter proof uses a permitted 20-percentage-point update and shows a stale-premium trade draining LP value under the corresponding adverse reference path while the post-update control is profitable to the pool. The future path remains conditional, so Medium is appropriate.

— RECOMMENDATION

Use atomic trusted publication, or a commit/activation design that freezes affected-market entry after an update is revealed and before it becomes active. Enforce strictly increasing report sequencing and clear supersession semantics.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-12**

ACKNOWLEDGED

Committee changes alter prior report authority

CATEGORY	Signatures
LOCATION	packages/hardhat/contracts/oracles/BaseTimeLockedOracle.sol:335-350,657-699
REVIEW	Remediation Review

DESCRIPTION

Signed oracle reports bind the current `forecastNonce` and report data, but not the signer set or quorum under which they were created. `_verifySignatures` evaluates signer membership and `requiredSignatures` only when a relayer submits the bytes. Activating a signer addition or quorum reduction does not advance the nonce or establish a new committee epoch. A signature created by an unauthorized candidate, or a partial quorum that was insufficient when signed, can therefore remain in public calldata and become valid byte-for-byte after activation. The newly authorized packet can consume the nonce ahead of a current report and its rate can be snapshotted into new swaps.

The inverse transition has the same missing-boundary problem. A report admitted immediately before a signer removal or quorum increase remains the live curve and cadence baseline after committee hardening, so the hardened committee can be unable to replace it immediately. An ordinary, honestly signed transition can thus leave swaps using a curve that the active committee never authorized under its current policy.

The focused current-code tests prove both relaxation cases at the exact three-day boundary: a candidate signature first reverts `UnauthorizedSigner` and succeeds unchanged after signer activation; a two-signature packet first reverts against a 3-of-3 threshold and succeeds unchanged after activation of 2-of-3. Helix's end-to-end scenarios show that a relayer can combine this boundary with changed rate conditions and materially transfer LP backing. Exploitation requires a scheduled committee transition, a still-fresh packet, and adversarial relay ordering, so the issue is Medium rather than High.

RECOMMENDATION

M-12 Committee changes alter prior report authority

CONTINUED

Bind a monotonically increasing committee epoch into every EIP-712 report and require it to equal the active epoch. Increment the epoch on every signer addition, removal, re-addition, and quorum change. On activation, invalidate the old stored curve and cadence baseline or atomically install a migration report authorized under both policies; do not let pre-activation signatures become newly authorized or continue governing post-transition reads by default.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-13**

RESOLVED

Early-exit fee can overflow before payout cap

CATEGORY	DoS
LOCATION	packages/hardhat/contracts/lib/Utils.sol:915-988; packages/hardhat/contracts/lib/VirtualShareLib.sol:826-877; packages/hardhat/contracts/adapters/KairosBuyerAdapter.sol:712-736
REVIEW	Remediation Review

DESCRIPTION

`calculateEarlyExitSettlement` multiplies the uncapped remaining obligation by the WAD fee before applying the position's collateral/backing payout cap. Under high but valid first-party Morpho rates, notional and tenor, the intermediate can exceed `uint256` even though the ultimate amount collectible from either side is bounded and representable. Both state-changing early exit and BuyerAdapter's closeout-based `realAssets()` reuse this arithmetic.

A valid position can therefore become unexitable and make every Vault accounting path that reads the adapter revert, freezing otherwise idle deposits/redemptions until maturity or a favorable rate change. The focused current-code proof reaches the overflow before the collateral cap; the fork-dependent Vault extension could not be rerun because no Base RPC was configured.

RECOMMENDATION

Use `Math.mulDiv(netObligation, earlyExitFee, WAD)` and cap intermediate obligations at the maximum amount that can affect the final payout before additional arithmetic.

RESOLUTION

Status: Resolved.

<https://github.com/kairosswap/protocol/pull/1119>

MEDIUM **M-14**

ACKNOWLEDGED

Dust top-up resets seasoned shares' anchor

CATEGORY	Validation
LOCATION	packages/hardhat/contracts/SwapCore.sol:689-763; packages/hardhat/contracts/SwapCore.sol:433-465
REVIEW	Remediation Review

— DESCRIPTION

When an LP's prior vest has expired, any new deposit replaces the aggregate position's `entryPrice` with the current burn price and starts a fresh deadline for every existing share. The deposit has `minSharesOut`, but there is no caller bound on the new anchor or mint-to-anchor spread. A dust top-up can therefore reset a large seasoned position at a transiently depressed price, causing all old shares to be capped at that anchor during a recovery.

The current-code PoC uses a 10,000,000-token seasoned position and shows its urgent withdrawal falling to about 660,090.909090 tokens after the reset while a co-LP later receives the residual value. The LP must submit the top-up, but automated adapters/allocators can do so without a bound on this independent state change; a third party can arrange the transient price around that transaction.

— RECOMMENDATION

Add a depositor-specified minimum anchor or maximum mint-to-anchor spread. Prefer separate share lots so a new deposit cannot re-vest or re-anchor previously seasoned shares.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-15**

ACKNOWLEDGED

Per-swap clipping makes exit order matter

CATEGORY	Logical Error
LOCATION	packages/hardhat/contracts/SwapCore.sol:1125-1164; packages/hardhat/contracts/SwapCore.sol:1608-1751; packages/hardhat/contracts/lib/PoolInternalsLib.sol:66-120
REVIEW	Remediation Review

— DESCRIPTION

An early-exit batch processes and books vesting one position at a time. `_bookVesting` clips every positive jump against collateral currently free after earlier reservations, while negative jumps never reduce a reserve booked earlier in the same atomic batch. Offsetting positions therefore do not telescope: one caller-selected permutation can withhold materially more LP NAV than the reverse permutation despite the same positions, timestamp, oracles and final physical collateral.

The withheld difference depresses adapter/LP NAV until it drips and can be captured through vault share timing. The current-code PoC proves the two orders end in the same economic closure state but book different reserves.

— RECOMMENDATION

Accumulate signed realized and fair-mark changes per market across the complete batch, then book `max(totalRealizedChange - totalFairChange, 0)` once with a conservative combined horizon.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-16**

ACKNOWLEDGED

Self-hedged Vault misreports internal payments

CATEGORY Configuration

LOCATION `packages/hardhat/contracts/adapters/KairosBuyerAdapterFactory.sol:81-105; packages/hardhat/contracts/adapters/KairosMarketAdapterFactory.sol:77-95; packages/hardhat/contracts/adapters/KairosBuyerAdapter.sol:698-736; packages/hardhat/contracts/adapters/KairosMarketAdapter.sol:789-794`

REVIEW Remediation Review

— DESCRIPTION

Both factories permit the same Vault to own a buyer adapter and an LP adapter for the same market. BuyerAdapter marks each live position to its executable closeout, including adverse remaining-term value. MarketAdapter reports conservative pool-share NAV that can defer the corresponding favorable receivable until accrual or settlement. When the Vault is both payer and recipient, its adapters therefore recognize the two sides at different times even though the eventual token payment is internal to the same beneficial owner.

A sufficiently adverse BUY_FLOATING closeout can make the Vault report a loss, lower its share price and allow new deposits to mint against the temporary mismatch; the value reappears when the matching LP side recognizes the payment. The path requires curator configuration of both adapters, so it is a deployment/integration hazard rather than a permissionless Core exploit and is rated Medium. Static code confirms the allowed topology and asymmetric valuation surfaces; the canonical Vault PoC could not be rerun without a configured Base fork.

— RECOMMENDATION

Forbid one Vault from pairing buyer and LP adapters for the same market, or provide a combined adapter that nets matched internal payer/recipient legs on one valuation basis. Document and enforce the supported topology at factory or Vault configuration time.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-17**

ACKNOWLEDGED

Settlement order poisons the utilization EMA

CATEGORY	Logical Error
LOCATION	packages/hardhat/contracts/SwapCore.sol:1481-1496; packages/hardhat/contracts/SwapCore.sol:1608-1751; packages/hardhat/contracts/lib/Utils.sol:1208-1222
REVIEW	Remediation Review

DESCRIPTION

`_settleSwaps` accrues and clamps the market EMA inside the per-swap loop before every collateral mutation. When multiple settleable swaps in one market realize offsetting gains and losses, the first loss ratchets the stored average downward; a later gain does not restore it because the second same-block accrual has zero elapsed time. Reversing the caller-selected order produces a different persistent EMA even though final pool collateral and settled positions are identical.

Subsequent buyers are charged from this poisoned state. The current-code differential proof ends with equal pool assets but EMAs of about 3.269 trillion versus 5 trillion atomic units; the affected buyer's fee is about 2.11% instead of 0.51%, and the attacker/co-LP receives roughly 1,315 USDC more at terminal settlement. No privilege is required because settlement is permissionless.

RECOMMENDATION

Accrue each touched market once from its pre-batch state and clamp once against the final post-batch collateral, or retain a block-start anchor and recompute later same-block mutations from that anchor. Settlement permutations with the same final state must not change future fees.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-18**

ACKNOWLEDGED

Unpoked base quote overstates BuyerAdapter NAV

CATEGORY	Oracle
LOCATION	packages/hardhat/contracts/lib/VirtualShareLib.sol:826-866; packages/hardhat/contracts/adapters/KairosBuyerAdapter.sol:698-736; packages/hardhat/contracts/SwapCore.sol:1342-1355
REVIEW	Remediation Review

DESCRIPTION

BuyerAdapter's view-only `realAssets()` reaches `getBuyerCloseoutValue`, which reads the cached base tenor quote but cannot call its permissionless `poke`. `updateMarketRateIndex` can immediately poke the same base oracle before the next read. A moved underlying source can therefore leave Vault share accounting on a favorable stale closeout even though any user can refresh the market and expose an adverse executable value in the next transaction.

An informed existing shareholder can redeem against inflated Vault assets and consume idle liquidity before refreshing the oracle; remaining holders retain the marked-down Kairos position. This requires a poke-dependent base stack and available Vault liquidity, both present in the intended ring-based configurations. The canonical Vault end-to-end PoC was not rerun because the required Base fork endpoint was unavailable, but the stale-view/fresh-poke divergence is explicit in the scoped code.

RECOMMENDATION

Make Vault share-price operations refresh the market first or fail closed when a supported base oracle is pokeable but not current. Another safe option is to accept only base feeds whose getter is freshness-equivalent to a same-block poke for adapter-backed markets.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-19**

RESOLVED

Arithmetic index interpolation underquotes TWAR

CATEGORY	Oracle
LOCATION	<code>\`IndexBaseRateV1.sol#L485-L504\`
\`IndexBaseRateV1.sol#L566-L580\`
\`SwapCore.sol#L990-L1004\`</code>
REVIEW	Remediation Review

DESCRIPTION

`IndexBaseRateV1` linearly interpolates raw levels of a multiplicative cumulative index. Between unequal endpoints, the arithmetic chord lies above the correct geometric or log-linear index even when the source compounds at a constant rate. Consequently, `_realizedTwar()` overstates the historical boundary index and understates TWAR, while `_applyTenorBump()` exponentiates this short-window bias across the full market tenor.

Permissionless observation timing can place the TWAR boundary inside the widest valid chord; recording the right endpoint immediately before entry does not correct the earlier interpolation. The production-path proof exercises `updateMarketRateIndex()`, `buySwap()`, and `makePayment()`. At the first-party Morpho `AdaptiveCurve` maximum, a valid 24-hour chord, two-year tenor, and 12x leverage allow a maximum-capacity `BUY_FIXED` trader to enter below the rate implied by the honest cumulative source. The resulting floating-minus-fixed difference exhausts the position's locked backing and virtually all pool collateral.

RECOMMENDATION

Interpolate multiplicative indices in log space by calculating `exp(lerp(ln(leftIndex), ln(rightIndex), weight))` with conservative rounding, or derive TWAR from time-weighted log returns. Add constant-exponential and Morpho Taylor-accrual invariants requiring quotes to remain independent of observation timing.

RESOLUTION

Status: Resolved.

<https://github.com/kairosswap/protocol/pull/1137>

LOW **L-01**

ACKNOWLEDGED

Market adapter omits idle balances

CATEGORY	Logical Error
LOCATION	KairosMarketAdapter.sol#L674-L678 KairosMarketAdapter.sol#L789-L795
REVIEW	Remediation Review

DESCRIPTION

`KairosMarketAdapter._realAssets()` only reports the adapter's LP-share value from `swapCore.getLpPosition()` and `swapCore.getVirtualSharePrice()`. It does not include `asset.balanceOf()`, even though `ACTION_SWEEP` can transfer the adapter's token balance to the parent vault. Until swept, idle tokens controlled by the adapter are omitted from `realAssets()` and from `lastReportedAllocation`, so vault share operations before `ACTION_SWEEP` can use a lower NAV than operations afterward.

RECOMMENDATION

Include `asset.balanceOf()` in `KairosMarketAdapter._realAssets()` and add the LP-share value to that balance. In `deallocate()`, account for partial `ACTION_WITHDRAW` flows where withdrawn tokens are temporarily held by the adapter before `VaultV2` pulls `assets`, so `lastReportedAllocation` stays synchronized with the vault allocation.

RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Zero-duration marks omit growth

CATEGORY	Logical Error
LOCATION	SwapFormulas.sol#L858-L870 Utils.sol#L1457-L1466 Utils.sol#L2371-L2374
REVIEW	Remediation Review

DESCRIPTION

`SwapFormulas._bucketPnLShared()` leaves `accruedFloatingRate` unset unless `rateDuration > 0`, while `Utils.calculateLiquidationObligation()` returns zero when `swapDuration == 0` and `Utils.getSwapNetAmount()` returns `(0, 1)` for same-timestamp swaps. For `Types.RateConvention.Cumulative`, the source index can change within the same timestamp and `RateIndexLib.update()` intentionally rereads it. The direct cumulative payment is determined by the index ratio and does not economically disappear merely because no wall-clock second elapsed. Same-timestamp cumulative growth can therefore be omitted from LP marks and liquidation checks until the timestamp advances.

RECOMMENDATION

For `Types.RateConvention.Cumulative`, calculate accrued floating payment directly from the index ratio instead of requiring positive elapsed time before recognizing it. Use elapsed duration only where annualized rates, fixed payments, or linear fees require it. Apply the same treatment in `_bucketPnLShared()`, `calculateLiquidationObligation()`, `getSwapNetAmount()`, and related valuation paths.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Dust buckets hide forced-exit liquidation risk

CATEGORY	Logical Error
LOCATION	packages/hardhat/contracts/lib/BucketBoundsLib.sol:322-338
REVIEW	Remediation Review

— DESCRIPTION

`BucketBoundsLib.poolBounds` short-circuits every bucket whose buyer collateral and pool backing are each at most one millionth of total pool collateral. It assigns the same unconditional lower bound to the fair and accrued marks and continues before `_activeBucketBounds` can determine whether a member's buyer-side cap binds. `VirtualShareLib.anyBucketCapBinds` reuses this output as a security signal for `KairosMarketAdapter`'s permissionless force-deallocation guard, so a live, independently liquidatable swap in a dust bucket can incorrectly report `capBinds == false`.

An opposite projected position can simultaneously close the adapter's market-wide accrued-versus-fair fallback. A vault shareholder can then force the adapter to burn its LP shares immediately before the dust position is liquidated, leaving the liquidation recovery to direct co-LPs. The current production-path PoC executes the liquidation, rewinds, proves both adapter signals are false, and successfully forces the full adapter stake out. Compared with an otherwise identical control, the vault forfeits 79,114 atomic USDC units. The loss is bounded by the dust threshold for each occupied bucket and the demonstrated route requires very large pool capital relative to the victim allocation, so Low severity is appropriate.

— RECOMMENDATION

Do not skip the cap-binding security signal for occupied active dust buckets. Before taking the pricing shortcut, conservatively set `capBinds` unless a cheap member-safe check proves no buyer-side cap can bind. Keep the conservative dust price bracket separate from the liquidation signal consumed by permissionless force-deallocation.

— RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Floored time moments create phantom margin

CATEGORY Rounding

LOCATION packages/hardhat/contracts/lib/SwapFormulas.sol:2648–2660; packages/hardhat/contracts/lib/SwapFormulas.sol:2711–2741

REVIEW Remediation Review

DESCRIPTION

The netting-margin calculation divides `weightedTimeSq` and `weightedEntryTime` independently before subtracting their squares. These floors do not preserve the translation/variance identity. Even swaps separated by only one second can therefore produce a materially overstated `Var(t)` term once large epoch timestamps are squared, inflating the conservative margin applied to LP withdrawal pricing.

The haircut remains in the pool and benefits surviving LP shares. A strategic survivor can construct the small time dispersion and withdraw after the victim, but the required scale and conservative-bound context limit practical likelihood.

RECOMMENDATION

Compute the variance numerator as `n * weightedTimeSq - weightedEntryTime^2` before division, using full-precision multiplication, and apply the same approach to covariance terms.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Forced-exit vesting ignores LP burn bounds

CATEGORY	Logical Error
LOCATION	packages/hardhat/contracts/lib/PoolInternalsLib.sol:122-195; packages/hardhat/contracts/lib/VirtualShareLib.sol:566-603
REVIEW	Remediation Review

— DESCRIPTION

The forced-close reserve is computed from the difference between two raw aggregate bucket marks. Executable LP mint/burn pricing uses a separate conservative, cap-aware bounds surface. When a departing member changes whether a collateral cap or certified box binds, the raw marginal delta can be smaller than the value actually released into the executable share price. The remainder is immediately withdrawable by LPs rather than vested over the canceled term.

The current-code proof constructs a cap-bound removal and measures a 937,330.002811-USDC terminal transfer to the strategic LP. The setup requires a large, specifically positioned bucket, so the finding is kept Low despite the scalable witness.

— RECOMMENDATION

Compute forced-close vesting as the pre/post difference on the same conservative cap-aware transaction surface used by LP burns, including any certified-box tightening caused by removal.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

Member dust margin makes splits nonadditive

CATEGORY	Rounding
LOCATION	packages/hardhat/contracts/lib/BucketBoundsLib.sol:668-692; packages/hardhat/contracts/lib/SwapFormulas.sol:2711-2731
REVIEW	Remediation Review

— DESCRIPTION

Bucket bounds add a fixed dust allowance proportional to `memberCount` even when every member has the same rate, index, timestamp, fee and premium and the aggregate mark is exact. Splitting one economic swap into many homogeneous members therefore changes LP burn pricing without changing exposure. The source proof measures a reversible 64-atom-per-member withdrawal haircut that remains for surviving shares.

The direct impact is token-atom dust and requires deliberately fragmented positions, so this is Low severity despite the deterministic accounting inconsistency.

— RECOMMENDATION

Collapse provably homogeneous buckets to the exact netted mark, or make the member allowance zero when all relevant boxes collapse. Derive any remaining pad from a proven rounding-error bound rather than member count alone.

— RESOLUTION

Status: Acknowledged.

LOW **L-07**

ACKNOWLEDGED

Negative premiums make new markets unusable

CATEGORY Validation
LOCATION packages/hardhat/contracts/lib/Utils.sol:318-322
REVIEW Remediation Review

DESCRIPTION

`probeOraclesForMarketCreation` calls the configured risk-premium oracle and checks only its validity flag. It discards the signed rate, while the runtime helper rejects any negative premium with `E612` at `contracts/lib/Utils.sol:1423-1435`. A permissionless creator can therefore publish both halves of a market with an oracle returning a valid negative rate, and LPs can deposit into those markets, even though every `buySwap` call reverts until the oracle reports a nonnegative value.

The intended `RiskRateOracle` stack prevents negative premiums, so this requires a different but contract-permitted risk oracle. The configuration is public and affects only the new market; LP principal remains withdrawable. The concrete impact is therefore market-local deployment waste and idle LP capital rather than corruption of an intended canonical market.

RECOMMENDATION

Use the same rate-domain validation at creation and runtime. Retain the rate returned by `getRate()` in `probeOraclesForMarketCreation` and reject a negative value with `E612`, in addition to checking `ok`. A shared helper would prevent the two validation paths from drifting again.

RESOLUTION

Status: Acknowledged.

LOW **L-08**

ACKNOWLEDGED

Payment flooring makes split swaps nonadditive

CATEGORY	Rounding
LOCATION	packages/hardhat/contracts/lib/SwapFormulas.sol:463-492; packages/hardhat/contracts/lib/Utils.sol:1050-1072
REVIEW	Remediation Review

— DESCRIPTION

`calculateSwapPayment` converts the signed rate leg and the positive utilization/risk-fee leg to token atoms independently for every position, flooring each before adding them. Splitting one economic exposure into many small positions can therefore erase fee income and, where opposite-signed legs nearly offset, turn discarded fractional legs into a buyer-favorable payout. The aggregate result differs from one otherwise identical unsplit position.

Each position contributes less than a few token atoms of error, so meaningful extraction requires many positions and gas expenditure. It is still a deterministic violation of payment additivity and can bias LP accounting in the attacker's direction.

— RECOMMENDATION

Keep signed legs in higher precision until after they are netted, then convert once. Alternatively carry per-account/market remainders and ceil nonzero fee charges. A minimum fee-bearing notional can provide additional protection.

— RESOLUTION

Status: Acknowledged.

LOW **L-09**

ACKNOWLEDGED

Vesting floor lets partial burns consume reserve

CATEGORY	Rounding
LOCATION	packages/hardhat/contracts/lib/VirtualShareLib.sol:504-519; packages/hardhat/contracts/SwapCore.sol:450-526
REVIEW	Remediation Review

— DESCRIPTION

`deductVesting` first converts the outstanding reserve into a per-share price deduction with floor rounding. When `outstanding * WAD < totalShares`, the deduction is zero even though reserve value remains. Partial withdrawals can then burn shares without bearing their proportional reserve deduction, progressively transferring the rounding residue away from shares that remain through the drip.

Each operation captures at most a small rounding amount, and the source construction needs an unusually large floor-recapitalized share supply to amplify it. The accounting invariant is nevertheless broken on the public withdrawal path.

— RECOMMENDATION

Subtract reserve value from aggregate pool NAV before the final share-price division. If the price-only interface is retained, round `outstanding * WAD / totalShares` upward and use the same primitive for every burn and view.

— RESOLUTION

Status: Acknowledged.

LOW **L-10**

ACKNOWLEDGED

Sole-owner adapter cannot recover wiped pool

CATEGORY	DoS
LOCATION	packages/hardhat/contracts/adapters/KairosMarketAdapter.sol:401-404
REVIEW	Remediation Review

DESCRIPTION

`KairosMarketAdapter.allocate` rejects every pool whose raw collateral-to-share ratio is at the dead-share floor. The guard does not distinguish a dangerous recapitalization that subsidizes third-party legacy shares from a vault adapter that owns every outstanding share. In the latter case, adding capital transfers no value outside the vault: the adapter is both the only legacy claimant and the only recipient of the new shares.

An ordinary adverse settlement can leave a vault adapter as the sole holder of nonzero shares in an idle pool with zero collateral. The trusted allocator's only supported allocation path then reverts with `WipedPoolWithDeadShares`, even though a direct `SwapCore.supplyCollateral` call immediately succeeds with the same recovery amount. The factory also records one adapter per `(vault, market)`, so the vault cannot create a replacement adapter for that market. The affected capital remains idle and the configured market cannot be reused through the supported adapter route. This is a recovery and availability failure rather than an asset loss.

RECOMMENDATION

Keep rejecting floor-priced recovery when another account owns pool shares, but permit an explicit manual allocator recovery when the adapter owns `pool.totalShares`. Bind the check and supply atomically, and require suitable mint bounds so ownership cannot change between validation and deposit.

RESOLUTION

Status: Acknowledged.

LOW **L-11**

ACKNOWLEDGED

Pool-sized dust routing taxes LP withdrawals

CATEGORY	Logical Error
LOCATION	packages/hardhat/contracts/lib/BucketBoundsLib.sol:472-535; packages/hardhat/contracts/lib/VirtualShareLib.sol:566-604; packages/hardhat/contracts/SwapCore.sol:611-680
REVIEW	Remediation Review

— DESCRIPTION

The bucket-bounds scan classifies a bucket as dust relative to mutable pool-level collateral. A deposit can therefore move an unchanged active bucket across the shortcut threshold and change which pricing route supplies the burn/mint bound. The same swaps and oracle state can receive a different executable withdrawal price solely because unrelated LP collateral was added.

An actor with deposit authority and a surviving LP position can trigger the route change before another LP withdraws. The current-code PoC shows the victim receiving less and the difference remaining for surviving shares, even though no swap exposure changed. The impact is limited by the dust threshold and required pool positioning, supporting Low severity.

— RECOMMENDATION

Make dust classification bucket-local and invariant under unrelated LP deposits, or remove the shortcut from transactable mint/burn prices. If a shortcut remains, prove it yields the same conservative bound on both sides of any supply/withdraw mutation.

— RESOLUTION

Status: Acknowledged.

LOW **L-12**

ACKNOWLEDGED

Stale disable call erases newer fee epochs

CATEGORY	Business logic errors
LOCATION	packages/hardhat/contracts/Admin.sol:293-335
REVIEW	Remediation Review

— DESCRIPTION

`disableMarketCreationFee` carries no expected fee token, amount, pending activation or configuration epoch. A transaction authorized for an earlier live/pending state can remain in a queue or public executor and execute after the owner has activated or queued a replacement. It then clears both the new live fee and the unrelated pending successor. The same state-unbound pattern lets stale activation intent target a replacement pending configuration.

Only the owner can call these functions, so exploitation requires asynchronous/multisig execution rather than an unprivileged attacker. The consequence is silent cancellation of a later approved fee policy, supporting Low severity.

— RECOMMENDATION

Maintain a monotonically increasing market-fee epoch and require every activation, cancellation and disable call to name the expected epoch and relevant live/pending state. Revert on mismatch.

— RESOLUTION

Status: Acknowledged.

LOW **L-13**

ACKNOWLEDGED

Tenor rounding can invert fixed and raw quotes

CATEGORY	Rounding
LOCATION	packages/hardhat/contracts/oracles/BaseRateAverageIndex/IndexBaseRateV1.sol:562-580; packages/hardhat/contracts/oracles/BaseRateAverageIndex/MorphoBaseRateV1.sol:795-817
REVIEW	Remediation Review

DESCRIPTION

Both `_applyTenorBump` implementations floor `r * tenor / YEAR`, then floor `exp(u) - 1`, and finally floor the annualization division. For small positive rates or short tenors, the compounded increment can round away enough that the returned bumped quote is below the raw continuous-compounding rate from the same observation ring. This violates the intended ordering used by paired Cumulative markets: BUY_FIXED should use the protective compounded-equivalent quote while BUY_FLOATING uses the corresponding raw quote.

The effect is bounded by fixed-point rounding, so the direct economic loss is small, but it can make a valid intended pair internally inverted and weakens the fixed-side protection at the exact low-rate/short-tenor boundary.

RECOMMENDATION

Retain remainder precision through exponent construction and round the final protective quote upward. At minimum return `max(bumped, inputRate)` for positive inputs.

RESOLUTION

Status: Acknowledged.

LOW **L-14**

ACKNOWLEDGED

Utilization seasoning depends on call cadence

CATEGORY	Math
LOCATION	packages/hardhat/contracts/lib/SwapFormulas.sol:708-748; packages/hardhat/contracts/lib/Utils.sol:1193-1259; packages/hardhat/contracts/SwapCore.sol:765-775
REVIEW	Remediation Review

— DESCRIPTION

`projectUtilAvg` linearly blends the stored average toward spot by `elapsed / window`, and every call persists the result and resets `lastUpdate`. Repeating the same blend over partitions of an interval is not equivalent to applying it once over the full interval. Permissionless buys and collateral mutations can therefore change the seasoned-liquidity value without changing the underlying collateral residence history.

This makes future utilization fees path-dependent. Dust top-ups or intervening buys can keep liquidity less seasoned and cause an otherwise bounded order to revert or pay more; in favorable configurations an LP can receive excess utilization income. The arithmetic defect is certain, but the stronger liquidation/extraction scenarios depend on fee bounds and market positioning and were not independently reproduced on the current harness, so this finding is rated Low.

— RECOMMENDATION

Use a time-composable accumulator, such as exact exponential decay or residence-time tranches, and avoid re-anchoring on calls that do not change total collateral. Identical collateral histories must produce identical seasoned values regardless of checkpoint frequency.

— RESOLUTION

Status: Acknowledged.

LOW **L-15**

ACKNOWLEDGED

Aave checkpoints change supply-index growth

CATEGORY	Unexpected Behavior
LOCATION	contracts/oracles/cumulativeIndex/CumulativeIndexSupplyAave.sol:69-85, contracts/oracles/BaseRateAverageIndex/IndexBaseRateV1.sol:433-481
REVIEW	Remediation Review

— DESCRIPTION

`CumulativeIndexSupplyAave` treats Aave's normalized supplier income as an interaction-invariant cumulative benchmark, but Aave projects that index linearly from the last reserve checkpoint. A reserve update capitalizes the projected value and resets its timestamp, so one long interval produces one linear factor while multiple checkpoints produce a product of linear factors.

The deployed Base Aave pool accepts `flashLoanSimple` with a zero amount and its repayment path still checkpoints the reserve without transferring principal or charging a premium. A buyer can therefore choose a finer capitalization cadence after locking a Kairos `BUY_FIXED` quote. Every reported index remains positive, monotone, continuous and individually valid, so the adapter's existing flash-premium route check does not detect the change in growth convention.

The pinned Base proof opens a capacity-sized position against 500,000 USDC, then performs 98 monthly zero-value checkpoints. The checkpointed path reaches 494,327.600055 USDC of accrued buyer entitlement and becomes pool-side liquidatable, transferring the full LP backing. At the identical timestamp, the no-checkpoint control remains on Aave's single linear epoch and instead favors the LP. The demonstrated loss is deliberately an edge case: it requires more than eight years, a reserve held artificially inactive between attacker-selected checkpoints, maximum term and leverage and no risk premium. Base Aave USDC is normally active and the intended upper-side risk premium can absorb the discrepancy, so this is a Low-severity benchmark-integration hazard rather than a practical full-pool attack on the intended live configuration.

— RECOMMENDATION

Do not treat an interaction-dependent normalized-income path as a cadence-invariant settlement benchmark. Either integrate supplier growth using a canonical fixed cadence, require a source whose cumulative index composes independently of permissionless checkpoints or explicitly bound accepted term, source inactivity and all-in pricing so the maximum checkpoint-partition difference is covered.

L-15 Aave checkpoints change supply-index growth

CONTINUED

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCs/HELIX2/0QM002AaveCheckpointCadence.t.sol

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

ACKNOWLEDGED

Short TWAR windows misprice long swaps

CATEGORY	Oracle
LOCATION	IndexBaseRateV1.sol#L123-L125 IndexBaseRateV1.sol#L374-L380 IndexBaseRateV1.sol#L503-L504
REVIEW	Remediation Review

DESCRIPTION

`IndexBaseRateV1` derives fixed quotes from a trailing realized window through `_realizedTwar()` and `getRateByTenor()`. The returned quote is parameterized by arbitrary nonzero `tenorSeconds`, but the observed source growth is always annualized over `twarWindowSec`. If `twarWindowSec` is short relative to the supported `swapTerm`, a trader can pay to move utilization during the TWAR window and cause long-tenor fixed quotes to reflect a short-lived regime. The observed source interest is real, not artificial, and any profit still depends on future floating accrual, so this is primarily a configuration risk.

RECOMMENDATION

Calibrate `twarWindowSec`, maximum `swapTerm`, and Kairos exposure caps against the capital and time required to move the referenced lending market's utilization. Consider longer TWAR windows, source-depth-based exposure caps, or deviation checks for long-tenor markets that reference utilization-sensitive lending sources.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Batch oracle comment overstates guard

CATEGORY	Documentation
LOCATION	UpgradableTenorRateOracle.sol#L150-L154 Utils.sol#L1362-L1363
REVIEW	Remediation Review

DESCRIPTION

`UpgradableTenorRateOracle.getRates()` states that the cumulative-profile negative-rate guard prevents the negative curve from being read even by a consumer that ignores `isValid`, and that `invalidate-without-revert` keeps `Utils.tryBaseRateOracle()` liquidation fallback alive. The implementation only zero-fills negative batch results when downstream `isValid` is true, so a downstream can still return negative payloads together with `isValid == false`. In addition, `Utils.tryBaseRateOracle()` calls `getRateByTenor()`, not `getRates()`, so the liquidation-fallback rationale does not apply to this batch path. Protocol batch consumers currently check `isValid`, so this is an overstatement of the documented invariant rather than a demonstrated valuation bypass.

RECOMMENDATION

Move the liquidation-fallback explanation to the `getRateByTenor()` / `_getRateByTenor()` documentation, and rewrite the `getRates()` comment to state that valid negative cumulative batches are invalidated for protocol callers that check `isValid`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-03

ACKNOWLEDGED

Require fresh data after L2 recovery

CATEGORY	Oracle
LOCATION	ChainlinkAdapter.sol#L67-L86 ChainlinkAdapter.sol#L95-L107 PythAdapter.sol#L136-L146
REVIEW	Remediation Review

DESCRIPTION

`ChainlinkAdapter.getPriceWithMetadata()` validates sequencer recovery through `_checkSequencer()` and price freshness through `_updatedAt` / `maxAge` independently. Once `gracePeriod` elapses, a price round published before recovery can become valid again if `_updatedAt` remains within `maxAge`. `PythAdapter.getRate()` reads `pyth.getPriceNoOlderThan()` and checks `publishTime` age, but has no sequencer recovery awareness, so a pre-recovery cached price can remain usable while `publishTime` is within `maxAge`.

RECOMMENDATION

For `ChainlinkAdapter`, require price `_updatedAt` to be at least the sequencer recovery `startedAt`. For `PythAdapter`, add an independent L2 uptime check and recovery grace or require an authenticated Pyth update with `publishTime` at or after recovery before price-sensitive actions.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-04

ACKNOWLEDGED

Wrapper activation can commit downstream cycles

CATEGORY Validation

LOCATION contracts/oracles/wrappers/BaseUpgradableOracleWrapper.sol:249-264, 422-436

REVIEW Remediation Review

DESCRIPTION

`BaseUpgradableOracleWrapper.activateDownstream()` validates a proposed downstream before assigning it to `downstream`. The interface check rejects only direct self-reference and `_validateDownstreamHealth(newDownstream)` therefore evaluates the candidate against the wrapper's old downstream graph rather than the graph that will exist after assignment.

For example, assume wrapper A initially points to a healthy oracle X and wrapper B points to A. B is healthy while the graph is `B -> A -> X`. If A's owner queues B and activates it after the three-day timelock, the activation health check follows `B -> A -> X` and succeeds. The subsequent assignment changes the final graph to `A <-> B`. All quote and description calls that forward through the downstream then recurse until the call fails. Cached or local selectors such as `decimals()` and `supportsInterface()` remain available, so this is specifically a failure of downstream-forwarded reads.

The owner-only activation and timelock make this a privileged configuration failure rather than an unprivileged value-extraction path. Nevertheless, the code explicitly promises to reject an unhealthy downstream at activation and the committed cycle can make an intended `UpgradableTenorRateOracle` unavailable to market entry, valuation and other consumers until another healthy downstream completes the timelock. The emergency cutoff stops recursive quote execution but returns invalid quotes and does not restore service; `description()` continues to recurse. This is distinct from the existing direct-self-reference finding: the `candidate == address(this)` guard fixes `A -> A`, but permits the separate candidate B in `A -> B -> A` and cannot repair the resulting graph.

RECOMMENDATION

Validate the post-assignment graph atomically. After the existing candidate checks, tentatively assign the new downstream and perform gas-bounded calls to the wrapper's own required quote surfaces, including configured tenor and batch checks where applicable. Revert the activation if any call fails, returns invalid data or returns a malformed shape; the revert will restore the old pointer. If wrapper relationships are registered, also walk the

I-04 Wrapper activation can commit downstream cycles

CONTINUED

bounded downstream graph and reject any candidate chain that reaches the activating wrapper. Do not rely only on recognizing current wrapper implementations, because another interface-compatible adapter can delegate back into the graph.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

ACKNOWLEDGED

Bucket removals mask invariant failures

CATEGORY Best Practices
LOCATION Utils.sol#L2200-L2299
REVIEW Remediation Review

DESCRIPTION

`removeSwapFromBucket()` uses clamp-to-zero logic when subtracting a removed swap's contribution from bucket accounting fields. This pattern is applied to `bucket.lpNotional`, `bucket.weightedEntryTime`, `bucket.weightedInverseIndex`, `bucket.weightedEntryIndex`, `bucket.weightedUtilFee`, `bucket.weightedRiskPremium`, `bucket.weightedFeeTime`, `bucket.weightedRateSq`, `bucket.weightedTimeSq`, `bucket.totalBuyerCollateral`, `bucket.totalPoolBacking`, and `bucket.memberCount`.

These clamps should be unreachable under correct bucket accounting. If a swap is actually present in the bucket, then the bucket must contain at least that swap's notional, collateral, and weighted aggregate contributions. If any subtraction would underflow, the protocol has encountered an invalid removal, duplicate removal, wrong bucket selection, or pre-existing bucket corruption.

Silently clamping these fields to zero makes the accounting path less strict. Instead of reverting before state is changed, `removeSwapFromBucket()` can continue after an invariant violation and still mutate other fields, including signed aggregates such as `bucket.weightedLpRate`, `bucket.weightedLnIndex`, and `bucket.weightedRateTime`. This can convert an impossible accounting state into partially corrupted bucket state.

RECOMMENDATION

Remove the clamp-to-zero subtraction pattern from `removeSwapFromBucket()` or replace it with explicit invariant checks.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-06**

ACKNOWLEDGED

Undocumented bucket convention parameter

CATEGORY Documentation
LOCATION Utils.sol#L1763-L1769
REVIEW Remediation Review

DESCRIPTION

`Utils.getBucketId()` accepts a `convention` parameter and forwards it to `Utils.rateBandIndex()`, but the `getBucketId()` NatSpec omits an `@param convention` entry. This makes the public helper's documentation incomplete, especially because `rateBandIndex()` currently ignores `convention` for ABI/caller stability.

RECOMMENDATION

Add an `@param convention` NatSpec entry explaining that the value is retained for caller/API stability and currently does not affect bucket selection.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-07

ACKNOWLEDGED

Expired rates veto strict bounds activation

CATEGORY	Oracle
LOCATION	packages/hardhat/contracts/oracles/BaseTimeLockedOracle.sol:906-921
REVIEW	Remediation Review

DESCRIPTION

Strict rate-bounds activation treats every initialized stored rate as relevant, even after `maxAgeSec` has made that rate unusable by every consumer. `_allStoredRatesWithinBounds` skips only `updatedAt == 0`; it never applies the read path's freshness rule. An out-of-bounds curve can therefore veto the documented graceful activation path indefinitely despite no longer producing a valid quote.

Governance must either obtain and post a replacement curve under the old policy before tightening, abandon the change, or use forced activation. Forced activation clears the curve and deliberately leaves dependent reads unavailable until another full signed report lands. The condition does not directly give an attacker funds or prevent the owner from using the explicit recovery path, so it is an operational safety-transition defect rather than an asset-loss vulnerability.

RECOMMENDATION

Ignore max-age-expired buckets when checking strict compatibility, using the same freshness predicate as consumer reads. Alternatively, require strict activation to atomically verify and install a fresh curve under the new bounds. Continue checking every initialized, consumer-valid side of `RiskRateOracle`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-08

ACKNOWLEDGED

Callback settles swap before wrapper event

CATEGORY	Events
LOCATION	packages/hardhat/contracts/SwapPositionWrapper.sol:115-129
REVIEW	Remediation Review

DESCRIPTION

`wrap()` transfers the swap into wrapper custody and publishes both wrapper mappings before calling `_safeMint`, but it emits `PositionWrapped` only after `_safeMint` returns. A contract recipient can use `onERC721Received` to call permissionless `SwapCore.makePayment()` for an expired position at that intermediate point. `SwapClosed` is then emitted while the wrapper relationship exists on-chain but before any wrapper-specific creation event has announced it.

An event processor that creates its wrapper record from `PositionWrapped` and applies `SwapClosed` only to already-known wrappers can discard the terminal event as unknown, then create an apparently active record when `PositionWrapped` arrives later in the same transaction. This does not corrupt on-chain custody, settlement, or redemption, but it can leave event-driven integrations with the opposite lifecycle state from `SwapCore`. The callback may also transfer the NFT before `PositionWrapped`, but that stale-owner facet is already documented separately in `AA_ALL/findings8/informational-positionwrapped-post-callback-owner.md`.

RECOMMENDATION

Emit a wrapper-creation event before invoking the ERC-721 receiver callback, then optionally emit a post-callback synchronization event containing the final owner and settlement state. Document that consumers must reconcile wrapper events with authoritative `SwapClosed` and ERC-721 `Transfer` logs.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-09

ACKNOWLEDGED

Aave adapters accept uncommittable projections

CATEGORY	Compatibility
LOCATION	packages/hardhat/contracts/oracles/cumulativeIndex/CumulativeIndexSupplyAave.sol:69-84; packages/hardhat/contracts/oracles/cumulativeIndex/CumulativeIndexBorrowAave.sol:32-38
REVIEW	Remediation Review

— DESCRIPTION

Aave projects its liquidity and variable-borrow indices as `uint256` view values, but a reserve checkpoint persists both indices atomically in `uint128` fields. Each Kairos adapter reads and validates only its selected index. The supply adapter can therefore report a representable normalized-income value as valid while the companion variable-borrow index already exceeds `uint128`; the borrow adapter has the symmetric failure when normalized variable debt is representable but normalized income is not. In either state, Aave cannot commit the reserve transition, yet Kairos can record and settle against the individually valid-looking projection.

This is distinct from the existing `AA_ALL/findings8/informational-aave-unpersistable-projected-indices.md`, where the selected adapter value itself exceeds its persistence bound. A selected-value-only range check, which that issue recommends, does not reject either companion-index variant. Reaching these states requires remote index growth near Aave's `uint128` boundary and would already freeze source-reserve mutations, so the practical risk is latent integration inconsistency rather than a present-day attack.

— RECOMMENDATION

Have both Aave adapters read normalized income and normalized variable debt and return invalid unless the complete projected reserve state is positive and atomically persistable. Include any other bounded values written by Aave's checkpoint transition, such as accrued-to-treasury headroom, in the same committability check.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-10

ACKNOWLEDGED

Dust deposit captures sole LP's vesting reserve

CATEGORY Unexpected Behavior
LOCATION packages/hardhat/contracts/SwapCore.sol:468-501
REVIEW Remediation Review

DESCRIPTION

`withdrawCollateral` protects an undripped early-exit reserve only when the withdrawal burns every share in the pool. If a sole LP tries to exit while a reserve is still live, the check `pool.totalShares == sharesToRedeem` reaches `E419` and blocks the withdrawal.

A second LP can defeat this protection by depositing a trivial amount before the withdrawal. The new shares make `pool.totalShares` larger than the incumbent's `sharesToRedeem`, so the reserve check is skipped. The incumbent can then burn its entire position at the reserve-adjusted price. Deposits mint at the undeducted price, so the attacker initially buys only a proportional dust claim. Once the incumbent burns all its shares, however, the attacker's shares become the only claim on the remaining reserve as it drips.

The current-code proof creates a real reserve through an early exit. A one-USDC deposit then lets the attacker capture more than 99% of the sole LP's withheld value. The attack requires only that the attacker deposit before the victim withdraws and it can cause a material loss of value already earned by the victim.

RECOMMENDATION

Do not protect the reserve solely through the global last-share check. Track each LP's reserve entitlement and pay it only to the shares that owned the pool when the reserve was created. As an immediate safeguard, reject an LP's complete position exit while that LP still owns an undripped reserve claim, even when other shares remain in the pool.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/POCs/packages/hardhat/test-foundry/zz_POCS/HelixSoleLpVestingReserveCapture.t.sol

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-11**

ACKNOWLEDGED

Seasoned sibling captures fresh LP profit

CATEGORY	Business logic errors
LOCATION	packages/hardhat/contracts/SwapCore.sol:433-501; packages/hardhat/contracts/SwapCore.sol:689-763
REVIEW	Remediation Review

— DESCRIPTION

LP profit vesting is tracked per address with one aggregate `entryPrice` and deadline, while value withheld when a fresh position exits remains ordinary pool collateral. A user can keep a seasoned dust position at a sibling address, create a fresh position that becomes profitable, and withdraw the fresh address while its cap binds. The withheld profit is not attached to that cohort; it immediately raises the value of all surviving shares, including the attacker's already-seasoned sibling, which can withdraw it without waiting for the fresh cohort's vest deadline.

This defeats the intended cooldown and transfers capped value from the fresh position to an attacker-controlled seasoned position. The current-code proof isolates the sibling path and shows the seasoned shares realizing the withheld amount immediately.

— RECOMMENDATION

Keep capped profit attached to the originating share lot or place it in a pool-wide reserve excluded from every LP's burn price until the originating deadline. Per-address aggregation must not allow a second address to launder the lock.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-12

ACKNOWLEDGED

Reserve clipping erases later settlement jumps

CATEGORY	Logical Error
LOCATION	packages/hardhat/contracts/lib/PoolInternalsLib.sol:89-120; packages/hardhat/contracts/SwapCore.sol:1514-1521
REVIEW	Remediation Review

DESCRIPTION

`_bookVesting` clamps each positive settlement jump to the pool collateral currently free after existing reserve. Any excess is discarded rather than recorded as deferred debt. If an earlier permissionless settlement or liquidation temporarily leaves the pool underwater, a later positive jump can therefore book little or nothing. Subsequent gains restore physical collateral, but the rejected amount is never re-withheld and becomes immediately withdrawable by LPs.

This makes final LP ownership depend on permissionless settlement order and allows a strategic LP to capture value that should remain vested. The two current-code tests show 161,615.492817 USDC of omitted vesting and about 53,871.830939 USDC of attacker excess in the ordering scenario.

RECOMMENDATION

Keep the physical reserve cap, but record rejected positive jumps as deferred vesting debt that does not reduce current NAV. On later pool gains, fund that debt into the live reserve before making the gain withdrawable. Alternatively define and enforce a loss haircut that consumes reserve entitlement consistently.

PROOF OF CONCEPT https://github.com/GuardianOrg/protocol-team1-1783698575429/blob/poc/underwater-vesting-ordering/packages/hardhat/test-foundry/zz_POCS/UnderwaterVestingOrdering.t.sol

RESOLUTION

Status: Acknowledged.

<https://github.com/kairosswap/protocol/pull/1132>

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.